

Architettura degli elaboratori

Sommario

Introduzione	1
Storia dei calcolatori	3
Organizzazione di un calcolatore	3
Porte logiche e circuiti combinatori	13
Mappa di Karnaugh	21
Rappresentazione dell'Informazione	23
Distanza di Hamming	29
ALU Hack	33
Circuiti Sequenziali.....	35
Microarchitettura.....	41
ISA HACK	50
C-instruction.....	52
A-instruction.....	53
Esercizi Assembly	58
Tutti gli esercizi Assembly Hack.....	62
Realizzazione del Computer Hack	64
Assemblatore.....	71
Altri tipi di ISA	75
Programmazione in C.....	84
Sistema Operativo	86
Paginazione	87
Virtual Machine.....	100
Simulazione d'esame	111

Introduzione

Durante il corso di Architettura andremo a studiare i principi alla base del funzionamento dei calcolatori, a partire dalle porte logiche NAND fino alla realizzazione completa (in un simulatore) di un calcolatore in grado di eseguire il videogioco pong.

Utilizzeremo la porta NAND perché essa è in primo luogo universale, ovvero ci permette, combinandola in vari modi con altre porte NAND, di realizzare tutte le logiche booleane (che

elaborano due valori: 0,1 - chiamati anche vero e falso); in secondo luogo la porta NAND è facile da realizzare utilizzando i transistor (presenti in tutte le odierne CPU).

Il progetto che andremo a realizzare, pong, è estremamente complesso; al termine esso sarà composto da centinaia, se non migliaia, di porte NAND, quindi per poterlo affrontare seguiremo il principio di *astrazione/implementazione*.

Il principio di astrazione/implementazione è uno dei concetti base dell'informatica. Esso si basa su una semplice operazione: ogni volta che vado a creare un componente che mi serve (implementazione), esso diventa un "mattoncino" che posso poi andare ad utilizzare, senza dovermi ricordare come l'ho costruito.

LIVELLO HARDWARE

Livello 0

È l'unico livello interamente fisico, su cui si trovano le porte logiche (XOR, NOT; AND ecc), i circuiti combinatori, i circuiti sequenziali (hanno una memoria).

Questi sono tutti fatti da porte NAND.

Livello 1: microarchitettura.

Questo livello governa il flusso di dati fra i vari componenti del livello logico digitale, può essere hardware, ma più comunemente è software.

Livello 2: livello ISA

Dà l'insieme di istruzioni eseguibili dalla microarchitettura.

È considerato il livello di interfaccia tra hardware e software

LIVELLO SOFTWARE

Livello3-4: Livello del sistema operativo

È molto semplice è strettamente correlato al processore. Utilizza il linguaggio Assembler, dà istruzioni riguardo a come accedere al processore ecc. Il S.O. aggiunge funzionalità al livello ISA (i programmi), ad esempio i servizi per gestire I/O, la memoria, la scheda di rete, ecc. Il linguaggio Assemblativo permette di programmare i livelli sottostanti (traducendo l'Assembly in linguaggio macchina). Il confine tra S.O. e Assembly è un po' sfumato, per questo si parla di livello ibrido.

Livello 5: linguaggio di programmazione ad alto livello



Storia dei calcolatori

Organizzazione di un calcolatore

Un calcolatore è un sistema composto da processori, memorie e dispositivi di input/output (I/O). Questa organizzazione (con l'unica differenza della presenza dei bus) è uguale a quella della macchina di Von Neumann, introdotto per gestire meglio il crescente numero di dispositivi I/O che si interfacciano al PC.

Organizzazione "bus oriented": Un BUS è un insieme di connessioni elettriche parallele utilizzato per trasportare informazioni da un componente all'altro. Generalmente si utilizzano due BUS: un BUS indirizzi, che viene usato dalla CPU per comunicare alla RAM quale indirizzo di memoria leggere, ed un BUS dati, che viene usato per lo scambio di informazioni.

Architettura di Von Neumann

- La grande innovazione che portò la macchina di Von Neumann fu utilizzare la memoria non solo per i dati ma anche per caricare i programmi. Quando fu proposta era rivoluzionaria: evitava complesse configurazioni con interruttori e cavi.
- Programmi e dati sono trasferiti entrambi attraverso il (sotto)bus dati. Il (sotto)bus indirizzi è utilizzato dalla CPU per indicare alla memoria le locazioni delle informazioni da trasferire

Parti della macchina di Von Neumann

La CPU

Esegue i programmi nella memoria centrale

È composta da:

- Control Unit (CU): legge e interpreta le istruzioni.
- ALU: esegue le operazioni (AND, OR, addizione, ...)

- Registri: memorizzano i risultati temporanei e le informazioni necessarie al funzionamento. Tipicamente sono indicati per nome, ma possono anche essere numerati.

Alcuni registri speciali

Sono presenti sia registri generici che registri dedicati; i principali sono

- Program Counter (PC): indica la prossima istruzione in memoria
- Instruction Register (IR): contiene l'istruzione che si sta per eseguire
- Memory Address Register (MAR): indirizzo della cella di memoria da usare nella prossima lettura/scrittura in MDR
- Memory Data Register (MDR): registro che accede al Bus dei Dati (sia in lettura che in scrittura), contiene sia le istruzioni appena lette dalla RAM (linea legge), sia i risultati di istruzioni appena eseguite e che devono essere scritti sulla RAM (linea scrive).
- Program Status Word (PSW): indica informazioni sull'ultima operazione eseguita (zero, overflow, ...)

Tipica esecuzione di un'istruzione

1. Il contenuto del PC viene copiato su MAR e attivazione linea leggi;
2. il contenuto in memoria all'indirizzo indicato da MAR viene scritto su MDR attraverso il bus dati ;
3. Il contenuto di MDR copiato su IR e viene decodificato;
4. l'istruzione passa in esecuzione sulla ALU;
5. se ci sono operandi da prelevare in memoria, si collocano in registri (usando come sopra MAR e MDR);
6. terminata l'esecuzione il risultato va su registro destinazione; se bisogna scrivere in memoria il valore calcolato si usano MAR / MDR attivando linea scrivi
7. Si ritorna al punto 1 dopo aver aggiornato il valore di PC

Il ciclo di esecuzione descritto nella slide precedente è conosciuto come ciclo “Fetch – Decode – Execute”:

- Caricamento (Fetch): acquisizione dalla memoria di un'istruzione del programma
- Decodifica (Decode): identificazione del tipo di operazione da eseguire
- Esecuzione (Execute): effettuazione delle operazioni corrispondenti all'istruzione

In particolare: i passi 1-2 corrispondono a Fetch, il passo 3 a Decode, i passi 4-5-6 ad Execute

CU

Come accennato prima, la Control Unit è la parte del processore che detta legge. Tutte le altre parti, quando non stanno eseguendo delle istruzioni, aspettano che la CU le dica cosa fare. Il modo più semplice di realizzare una CU è costruendo un circuito hardware con un insieme fisicamente fissato di istruzioni. Andando a inserire molte istruzioni, però, realizzare l'unità di controllo in questo modo diventerebbe troppo costoso, oltre che complesso; per ovviare al problema, solitamente si permette una (micro)programmazione del comportamento della CPU, si crea cioè il livello della microarchitettura. In questo modo il processore può:

- eseguire più istruzioni a partire dagli stessi componenti di base;

- cambiare il proprio comportamento, cambiando il microprogramma (patch di errori, aggiunta di nuove istruzioni, ecc).

ALU

L'Unità Aritmetica e Logica (ALU) è la parte del processore predisposta alle operazioni aritmetiche e logiche. Un processore può avere una o più ALU.

Il Data Path è la parte della CPU che comprende la ALU, i suoi input ed i suoi output (solitamente registri) ed è il percorso che fanno i dati e le istruzioni all'interno del processore.

Il flusso di dati, dai registri alla ALU e viceversa, avviene nell'arco del cosiddetto ciclo di clock, la cui durata è decisa appunto dal clock.

Clock

Il clock è un meccanismo che è stato introdotto per ovviare al fatto che i processori non sono macchine perfette.

Ogni operazione si svolge nell'arco di un ciclo di clock (o semplicemente clock). Anche se l'operazione e la scrittura del risultato terminano prima della fine del ciclo, la ALU o gli altri componenti non potranno leggere, scrivere o eseguire altre istruzioni prima del nuovo ciclo di clock.

In un processore ideale, ogni qual volta si passa da una situazione di non passaggio di corrente ad una in cui passa della corrente e viceversa (ricordiamo che il non-passaggio corrisponde a 0 e il passaggio ad 1), l'aumento e il calo di corrente sarebbero netti e i valori salvati (nei registri, oppure nella RAM, oppure in altri posti) potrebbero essere letti immediatamente.

In un ciclo di clock succede tutto: quando il segnale del clock passa da 0 a 5v (valore a caso per esemplificare) avviene il recupero dalla memoria; questo deve avvenire necessariamente in un ciclo, che è sufficientemente lungo per far avvenire il tutto. In diversi elaboratori gli stessi processi possono richiedere uno o più cicli; tutto è regolato dalla control unit.

La control unit riceve le istruzioni nei fili d'ingresso, le fa passare dalle porte logiche e manda i segnali ai registri. Il segnale 0 non viene considerato; il segnale 1 viene sovrascritto nei registri. Un circuito elettrico invia sempre un valore (0 oppure 1).

Quando la corrente varia da 0 a 5 volt, non fa uno stacco netto, ma oscilla fino a che non si avvicina maggiormente al valore 1. Il ciclo di clock serve a dare al segnale il tempo di stabilizzarsi, facendo in modo che quando le varie parti della CPU andranno a leggere i dati, leggeranno il valore giusto. Il valore intermedio tra 0 e 1 in genere non viene considerato; va comunque tenuto a mente perché a volte approssimando possono esserci degli errori.

La frequenza del clock è l'inverso del periodo di clock ($1/T$)

In base alla frequenza, capiamo quanto è veloce il calcolatore, ma non è un valore preciso e utile a confrontare diversi calcolatori perché non dice veloce a fare che cosa; è influente per il confronto solo a parità di architettura.

La velocità del processore dipende quindi in gran parte dalla durata del clock. Un clock solitamente dura pochi microsecondi (se non nanosecondi) e solitamente si parla di frequenza di clock, che indica quanti cicli vengono effettuati in un secondo. La durata del ciclo di clock corrisponde anche alla durata del ciclo di data path.

Dalla durata del data path possiamo risalire alla durata di un'istruzione ISA, che è uguale al prodotto di n per la durata del ciclo di path ($n * \text{durata del ciclo di data path}$), dove n varia da istruzione ad istruzione, a seconda della specifica architettura.

Uno degli aspetti principali dell'evoluzione dei computer è stato l'aumento della velocità di calcolo.

Se aumentiamo troppo la velocità del ciclo, i circuiti non fanno in tempo a stabilizzarsi prima della prossima istruzione e il calcolatore inizia a mettere valori a caso.

I costruttori fanno una stima, sapendo come è fatto il calcolatore, di quanto ci mettono a stabilizzarsi, e mettono un ciclo di clock leggermente più lungo per compensare le imprecisioni. Overclock → aumento eccessivo del ciclo di clock. Se rimango nell'intervallo in cui le frequenze sono stabili ho vinto, altrimenti il calcolatore inizia a dare i numeri. Se la CPU non ha impurità particolari sfrutto il margine di tranquillità messo dal costruttore per farlo funzionare; altrimenti devo tornare al clock precedente.

Un modo per migliorare la tecnologia è usare circuiti più veloci; altrimenti, se i circuiti rimangono quelli, posso usare il parallelismo, cioè far eseguire più istruzioni in parallelo al computer.

Esempi

1.

Registri

ADD %EAX, %EBX %EAX = %%EAX + %EBX

2345: 101110011...111100 → questa è l'istruzione da inviare contenuta nella cella 2345

PC: 2345 (indirizzo della cella di memoria)

PC → MAR → address bus

Per eseguire un'istruzione si mette l'impulso sul MAR, poi da lì si attiva il segnale sull'address bus. La memoria mette sul data bus il contenuto della cella (ad es cella 2345)

Dal data bus il segnale arriva al registro MDR. A questo punto la control unit decide cosa fare del registro MDR. La control unit fa in modo che il valore vada dall'MDR all'IR.

IR: 101110011...111100

Questa è la stessa procedura qualunque sia l'istruzione

La control unit dice alla ALU di fare, in questo caso, ADD.

Uso dei multiplexer → prendono tanti dati in ingresso (ad es a 64 bit) e ne fanno uscire uno solo. La Control Unit manda questi segnali all'ALU. L'ALU e i registri non sanno cosa sta succedendo è il multiplexer che ha 4 ingressi che decide che cosa far passare in base all'istruzione.

4 registri (ognuno a 64 bit)

%EAX -

%EBX - >>>>

%ECX -

%EDX -

L'uscita della ALU va a tutti i registri su cui posso scrivere: come faccio a capire su quale registro devo andare a scrivere? Ogni registro ha un ulteriore filo; solo il filo del registro corretto viene attivato dalla control unit.

Quindi, l'informazione parte dai registri e va alla ALU (che operazione fare) e ai multiplexer (che informazione far passare), e poi la CU decide su quale registro memorizzare l'output.

%EAX (5) + %EBX (3) → ADD → 8 → viene sovrascritto in %EAX

2.

Registro + valore in memoria del registro

ADD %EAX, [%EBX] %EAX = %EAX + il valore della cella di RAM di indirizzo %EBX

2346: 010110011...1011100

PC → MAR → address bus → MDR → IR

La IR capisce che deve recuperare il secondo operando dalla memoria.

Manda su MAR il valore di %EBX (locazione 3 in RAM con valore 82) mandando ai multiplexer indicazioni di mandare l'uscita di %EBX su MAR attivando la scrittura su MAR.

Attivo la linea leggi

La memoria si vede arrivare un 3 e manda il valore contenuto nella locazione 3 sul bus dati

82 arriva su MDR

La CU indica ai multiplexer di mandare il valore di %EAX al primo ingresso della ALU

La CU indica ad altri multiplexer di mandare il valore di MDR al secondo ingresso della ALU

La CU indica alla ALU di fare la somma

La CU indica a %EAX di memorizzare il risultato

La PSW azzerà il flag di ZERO → assumiamo che se l'ultima operazione ha dato risultato 0 il bit è 1, se ha dato risultato non zero il bit è 0

La CU dice a PC di prendere PC+1

3.

JZ 123 Jump if zero 123

È un comando che controlla il valore del flag ZERO della PSW. Se l'operazione precedente ha dato come risultato 0, la PSW marca con valore 1; di conseguenza il programma salta a una data posizione di memoria che dipende dall'istruzione successiva. Altrimenti, se l'operazione ha dato un risultato diverso da 0, la marca con 0 (=falso)

La CU controlla il valore del flag ZERO della PSW. La CU vede che il flag è zero, quindi l'operazione non ha dato zero.

2347: 01011100011...100000

PC: 2347

CISC & RISC

Alla fine degli anni '70, la microprogrammazione permise di realizzare elaboratori con istruzioni molto complesse: CISC - Complex Instruction Set Computer (CPU più complesse)

In contrapposizione a questa tendenza nacque l'idea di realizzare architetture con set di istruzioni più semplici: RISC – Reduced Instruction Set Computer → architetture molto complesse e grandi possono rimetterci a livello tecnico (ad esempio si surriscalda più facilmente, il sistema è impreciso a causa dell'overclock ecc.)

In CISC dà una singola istruzione molto complessa, in RISC la stessa istruzione è scomposta in più istruzioni più semplici. In RISC è più facile mettere le istruzioni in parallelo e riorganizzarle, perché sono più semplici.

A volte i processori hanno sia parti CISC che RISC per combinare i vantaggi di entrambe.

- Istruzioni più semplici possono essere eseguite più velocemente (con un ciclo di clock ridotto), possibilmente evitando la microprogrammazione
- Ecco perché il livello che abbiamo chiamato “microarchitettura” può essere implementato tramite software (microprogrammazione) oppure hardware

Pipelining

Un modo per migliorare le prestazioni di un processore è eseguire contemporaneamente più cicli FDE, usando per ognuno di essi parti diversi della CPU.

Il pipeline è come una catena di montaggio. I pezzi vengono assemblati in momenti diversi lungo la catena di montaggio, e solo alla fine della catena l’output è concluso. Abbiamo quindi non un unico circuito che prende le istruzioni in memoria, decodifica, prende gli operandi, calcola ALU e scrive il risultato; ma abbiamo tanti pezzetti che vanno a creare il risultato finale, e ognuna fa qualcosa.

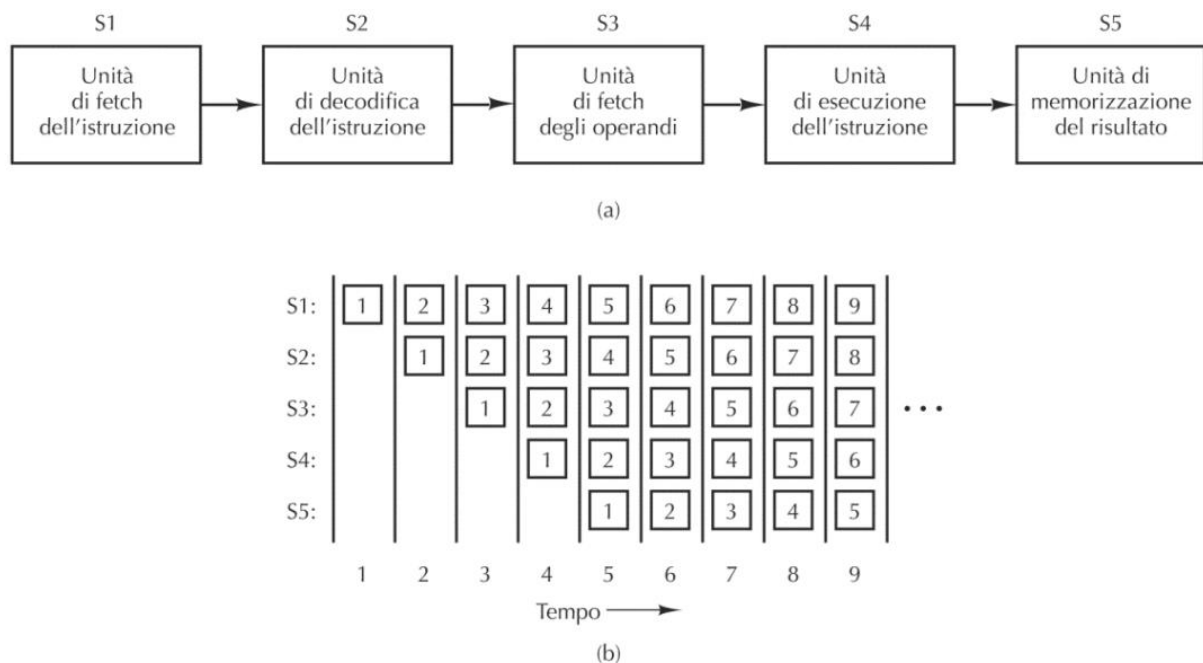
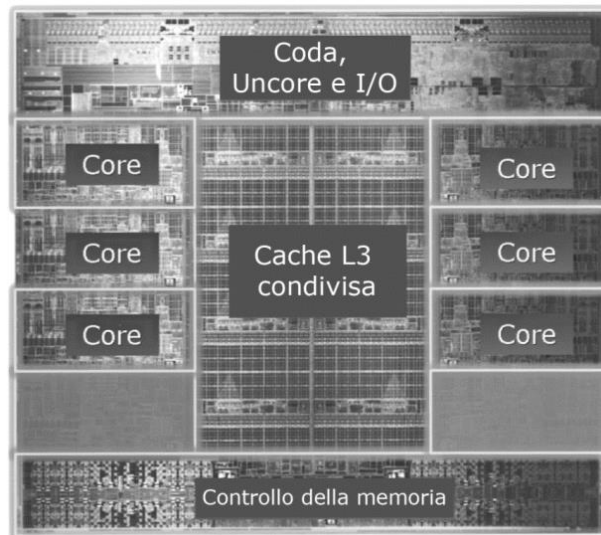


Figura 2.4 (a) Pipeline a cinque stadi. (b) Lo stato degli stadi in funzione del tempo. Sono mostrati nove cicli di clock.

La prima unità fa il fetch. La seconda unità fa decode; nel frattempo inizia già a fare il fetch della seconda istruzione perché il primo circuito è libero; quando la prima istruzione è al terzo stadio ho già iniziato la terza istruzione. Se tutto va a regime, in 9 cicli di clock finisco un’istruzione. Il ciclo di clock può essere più veloce di 3 o 4 volte.

Siccome ogni stadio mi fa una sola cosa, anche se vado più veloce posso avere dei pezzi hardware singoli. Ad esempio solo lo stadio S4 richiede la ALU; la CU sta solo in S2; ecc.

Il pipeline si trova in qualunque calcolatore di fascia medio-alta perché va veloce e costa poco. Il pipeline è limitato; non può avere più di sette stadi.



Calcolatori multicore → hanno più processori, i quali a loro volta hanno più pipeline

Il core contiene quasi tutte le cose che contiene il processore; non contiene però la memoria, o al massimo ha una piccola cache. Il multicore ha una memoria condivisa e condivide anche le unità input/output. Ogni core esegue un'istruzione; ad esempio se abbiamo 6 core, fanno 6 istruzioni alla volta.

Parallelismo

Array computer (SIMD)

Ho una sola Control Unit con più processori ALU che svolgono tutte la stessa funzione, ma con registri diversi (e quindi dati diversi). Ad esempio con un processore del genere posso fare molto velocemente lo schiarimento di un'immagine: posso fare la stessa operazione su ogni singolo pixel che però contengono tutti dati diversi (un colore diverso).

L'altro esempio è la modifica di figure 3D: ogni singolo triangolino che compone la figura può essere modificato con dati diversi. È utile anche per operazioni con i bitcoin o il machine learning.

I calcolatori che vengono venduti normalmente non sono di questo tipo.

La scheda grafica in realtà ha questo tipo di struttura, perché svolge molto bene il tipo di lavoro a livello grafico.

Multiprocessore (MIMD)

Alternativamente alla SIMD, se devo eseguire pochi calcoli ma molto complessi, posso replicare tutta la CPU e mettere tutti questi processori in parallelo, ognuno che si occupa del suo set di istruzioni sui suoi dati, permettendomi di fare più operazioni contemporaneamente.

Ogni processore può essere multicore o pipeline. Accedono a una shared memory e collaborano tutti insieme, non eseguono necessariamente le stesse istruzioni.

Posso anche implementare le memorie locali.

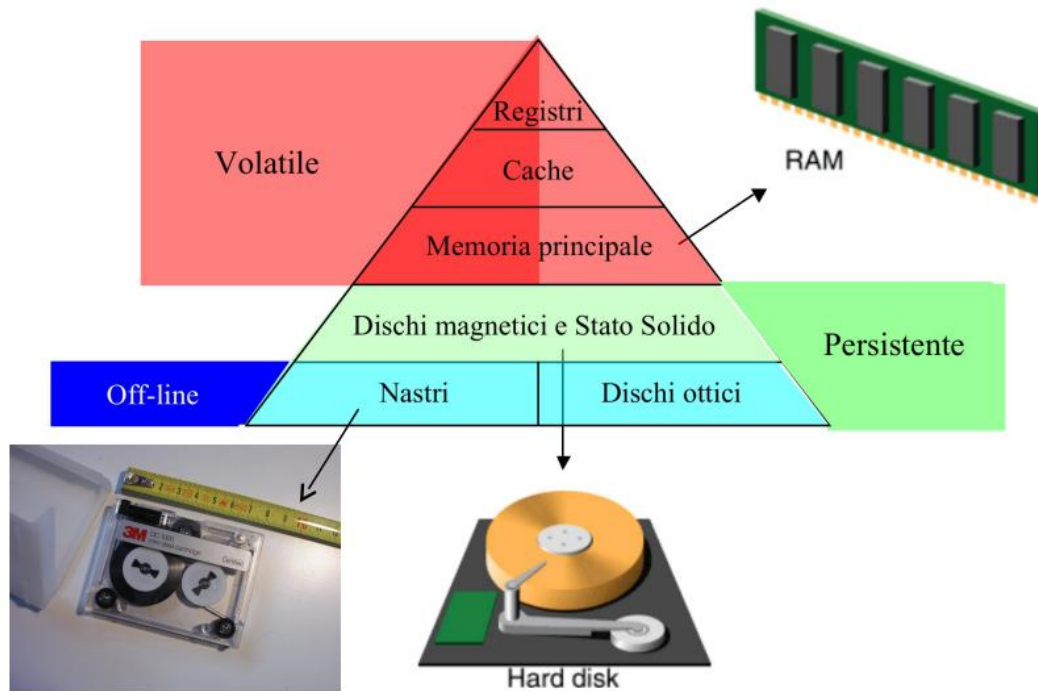
Se ho troppe CPU che fanno riferimento a una sola memoria, posso trovarmi di fronte a un collo di bottiglia che rallenta tutto;

Posso quindi implementare un **multicomputer**, la forma più estrema di parallelismo, che fa in modo che moltissime CPU siano collegate.

Più i problemi sono parallelizzabili (= eseguire diverse informazioni in contemporanea che non sono troppo collegate tra di loro) più questo meccanismo è utile.

Memorie

Le memorie sono le componenti del calcolatore in grado di memorizzare le informazioni: dati, programmi e risultati indispensabili per il suo funzionamento



Se ho un po' di registri e un po' di RAM posso dividere i dati tra le due e in questo modo posso combinare grandi quantità di dati all'interno della RAM con la velocità dei registri. I registri sono più piccolini della RAM ma più veloci.

La cache è una memoria intermedia tra registri e RAM che sta dentro al core. La differenza principale tra cache e registri è che i secondi vengono trattati a livello assembler. Chi programma in assembler deve avere presente quali sono i registri. Il programmatore assembler invece non vede la cache, la quale si ricorda i segnali che ha visto passare; se ha un segnale nuovo lo recupera dalla RAM.

Quando abbiamo dei dati elaborati che vogliamo salvare e mantenere, usiamo i dischi rigidi o ottici. Sono molto lenti, ma sono grandi memorie che funzionano anche offline.

Si usano vari tipi di memoria diversi per scopi diversi:

- Volatile: l'informazione rimane memorizzata fino a che il calcolatore è alimentato
- Persistente: l'informazione rimane memorizzata anche quando il calcolatore non è alimentato (spento)
- On-line: i dati sono sempre accessibili
- Off-line: il supporto deve essere montato per poter accedere ai dati
- ➔ Il costo di memorizzazione per byte cresce salendo la piramide
- ➔ La capacità (quantità di byte) cresce scendendo la piramide

Le memorie organizzano i dati in celle. Una cella è una sequenza di bit con un suo specifico indirizzo

La grandezza dei registri determina la grandezza dei dati che la CPU può gestire (8bit, 16bit, 32bit, ecc.), questi blocchi di byte vengono detti parole (in inglese **word**). Esistono 2 modi per memorizzare le “word” in celle di memoria di dimensione standard (1 byte):

- big endian (indirizzi assegnati da sx a dx, byte più significativo a indirizzo più basso)
- litte endian (opposto)

Nome	Significato
Bit	cifra binaria
Byte	8 bit
KByte (KB)	2^{10} ($\sim 10^3$) byte
MByte (MB)	2^{20} ($\sim 10^6$) byte
GByte (GB)	2^{30} ($\sim 10^9$) byte
TByte (TB)	2^{40} ($\sim 10^{12}$) byte

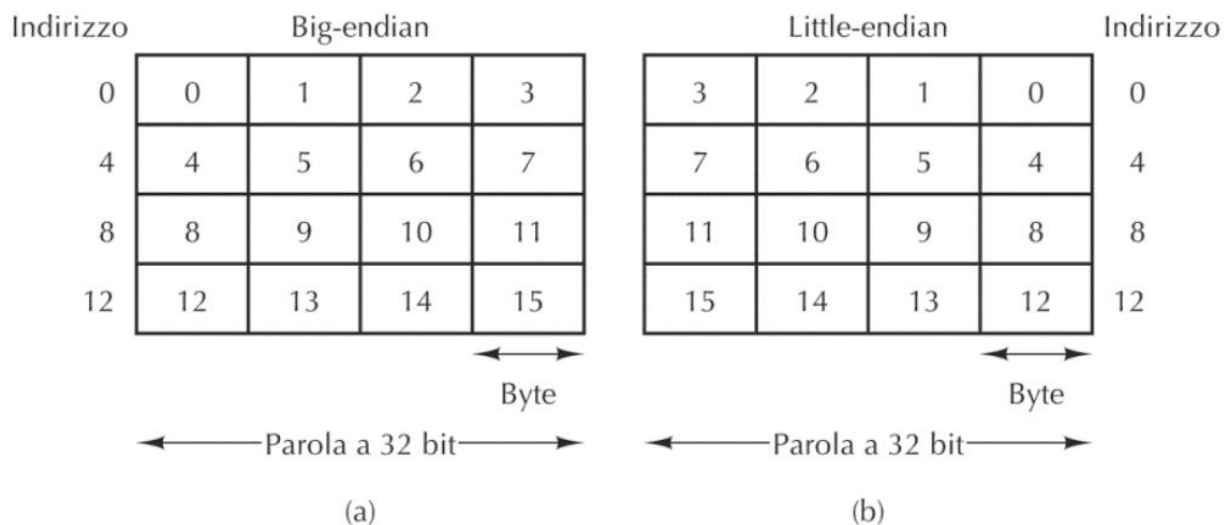


Figura 2.11 (a) Memoria big endian. (b) Memoria little endian.

La **memoria cache** è una memoria poco capiente ma molto veloce, che intercetta le chiamate che vanno dal processore alla memoria. Quando c'è una richiesta dal processore alla RAM la cache la intercetta e vede se conosce il contenuto della memoria a cui vuole arrivare il processore. Se la conosce, bene; altrimenti la manda alla RAM e tiene traccia della risposta. In questo caso siamo più lenti, perché abbiamo un tempo più piccolo per verificare se la cache conosce la risposta, e in più dobbiamo arrivare alla RAM; molto spesso la cache riesce comunque a velocizzare il processo. La maggior parte dei programmi esibiscono il cosiddetto principio di località: se accediamo a una cella di memoria è molto probabile che nel futuro immediato ci accederemo di nuovo.

Gli accessi alla RAM non sono fatti a caso: molto spesso accede o alla stessa locazione o a locazioni vicine. Quando scrivo un programma in genere lo eseguo in modo sequenziale dall'alto verso il basso. Questo è l'ideale per la cache: la CPU copia in cache anche i blocchi di locazioni contigue, aumentando la probabilità di trovare il dato necessario nei successivi accessi alla cache, evitando quindi di andare a prenderli dalla memoria.

Prestazioni di una cache

- c è il tempo di accesso alla memoria cache
- m è il tempo di accesso alla memoria centrale

- h (hit-ratio) probabilità di successo del recupero dei dati. In genere è un valore tra 0 e 1; più è vicino a 1, più è efficiente.

$c \ll m \rightarrow c$ è molto più piccolo di m

Tempo medio di accesso alla cache

$$t = hc + (1 - h)(c + m)$$

$$t = c + (1 - h)m$$

$hc \rightarrow$ caso di successo

$(1 - h)(c + m) \rightarrow$ caso di fallimento

Osserviamo che quando h si avvicina ad 1, il tempo medio t si avvicina a c , mentre quando h si avvicina a 0, t si avvicina a $c + m$.

Il termine cache si usa in tanti contesti: esistono altre cache in cui il funzionamento è simile. Ad esempio, la cache del disco usa meccanismi un po' diversi rispetto alla cache della RAM. La cache della RAM si pulisce da sola, mentre quella del disco ad esempio ogni tanto può essere pulita dall'utente. Anche online possiamo avere tante cache, ovvero sistemi che intercettano le richieste provando a rispondere più velocemente.

Dischi magnetici

Un hard disk (HD) è un dispositivo elettro-meccanico per la conservazione di informazioni sotto forma magnetica.

- Testina $\rightarrow$ legge il disco
- Traccia $\rightarrow$ sequenza circolare di bit
- Settore $\rightarrow$ porzione di traccia che contiene una quantità prefissata di bit (uguale per tutti i settori)

Il driver relativo da un lato comunica con il disco dicendogli cosa fare a livello fisico (gira per 1 secondo e mezzo... ecc), dall'altro parla con il S.O. che si occupa di recuperare i file giusti.

Memorie allo stato solido (SSD)

Somigliano alle chiavette USB o alle memorie ROM, ma fanno un lavoro più simile a quello del disco. È un circuito elettrico che contiene file, che ha lo stesso identico impiego dei dischi rigidi. È più resistente agli urti e alle vibrazioni

Dischi ottici

Il disco viene letto da un laser che riflette le zone piane e di avvallamenti che codificano zeri ed uni.

- CD-ROM: scritti dal costruttore (Read Only Memory)
- CD-R o CD Registrabili: scrivibili una sola volta
- CD-RW: ri-scrivibili più volte
- DVD: Digital Video Disk o Digital Versatile Disk
- Blu Ray: laser blu e non rosso (diversa frequenza)

Oltre alle memorie, i calcolatori includono anche altri componenti: tastiera, mouse, stampanti, schede di rete, monitor (il quale richiede una specifica scheda video che contiene un processore

detto GPU (Graphical Processing Unit). Lo schermo è una griglia di punti detti pixel, che possono assumere svariati colori. La GPU permette di evitare di mandare uno alla volta il colore di ogni pixel, ma manda informazioni compresse che velocizzano il processo. Le GPU sono oggetti molto sofisticati e potenti in questo tipo di calcolo.

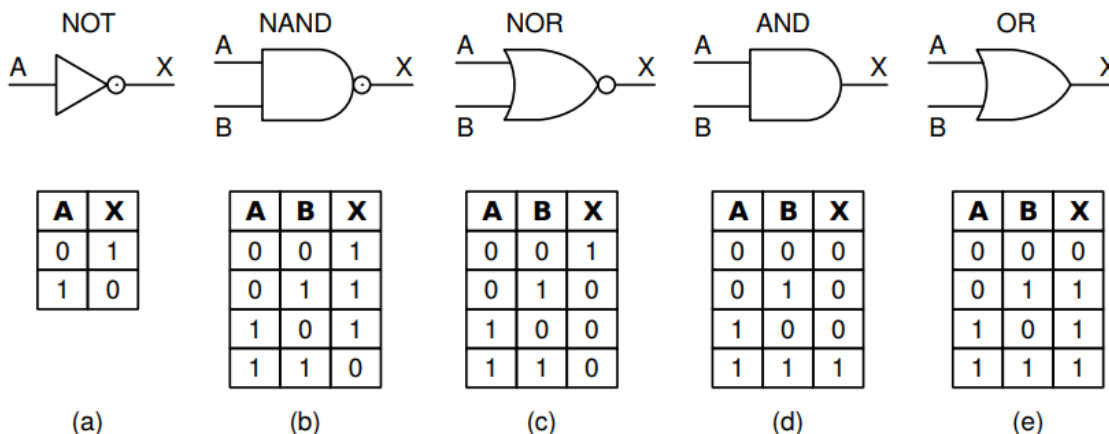
Dal punto di vista tecnico, abbiamo tanti dispositivi input/output, ognuno con un controller, che gestisce il passaggio dei dati. I dati viaggiando devono andare alla CPU. Posso mandarli direttamente al processore (direct memory access). Per parlare direttamente con la memoria il processore ha bisogno di un BUS, ma anche dalle periferiche per parlare con il processore o dalle periferiche per parlare con la memoria. I controller collegano la periferica al bus.

Porte logiche e circuiti combinatori

Alla base del funzionamento dei calcolatori ci sono le porte logiche e i circuiti combinatori. A partire dalla porta logica **NAND**, siamo in grado di creare i circuiti logici digitali alla base del funzionamento del computer: le porte NOT, NAND, NOR, AND e OR.

Un circuito combinatorio agisce solo con i valori dati istantaneamente e non conserva memoria (ad es. la ALU). I circuiti sequenziali invece tengono traccia delle operazioni avvenute nel passato (ad es. le memorie).

Tabelle di verità: sono uno dei modi possibili per descrivere un circuito.



Ho un certo numero di ingressi nelle porte, a seconda del circuito; una colonna per ogni ingresso e una per ogni uscita; e abbiamo una colonna che dà gli output possibile.

Algebra di Boole

Abbiamo degli operatori (AND, OR, NOT), delle costanti (solo 0 e 1) e delle variabili. Mi serve a scrivere relazioni tra le variabili e semplificare espressioni complicate.

OR $\rightarrow$ addizione

Ha come risultato 1 se almeno uno dei due parametri è 1.

- $0+0=0$;
- $0+1=1$;
- $1+0=1$;
- $1+1=1$

AND → Moltiplicazione

Ha come risultato 1 solo se entrambi solo 1; altrimenti viene 0

- $0 \cdot 0=0$;
- $0 \cdot 1=0$;
- $1 \cdot 0=0$;
- $1 \cdot 1=1$

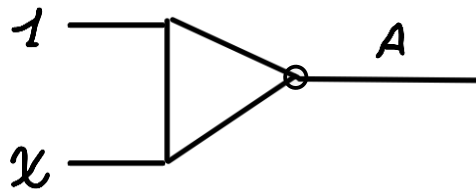
NOT → negazione

Ha come risultato l'opposto del parametro in input

- $\overline{0} = 1$
- $\overline{1} = 0$

Con quest'algebra possiamo ragionare sui circuiti digitali.

$A = \overline{1 + x}$ corrisponde al circuito



OR, AND sono operazioni sia commutative che associative

Se in un circuito ABC faccio l'AND tra AB e poi con C; oppure se lo faccio tra BC e poi con A, i circuiti sono equivalenti.

Proprietà dell'algebra di Boole

Name	AND form	OR form
Identity law	$1A = A$	$0 + A = A$
Null law	$0A = 0$	$1 + A = 1$
Idempotent law	$AA = A$	$A + A = A$
Inverse law	$A\bar{A} = 0$	$A + \bar{A} = 1$
Commutative law	$AB = BA$	$A + B = B + A$
Associative law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption law	$A(A + B) = A$	$A + AB = A$
De Morgan's law	$\overline{AB} = \bar{A} + \bar{B}$	$\overline{\bar{A} + \bar{B}} = \bar{A}\bar{B}$

***idempotent law → non ha corrispondenza nell'algebra degli interi.

Queste proprietà permettono di semplificare le forme e di conseguenza semplificare i circuiti.

La legge di De Morgan dice che se abbiamo una negazione di una AND, possiamo negare i singoli elementi e poi fare una OR.

La negazione della OR è come negare i singoli elementi e poi fare una AND.

Se ho una negazione in un'espressione complicata fatta di OR ed AND nego i singoli elementi e scambio gli OR con gli AND e viceversa.

Usando De Morgan possiamo

$$1A = A$$

$$\overline{(1A)} = \bar{A} \text{ [Negazione applicata ad una AND]}$$

Nego i singoli elementi e metto una OR

$$\bar{1} + \bar{A} = \bar{A}$$

$$0 + \bar{A} = \bar{A}$$

$$\bar{A} = B \text{ [definisco A negato come B]}$$

$$0 + B = B$$

Sono partito da un'equazione valida, usando le proprietà dell'algebra di Boole ho derivato un'equazione più semplice.

Le proprietà dell'algebra di Boole AND e OR sono simmetriche perché grazie a De Morgan posso passare da un lato all'altro

Esercizio. passare da inverse law OR $\rightarrow$ AND $[A + \bar{A} = 1 \rightarrow A\bar{A} = 0]$

$$A + \bar{A} = 1$$

$$\overline{(A + \bar{A})} = \bar{1}$$

$$\bar{A}A = 0 \text{ [commutativa} \rightarrow A\bar{A} = 0]$$

Dato un circuito, possiamo operarci sopra con l'algebra di Boole, in modo da renderlo più semplice. L'altro modo per rappresentare i circuiti è tramite le tabelle di verità; è comodo però solo con numeri di input relativamente piccoli

Tabella di verità

La tabella di verità ci consente di derivare un circuito in forma canonica.

Un letterale è

- una variabile (ad esempio A)
- oppure una variabile negata (ad esempio $\bar{A}$)

Un mintermine su n variabili è l'AND fra n letterali corrispondenti alle n variabili (ogni riga della tabella)

Ogni combinazione delle variabili di una funzione booleana ha un corrispondente mintermine (vero per quella specifica combinazione).

La variabile è negata se ho 0 nella tabella, è non negata se ho 1.

Alla fine devo fare l'OR dei mintermini.

A	B	C	F	$\overline{A} \overline{B} \overline{C}$
0	0	0	0	$\overline{A} \overline{B} \overline{C}$
0	0	1	0	$\overline{A} \overline{B} C$
0	1	0	1	$\overline{A} B \overline{C}$
0	1	1	0	$\overline{A} B C$
1	0	0	0	$A \overline{B} \overline{C}$
1	0	1	0	$A \overline{B} C$
1	1	0	1	$A B \overline{C}$
1	1	1	1	$A B C$

La tabella di verità è già data, e i risultati output sono già forniti ma la tabella non dà informazioni su come calcolarli. L'operazione interna ad ogni circuito è AND, ma non corrisponde ai risultati forniti nelle tabelle di verità.

Per fare l'OR di questi mintermini, posso considerare solo quelli che in uscita hanno valore 1 e otteniamo una funzione booleana che corrisponde esattamente alla funzione di verità.

$$\overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}\overline{C} + A\overline{B}C + AB\overline{C} + ABC$$

→ questa funzione corrisponde alla tabella di verità sopra.

Impareremo a costruire circuiti combinatori, cioè “scatole” che implementano funzioni booleane.

Ad esempio potremmo pensare ad un circuito che implementa la funzione:

And(a,b) = a AND b

In questo caso in particolare useremo la porta NAND che implementa la seguente funzione:

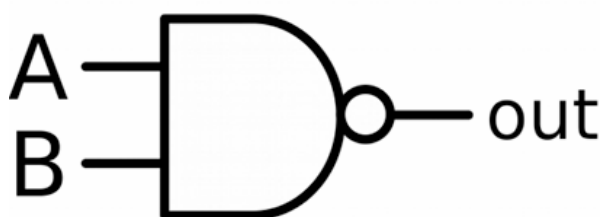
Nand(a,b) = NOT(AND(a,b))

Porta logica NAND

Interfaccia: And(a,b) = 1 se e solo se a=b=1

Per costruire questi circuiti useremo dei “mattoncini” di base (dette porte logiche)

La porta logica NAND ha la seguente rappresentazione grafica e la seguente tabella di verità



A	B	out
0	0	1
0	1	1
1	0	1
1	1	0

Si usa questa porta per due principali motivi:

- Facile da realizzare fisicamente tramite transistor
- Porta universale (si implementano AND, OR e NOT usando solo porte NAND)

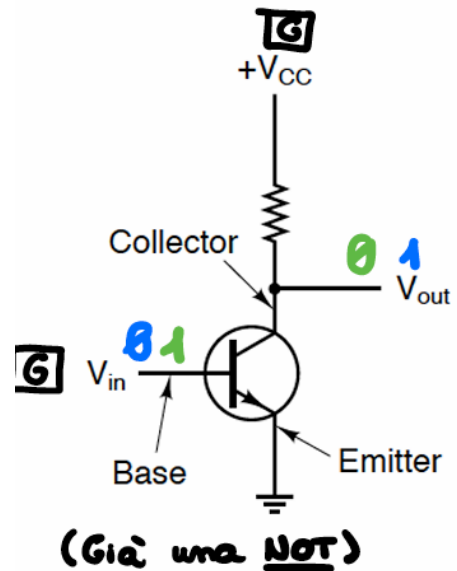
Teorema: universalità → con NAND si possono creare NOT, AND, OR

Transistor

Un transistor è un dispositivo a 3 connessioni: collettore, base, emettitore. Se non c'è tensione sulla base, si comporta come una resistenza infinita fra collettore ed emettitore. In presenza di tensione sulla base, si comporta da conduttore ideale tra collettore ed emettitore.

Il sistema in figura implementa un NOT. Se non diamo tensione a V_{in} , la tensione V_{cc} viene trasmessa a V_{out} . Se invece diamo tensione a V_{in} , non ci sarà tensione su V_{out} in quanto la tensione V_{cc} scarica a terra passando da collettore a emettitore.

Con uno o due transistor, possiamo creare le porte logiche principali. In questo modo posso inserire milioni di transistor in un circuito.



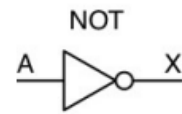
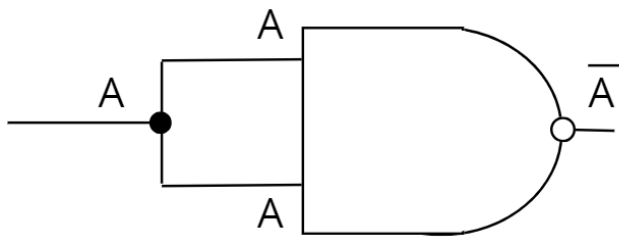
Assumendo che la presenza di tensione rappresenti la costante booleana 1, mentre la sua assenza la costante booleana 0, il seguente dispositivo implementa la porta logica NAND.

La tensione V_{cc} scarica a terra solo se entrambi gli input V_1 e V_2 sono sottoposti a tensione (in altri termini, l'output è uguale a 1 per ogni combinazione di input, ad esclusione del caso in cui entrambi gli input siano uguali a 1)

Anello universale

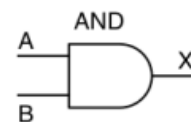
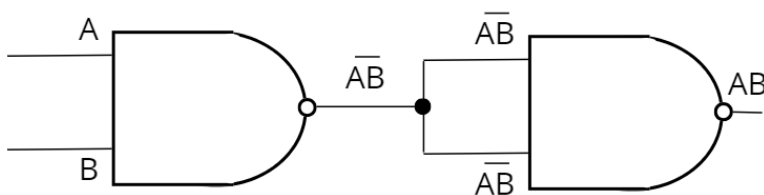
In base a come collego la NAND che è la struttura più semplice, posso implementare tutte le porte logiche.

Implementazione porta NOT



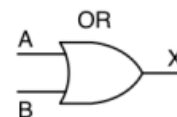
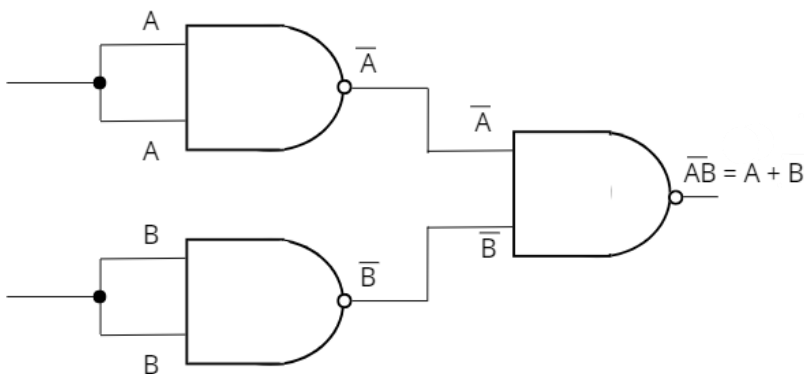
A	X
0	1
1	0

Implementazione porta AND



A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

Implementazione porta OR

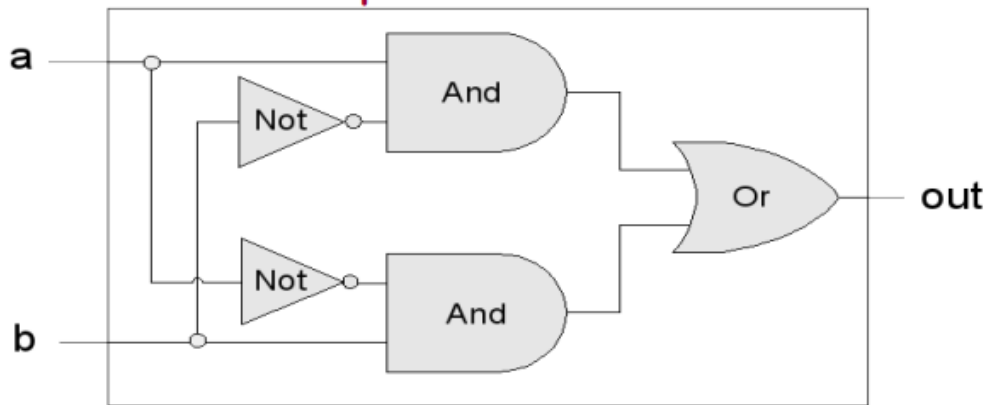


A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Per realizzare nuovi circuiti di solito si descrive prima l'interfaccia, ed in seguito si fornisce un'implementazione. Possono esistere diverse implementazioni per la medesima interfaccia, grazie alla proprietà commutativa e associativa dei circuiti; ad esempio mettere due NOT di fila non cambia il risultato.

Porta Xor: ha due ingressi e un'uscita. È identica ad una porta OR, con la differenza che con input 1 1 mi dà output 0. Quindi dà 1 solo se in input ha 0 1.

Implementazione



$$\text{Xor}(a,b) = \text{Or}(\text{And}(a,\text{Not}(b)), \text{And}(\text{Not}(a),b))$$

Con la NAND posso fare AND, OR, NOT, e da lì grazie alle funzioni booleane posso fare qualsiasi circuito.

La AND da sola non è una porta universale. Se abbiamo solo porte AND, e metto solo 0, non riuscirò mai ad avere un 1 → quindi con la AND non posso fare la NOT.

Multiplexer

Caso di multiplexer 2x1 → entrano due fili e ne esce uno.

La tabella di verità può essere fatta anche per il multiplexer.

I multiplexer più grandi hanno non necessariamente solo due ingressi ma hanno sempre un'uscita.

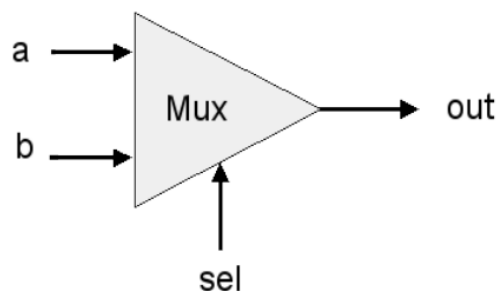
Se ho n ingressi di selezione, io posso scrivere 2^n valori.

Ogni ingresso è fatto da un certo numero di bit; nel caso più semplice hanno un bit solo in ingresso.

Sel è un numero 0 - 1 : se è 0 l'output prende il valore di a, se è 1 l'output prende il valore di b.

Sel è un valore di input come a, b.

a	b	sel	out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1



sel	out
0	a
1	b

Scriviamo la forma canonica, negando quelle che bisogna negare

Ci sono 4 porte OR a tre ingressi

$$\begin{aligned}
 &ABSel + \bar{A}BSel + \overline{ABSel} + AB\bar{Sel} \\
 &BSel(A + \bar{A}) + ASel(\bar{B} + B) \\
 &BSel \cdot 1 + \overline{ABSel} \cdot 1 \\
 &BSel + \overline{ABSel}
 \end{aligned}$$

PLA (Array logico programmabile)

Sono circuiti universali pre-costituiti che data una quantità di input prevedono porte AND per tutti i possibili mintermini, con uscite che entrano in OR

È possibile programmare qualsiasi funzione grazie ai fusibili che interrompono il collegamento fra gli AND dei mintermini che non interessano, e l'OR finale.

La PLA è già pre-fatta, io poi rompo i fusibili che non mi servono (ad esempio quando devo negare degli input) senza bisogno di saldare. Funziona solo per i circuiti in forma canonica.

A ogni tabella di verità corrisponde una sola forma canonica e viceversa.

Mappa di Karnaugh

Le mappe di Karnaugh sono un modo per rappresentare funzioni booleane simili alle tabelle di verità. Tale rappresentazione permette di costruire un circuito combinatorio "minimale"

AB		B	
A		1	1
		1	0

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

Se abbiamo tre variabili:

		AC			
B		00	01	11	10
	0	1	1	1	1
	1	0	0	1	0

è importante usare sempre l'ordine 00 01 11 10 (codice Gray, oppure codice a specchio)

Abbiamo le stesse informazioni della tabella di verità ma scritte in maniera diversa.

I rettangoli (ricoprimenti) della mappa di Karnaugh corrispondono a un raggruppamento corretto del circuito. I ricoprimenti devono coprire solo uni, devono prendere i rettangoli più grandi possibili, e non possono lasciare un 1 scoperto. Se i rettangoli si sovrappongono non è un problema. I rettangoli devono sempre essere potenze di 2: 2^0 , 2^1 , 2^2 ecc...

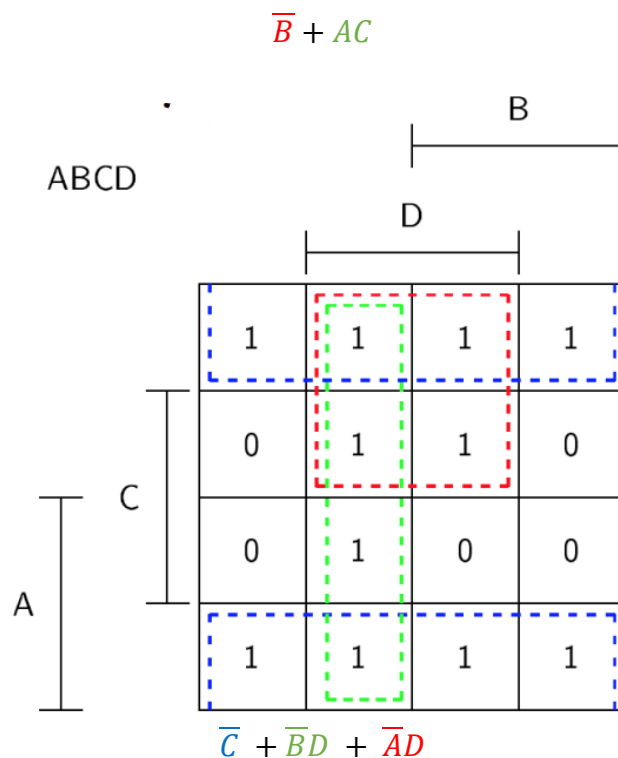
La forma minimale si scrive così: se la variabile ha sempre lo stesso valore, allora la segno come mintermine. Se ha valore 0 la segno come negata, altrimenti se ha valore 1 la segno come non negata. Se invece la variabile cambia il valore tra 0 e 1, la ignoro

Con la tabella di prima: B vale sempre 0 ($\overline{B}$), A ha qualunque valore ($A + \overline{A}$), C vale sempre 1 (C)

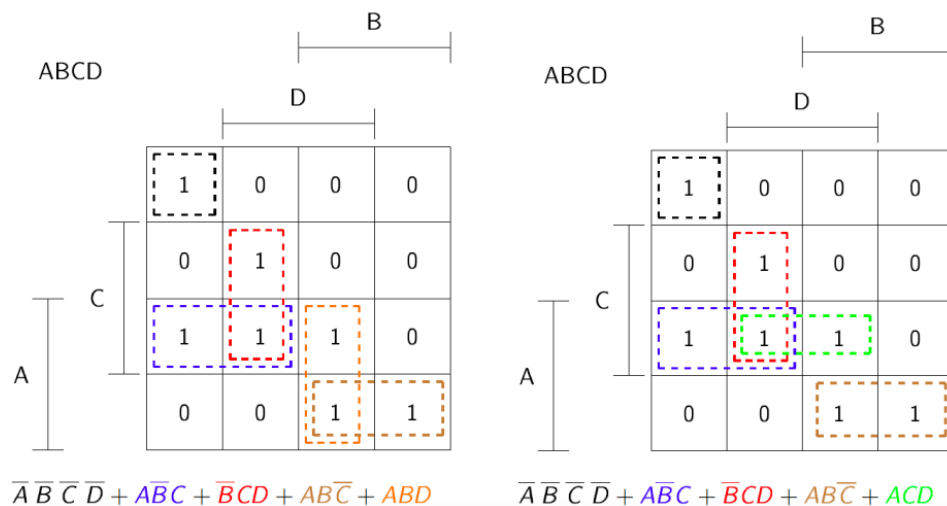
Nel rettangolino verde ho 2 mintermini da 3 letterali inclusi.

Nel rettangolo rosso, ho quattro righe della tabella di verità;

Dobbiamo fare insieme di rettangoli che coprano tutti gli uni e non gli zeri in modo da individuare la forma canonica. Il rettangolo più piccolo possibile è l'1 da solo.



La mappa di Karnaugh dà un aiuto grafico per capire quali sono i raccoglimenti giusti. Le segnalazioni a lato di A, B, C, e D indicano in quali aree della mappa il valore corrispondente è 1.



In questo caso, entrambe le schematizzazioni sono valide, e ci sono due forme minimali possibili diverse.

La forma canonica deriva dalla tabella di verità ed è unica, la forma minimale invece può anche non essere unica.

Rappresentazione dell'Informazione

In un calcolatore, ogni carattere, numero ecc. viene codificato in una sequenza binaria. Anche le informazioni più complesse chiaramente sono codificate in numeri binari. Esistono diversi modi per tradurre le informazioni in binario:

Il codice binario utilizza la potenza di due per assegnare un valore a ogni cifra binaria. Partendo da destra, la prima cifra (bit meno significativo) ha valore 1, la seconda ha valore 2, la terza ha valore 4, la quarta ha valore 8 e così via.

Esempi:

$$1010 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 1 \cdot 2^3 + 1 \cdot 2^1 = 8 + 2 = 10$$

$$1011 = 1 \cdot 2^3 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11$$

Per convertire un decimale in binario, basta fare la stessa cosa: moltiplicare le singole cifre per la potenza di 10 con esponente corrispondente alla posizione della cifra stessa e poi sommare il risultato.

Esempio: 122 in base 10 in binario viene $1 \cdot 10^2 + 2 \cdot 10^1 + 2 \cdot 10^0$

Oltre alla tecnica generale per il calcolo del numero espresso tramite una base b, è possibile usare la tecnica delle moltiplicazioni successive. Partendo da sinistra e da un accumulatore uguale a 0, per ogni cifra si moltiplica il valore dell'accumulatore per 2 e si aggiunge la cifra considerata.

Esempio: 111001 = 57

$$1 \rightarrow 1$$

$$1 \rightarrow 1 \cdot 2 + 1 = 3$$

$$1 \rightarrow 3 \cdot 2 + 1 = 7$$

$$0 \rightarrow 7 \cdot 2 + 0 = 14$$

$$0 \rightarrow 14 \cdot 2 + 0 = 28$$

$$1 \rightarrow 28 \cdot 2 + 1 = 57$$

Al contrario, partendo dal numero decimale:

$$57 \div 2 = 28 \text{ \% } 1$$

$$28 \div 2 = 14 \text{ \% } 0$$

$$14 \div 2 = 7 \text{ \% } 0$$

$$7 \div 2 = 3 \text{ \% } 1$$

$$3 \div 2 = 1 \text{ \% } 1$$

$$1 \div 2 = 0 \text{ \% } 1$$

Da notare che le cifre sono in ordine dalla meno alla più significativa. L'ordine è quindi 111001.

Lo stesso algoritmo di conversione funziona anche con altri sistemi numerici: ad esempio, ottale ed esadecimale

Ad esempio, ho il numero ottale 123 $(123)_8$.

Per convertirlo in un numero decimale seguo lo stesso procedimento.

$$(123)_8 = 1 \cdot 8^2 + 2 \cdot 8^1 + 3 \cdot 8^0$$

$$= 1 \cdot 64 + 2 \cdot 8 + 3 \cdot 1$$

$$= 64 + 16 + 3 = 83$$

L'algoritmo funziona anche con il sistema esadecimale. Basta ricordarsi che i sistemi con più di dieci cifre usano le lettere per indicare i numeri più grandi (A=10, B=11, C=12, D=13, E=14, F=15).

Ad esempio, ho il numero esadecimale 2A0.

Per convertirlo in un numero decimale seguo lo stesso procedimento usando 16 come base di origine

$$(2A0) = 2 \cdot 16^2 + A \cdot 16^1 + 0 \cdot 16^0$$

$$(2A0) = 2 \cdot 256 + A \cdot 16 + 0 \cdot 1$$

Sapendo che A=10 nel sistema esadecimale

$$(2A0) = 2 \cdot 256 + 10 \cdot 16 + 0 \cdot 1$$

$$(2A0) = 512 + 160 + 0 = 672$$

Rappresentazione di numeri con la virgola

L'usuale tecnica di rappresentazione dei numeri con virgola fissa non è sempre efficace.

Ad esempio, potrebbe essere necessaria una notazione scientifica per numeri molto grandi o molto piccoli.

Per questo motivo nei calcolatori si usa la codifica a virgola mobile.

$$n = f \cdot 10^e$$

f viene detto frazione o mantissa, e viene detto esponente

Immaginiamo di usare un'ipotetica codifica che usa per la frazione un numero a tre cifre uguale a 0 oppure compreso tra 0,001 e 0,999; per esponente un numero con aggiunta di segno compreso tra 0 e 99.

Riusciamo a rappresentare i numeri negli intervalli 2 e 6

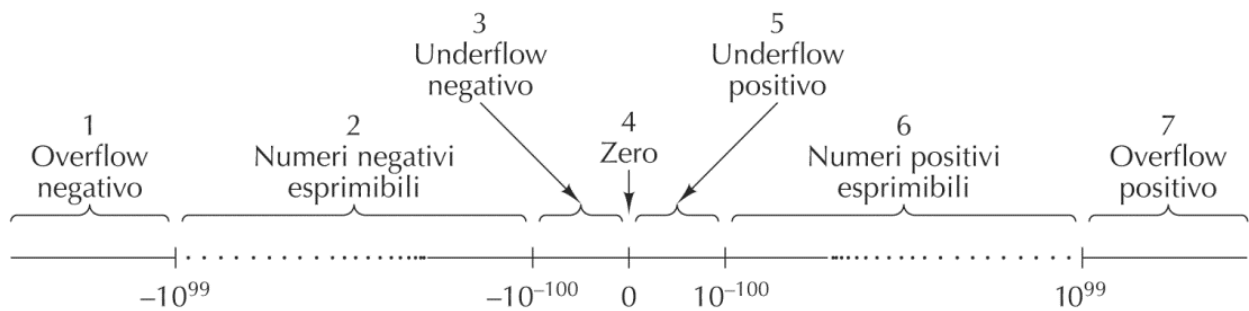


Figura B.1 La rappresentazione *R* ripartisce la retta reale in sette regioni.

Overflow positivo e negativo: quando i numeri sono troppo grandi oppure troppo piccoli è necessario approssimare. Questa approssimazione spesso è molto piccola ed influente.

L'infinità di numeri reali non può essere rappresentata con precisione dal calcolatore; non posso evitare degli errori di approssimazione.

Nei calcolatori di solito non si considera base 10, ma base 2, 4, 8 o 16.

Si usa una frazione minore di 1.

Inoltre, si tende a normalizzare la frazione:

Frazione normalizzata

- La cifra più significativa non può essere uguale a 0.
- 3,14 diventa $3,14 \cdot 10^0$

La normalizzazione serve a non avere problemi di confronti.

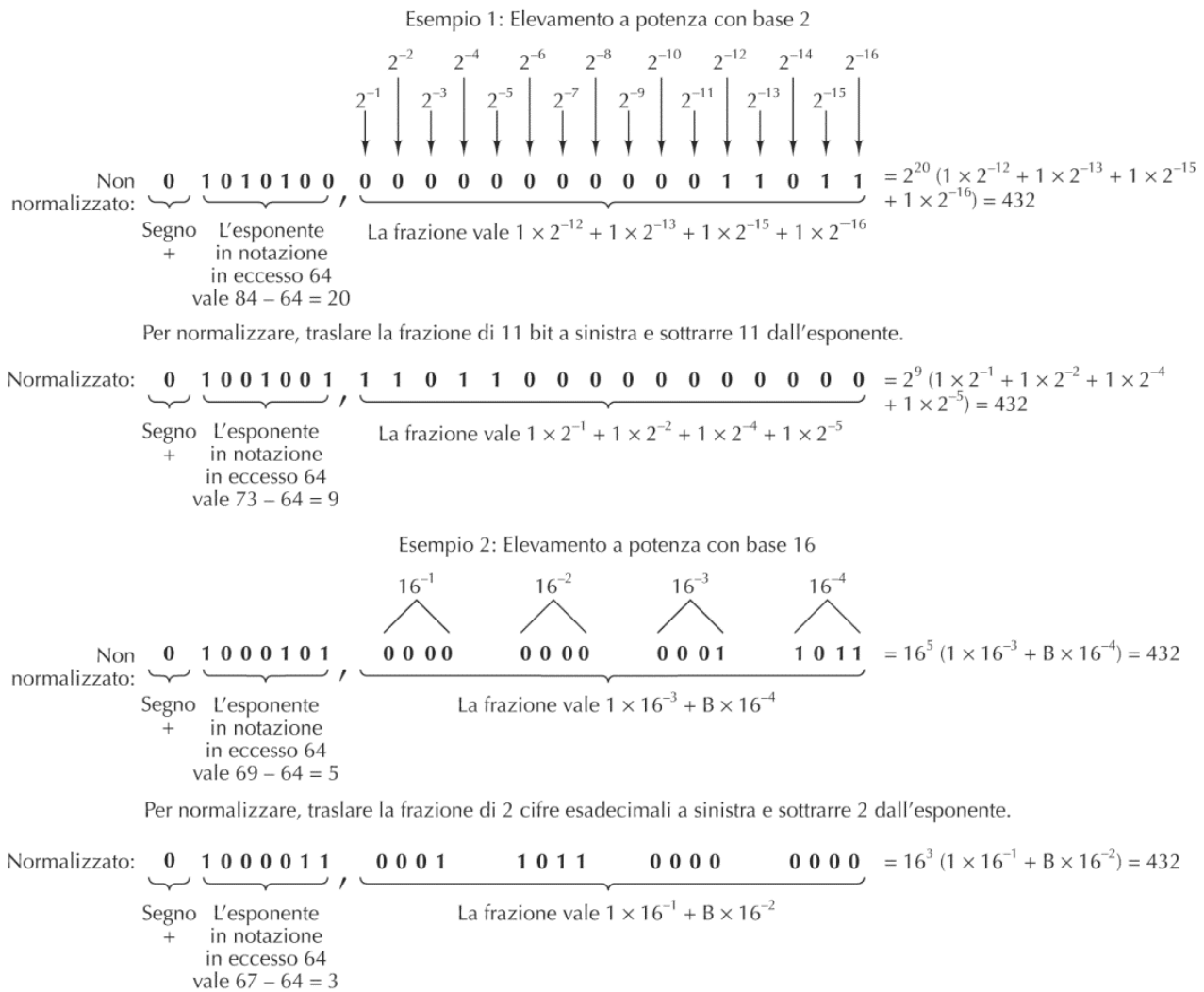


Figura B.3 Esempi di numeri in virgola mobile normalizzati.

Note → 1010100 in decimale è 84.

Un altro modo per rappresentare i numeri con la virgola è quello di usare il metodo delle moltiplicazioni, ovvero moltiplicando il numero decimale considerato e prendendo come cifra binaria (per ogni moltiplicazione eseguita) il valore dell'intero, se poi esso equivale a 1, si sottrae proprio 1 e si continua fino a quando non si arriva ad avere 0 come risultato finale. Può capitare che si arrivi ad ottenere numeri periodici, in questo caso basta considerare tanti bit quanti sono quelli da rappresentare, ed eventualmente effettuare opportuni arrotondamenti. Inoltre, bisogna calcolare l'esponente del numero: partendo dalla cifra a sinistra si normalizza il numero, successivamente bisogna calcolare il relativo numero in binario e porlo successivamente al numero.

I float e i double fanno queste scelte: dicono quale standard selezionare. Il più usato è IEEE 74, che definisce diversi formati inclusi single (BINARY32) e double (BINARY64).

Sono codifiche in binario, che prevedono bit di segno, bit di esponente e bit di mantissa.

Nel caso in cui ci sia mantissa (=frazione) 0 e esponente non 0, restituisce il valore $+\infty$.

NaN = not a number, codifica speciale

Una codifica per caratteri largamente diffusa è la codifica ASCII: American Standard Code for Information Interchange. Usa 7 bit per i principali simboli alfabetici anglosassoni e per alcuni

caratteri speciali (necessari in passato nei terminali/telescriventi). Ci sono le varie cifre, lettere minuscole e maiuscole. Con 7 bit possiamo fare 128 simboli che per l'alfabeto anglosassone sono abbastanza. Per codificare altri alfabeti, si rende necessaria una codifica che usa una quantità superiore di bit.

Se sommo in un programma ad un carattere maiuscolo in input la distanza tra una lettera maiuscola e una lettera minuscola (ad esempio 'a' e 'A', ma anche 'b' e 'B' ecc) allora stamperò la corrispondente lettera minuscola.

A tal scopo è nata la codifica UNICODE.

- Usa 16 bit
- Compatibile con ASCII (se si mettono a 0 i primi 9 bit i restanti 7 possono essere usati come in ASCII)
- Si usano intervalli contigui di codici per i diversi alfabeti (es. latino, greco, cirillico, armeno, ebraico,...)
- Ci sono codici non ancora assegnati per poter rappresentare in futuro caratteri al momento non considerati

UNICODE ha alcuni limiti:

- Usa sempre 16 bit anche per caratteri ASCII per i quali ne sarebbero sufficienti 7
- È oramai vicino all'esaurimento dei possibili codici

UTF-8 Unicode Transformation Format può dinamicamente occupare da 1 a 4 byte a seconda dell'informazione da codificare

- I bit iniziali indicano il formato specifico e la quantità di bit utilizzati
- Se il bit iniziale è 0, i restanti 7 contengono una codifica ASCII
- Altre codifiche esistono (UTF-16, ...)

Indipendentemente dal tipo di dati memorizzati, occasionalmente le memorie sono soggette ad errori sia durante le operazioni di lettura che durante le operazioni di scrittura. Analogamente ci possono essere errori trasmettendo i dati. Per proteggersi, alcune memorie utilizzano dei codici di rilevazione (ed in alcuni casi anche correzione) di errori. Se una parola consiste di m bit, si aggiungono r bit di controllo ottenendo una "parola di codice" a $n=m+r$ bit.

Tecniche di codifica dei numeri binari negativi

- modulo e segno = si usa il bit più a sinistra come segno (0 +, 1 -)
- complemento a 1 = il bit più a sinistra indica il segno, ma se il numero è negativo il modulo viene complementato (gli 0 diventano 1 e viceversa)

Massimo range esprimibile a 4 bit: da -7 a +7.

- Esempio: 00000110 = 6, 11111001 = -6.

Anche in questo caso, purtroppo, non riusciamo a fare a meno di rappresentare due volte lo 0.

- complemento a 2 = come per il complemento a 1, ma se il numero è negativo dopo il complemento si aggiunge 1

Massimo range esprimibile a 4 bit: da -8 a +7.

Simile al complemento a 1, ma dopo il complemento sommo 1.

- Esempio: 00000110 = 6, 11111010 = -6.

Perché si usa il modulo a 2 rispetto al modulo a 1?

Usando il modulo a 2 il numero binario che ottengo, se letto come intero positivo, rappresenta esattamente il numero massimo esprimibile a 8 bit a cui sottraggo il numero negativo che ho rappresentato.

- Esempio: Se $11111010 = -6$, posso leggerlo, ignorando la convenzione del complemento a due, come $11111010 = 250$, che è esattamente il risultato di 256 (in decimale 11111111), a cui sottraggo 6, il numero che ho rappresentato.

Questa proprietà torna molto utile per fare le sottrazioni, semplificando di molto il funzionamento della ALU.

Numeri che possono essere rappresentati

con k bit si possono rappresentare al più 2 numeri diversi -- 2

- Se vengono presi in considerazione solo i numeri naturali (non negativi) con k bit si rappresentano i numeri nell'intervallo $[0 \dots 2^k]$
- se si usano modulo e segno, o complemento a 1, vengono rappresentati i numeri nell'intervallo $[-2^{k-1} + 1 \dots 2^{k-1} - 1]$ → esistono due codifiche per lo 0
- in complemento a 2 possono essere rappresentati i numeri nell'intervallo $[-2^{k-1} \dots 2^{k-1} - 1]$

Codifica "in eccesso" = permette di rappresentare i numeri nell'intervallo $[-2^{k-1} \dots 2^{k-1} - 1]$

sommando 2^k alla rappresentazione binaria del numero stesso, spostando così l'intervallo ai numeri non negativi $[0 \dots 2^{k-1} - 1]$

- la decodifica si ottiene applicando la decodifica standard e poi sottraendo 2 al numero ottenuto

È un altro modo di codificare i numeri negativi, seppur meno efficace del complemento a due.

Un po' come si fa per i gradi Kelvin, si utilizza un offset, in questo modo la sequenza 00000000 non rappresenta più lo 0 ma il numero -128. In sostanza dopo aver fatto il passaggio da binario a decimale si sottrae 2^{k-1} al numero ottenuto.

- Numero in decimale -23, 17
- Numero in binario (8 bit) 00010111, 00010001
- Modulo e segno 10010111, 00010001
- Complemento a 1 11101000, 00010001
- Complemento a 2 11101001, 00010001
- Codifica "in eccesso" $2 - 23 = 128 - 23 = 105$ 01101001, $2 + 17 = 128 + 17 = 145$ 10010001

Somma binaria

- standard = basata sulla somma e sul riporto
- in complemento a 1 = basata sulla somma e sul riporto, dove il riporto finale viene sommato
- in complemento a 2 = basata sulla somma e sul riporto, dove il riporto finale viene scartato

Overflow = situazione che si verifica quando viene superata la capacità massima del registro aritmetico di un calcolatore elettronico. Non si verifica se gli addendi hanno segno opposto; si verifica se gli addendi hanno lo stesso segno, ma il risultato ha un segno diverso.

Esempi:

- $01111111 + 00000001 = 10000000$ (overflow)
- $10000000 + 11111111 = 01111111$ in complemento a 2 (overflow)

- $10000000 + 11111111 = 10000000$ in complemento a 1 (no overflow in quanto $11111111 = 0$)

<u>Decimale</u>	<u>In complemento a 1</u>	<u>In complemento a 2</u>
$\begin{array}{r} 10 + \\ (-3) = \\ \hline +7 \end{array}$	$\begin{array}{r} 00001010 + \\ 11111100 = \\ \hline 1 \ 00000110 \\ \text{riporto } 1 \rightarrow \\ \hline 00000111 \end{array}$	$\begin{array}{r} 00001010 + \\ 11111101 = \\ \hline 1 \ 00000111 \\ \text{scartato} \downarrow \\ \hline \end{array}$

Codici correttori

Indipendentemente dal tipo di dati memorizzati, occasionalmente le memorie sono soggette ad errori sia durante le operazioni di lettura che durante le operazioni di scrittura. Analogamente ci possono essere errori trasmettendo i dati. Per proteggersi, alcune memorie utilizzano dei codici di rilevazione (ed in alcuni casi anche correzione) di errori.

Se una parola consiste di m bit, si aggiungono r bit di controllo ottenendo una “parola di codice” a $n = m + r$ bit

Un codice è un meccanismo atto a determinare gli r bit di controllo relativi ad ogni parola di m bit.

Distanza di Hamming

La distanza di Hamming tra due sequenze di bit è il numero di bit rispetto ai quali le due parole differiscono: 001101 e 011100 hanno distanza 2 (differiscono per 2 bit). La distanza di Hamming di un codice è la minima distanza di Hamming tra “parole di codice” (cioè le parole corrette che non hanno subito errori)

Regola generale:

- Per rilevare d bit errati è necessario un codice con distanza di Hamming maggiore o uguale a $d+1$
- Per correggere d bit errati è necessario un codice con distanza di Hamming maggiore o uguale a $2d+1$

Uno dei codici più semplici è il cosiddetto “bit di parità”.

- un unico bit di controllo ($r=1$), che è scelto in modo che il numero di bit “1” nella “parola di codice” sia pari
- Il codice ha distanza di Hamming uguale a 2

Il codice può essere usato per rilevare singoli bit errati: basta controllare se i bit “1” nella parola sono pari o no.

Inventiamo un nuovo codice con le seguenti “parole di codice” di lunghezza 10:

0000000000 – 0000011111 – 1111100000 – 1111111111

➔ distanza di Hamming uguale a 5 ed è possibile correggere fino a 2 bit errati

Supponiamo che un codice con m bit di dati e r bit di controllo sia in grado di correggere tutti i possibili errori su singolo bit. Ciascuna delle 2^m "parole di codice" necessita di $n+1$ parole ad essa dedicate (con $n=m+r$):

- 1 per la "parola codice" corretta
- n per i possibili errori di un solo bit

Dato che il numero totale di combinazioni di bit è 2^n deve valere $(n + 1)2^m \leq 2^{m+r}$ da cui deriva

$$m + r + 1 \leq 2^r$$

Il codice di Hamming riesce a correggere tutti i possibili errori su singolo bit usando proprio il numero minimo di bit di controllo r che soddisfa la disequazione di cui sopra.

1. Ad m bit si aggiungono r bit di controllo, con r scelto in modo tale da essere il numero minimo per cui

$$m + r + 1 \leq 2^r$$

2. Si identifica la posizione di un bit nella "parola di codice" usando i numeri binari tra 1 e $m+r$
3. I bit di controllo vengono messi in posizioni i identificate dalle potenze di 2 (posizioni 1, 2, 4, 8, ..., ovvero le posizioni le cui codifiche in binario contengono una sola cifra uguale a "1"), nelle restanti posizioni viene messo il numero da codificare
4. A questo punto, il bit di controllo nella posizione i conterà i bit corrispondenti a "1" partendo dalla posizione in cui si trova, controllando in i posizioni consecutive, saltando i posizioni, e così via fin quando non avrà finito il numero, passando poi al successivo bit di controllo. Ad esempio, con una "parola di codice" da 16 bit, i bit di controllo vanno in posizione 1, 2, 4, 8, 16 e i cinque bit di controllo conterranno rispettivamente gli uni nelle posizioni:
 - 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21
 - 2, 3, 6, 7, 10, 11, 14, 15, 18, 19
 - 4, 5, 6, 7, 12, 13, 14, 15, 20, 21
 - 8, 9, 10, 11, 12, 13, 14, 15
 - 16, 17, 18, 19, 20, 21

Ovviamente il controllo non si effettua sui bit di posizione corrispondenti ai bit di controllo, in quanto essi saranno vuoti fin quando non avranno contato i bit interessati.

Una volta contati i bit, il bit di controllo assumerà valore "1" se il numero di bit contati è dispari, altrimenti assumerà il valore di "0".

$m = 16$

$r = 5$ ($(16 + 5 + 1) \leq 32$ mentre $(16 + 4 + 1) > 16$)

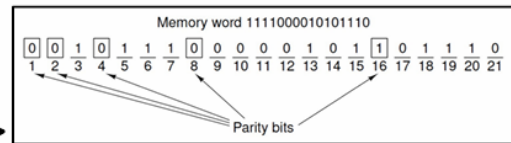
i bit di controllo vanno in posizione 1, 2, 4, 8, 16

calcolo dei gruppi a cui appartiene il bit in posizione j

- ☐ la posizione j appartiene ai gruppi corrispondenti ai bit = 1 nella codifica binaria di j

☐ in caso di errore basta guardare i gruppi che contengono l'errore controllando i bit di parità

la posizione dell'errore è quindi identificata dal numero binario che si ottiene mettendo a 1 solo i bit corrispondenti ai gruppi con l'errore



1	00001
2	00010
3	00011
4	00100
5	00101
6	00110
7	00111
8	01000
9	01001
10	01010
11	01011
12	01100
13	01101
14	01110
15	01111
16	10000
17	10001
18	10010
19	10011
20	10100
21	10101

Esempio da ChatGPT

Supponiamo di voler trasmettere il messaggio di 4 bit 1011.

1. Determinazione dei bit di parità

Per un messaggio di 4 bit ($m = 4$), servono 3 bit di parità ($r = 3$) per soddisfare la condizione

$$m + r + 1 \leq 2^r$$

Le posizioni dei bit di parità sono quelle che corrispondono alle potenze di 2: **1, 2 e 4**.

Quindi la nostra sequenza finale avrà una lunghezza di $m+r= 4+3 =7$ bit, organizzati come segue:

Posizione: 1 2 3 4 5 6 7
 Bit: P1 P2 D1 P4 D2 D3 D4

Dove:

- **P1, P2 e P4** sono i bit di parità.
- **D1, D2, D3 e D4** sono i bit di dati del messaggio 1011.

2. Inserimento dei bit di dati

Inseriamo il messaggio 1011 nelle posizioni corrispondenti ai bit di dati:

Posizione: 1 2 3 4 5 6 7
 Bit: P1 P2 1 P4 0 1 1

3. Calcolo dei bit di parità

Ora calcoliamo il valore dei bit di parità.

- **P1** controlla le posizioni 1, 3, 5, 7 (quelle che hanno il primo bit nella posizione binaria uguale a 1):
 - Bit in posizione 3 = 1
 - Bit in posizione 5 = 0
 - Bit in posizione 7 = 1
 - Conto il numero di "1" = $1 + 0 + 1 = 2$ (pari), quindi $P1 = 0$
- **P2** controlla le posizioni 2, 3, 6, 7 (quelle che hanno il secondo bit nella posizione binaria uguale a 1):
 - Bit in posizione 3 = 1
 - Bit in posizione 6 = 1
 - Bit in posizione 7 = 1
 - Conto il numero di "1" = $1 + 1 + 1 = 3$ (dispari), quindi $P2 = 1$
- **P4** controlla le posizioni 4, 5, 6, 7 (quelle che hanno il terzo bit nella posizione binaria uguale a 1):
 - Bit in posizione 5 = 0
 - Bit in posizione 6 = 1
 - Bit in posizione 7 = 1
 - Conto il numero di "1" = $0 + 1 + 1 = 2$ (pari), quindi $P4 = 0$

Ora possiamo inserire i bit di parità:

```
Posizione: 1 2 3 4 5 6 7
Bit:       0 1 1 0 0 1 1
```

Il messaggio codificato è quindi: **0110011**

4. Rilevamento e correzione dell'errore

Supponiamo che durante la trasmissione un errore alteri il bit in posizione 5, modificandolo da 0 a 1. Il messaggio ricevuto sarà:

```
Posizione: 1 2 3 4 5 6 7
Bit:       0 1 1 0 1 1 1
```

Ora verifichiamo i bit di parità per trovare l'errore (il numero di "1" deve essere sempre pari):

- **P1** controlla le posizioni 1, 3, 5, 7:
 - Bit in posizione 1 = 0
 - Bit in posizione 3 = 1
 - Bit in posizione 5 = 1 (cambiato)
 - Bit in posizione 7 = 1
 - Somma = $0 + 1 + 1 + 1 = 3$ (dispari), quindi **P1 rileva un errore.**
- **P2** controlla le posizioni 2, 3, 6, 7:
 - Bit in posizione 2 = 1
 - Bit in posizione 3 = 1
 - Bit in posizione 6 = 1
 - Bit in posizione 7 = 1
 - Somma = $1 + 1 + 1 + 1 = 4$ (pari), quindi **P2 non rileva errori.**
- **P4** controlla le posizioni 4, 5, 6, 7:

- Bit in posizione 4 = 0
- Bit in posizione 5 = 1 (cambiato)
- Bit in posizione 6 = 1
- Bit in posizione 7 = 1
- Somma = $0 + 1 + 1 + 1 = 3$ (dispari), quindi P4 rileva un errore.

I bit di parità che rilevano un errore sono **P1** e **P4**. Se combiniamo questi valori in binario, otteniamo 101, che corrisponde alla posizione 5 in decimale.

Quindi sappiamo che l'errore si trova nella posizione 5. Possiamo correggerlo invertendo il bit in posizione 5 (da 1 a 0), ripristinando il messaggio originale:

```
Posizione: 1 2 3 4 5 6 7
Bit:       0 1 1 0 0 1 1
```

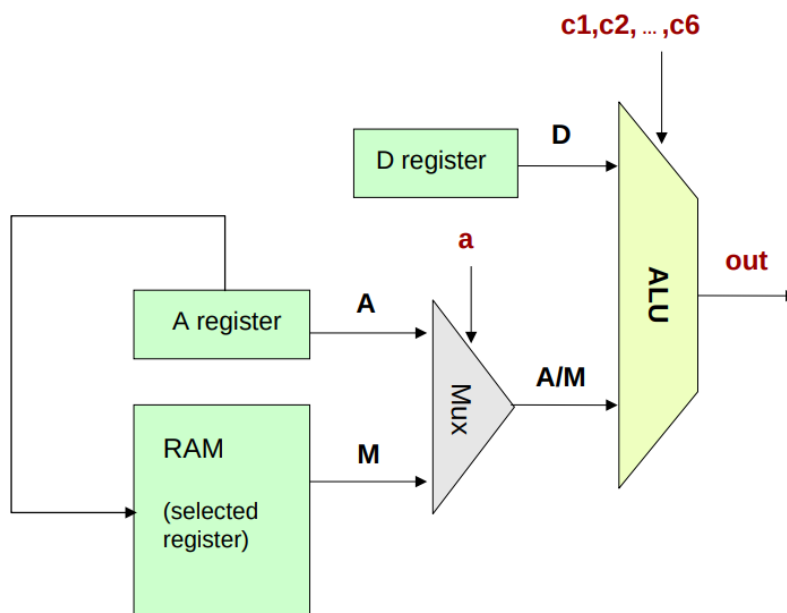
Il messaggio corretto è quindi **0110011**, che decodificato restituisce 1011, il messaggio originale!

ALU Hack

L'ALU Hack è un computer a 16 bit, con due ingressi e un'uscita. Ha anche due operatori di controllo. Fa una serie di operazioni, e ogni selettore dà informazioni su quali operazioni fare. Ci sono anche dei **flag** che tengono traccia di ciò che è successo nel sistema; lavora in complemento a due.

All'interno dei calcolatori più complessi in genere ci sono più di un ALU specializzate.

L'ALU Hack non sa fare moltiplicazioni e divisioni. Le ALU più complesse sanno fare tutte le varie operazioni e le operazioni a virgola mobile.



L'ALU prende un operando dal registro D, l'altro dal registro A oppure dalla RAM, selezionando con il Multiplexer. Questi dati arrivano dalla control unit.

Numeri complemento a due: ci può essere l'overflow. L'ALU Hack non ha questo problema.

L'ALU Hack è progettata per semplificare al massimo la logica e la circuiteria, e quindi:

- Non esiste un flag di overflow o una circuiteria dedicata per rilevarlo.
- Le operazioni sono limitate alle dimensioni dei 16 bit, e qualsiasi overflow o underflow (ad esempio, $32,767+132,767 + 132,767+1$ o $-32,768-1-32,768 - 1-32,768-1$) viene gestito automaticamente tramite il troncamento dei bit.

Adder: circuito progettato per sommare due interi.

Prende due input a 16 bit e un output a 16 bit + 1 bit di riporto.

Per fare la somma di due interi a n bit, le tecniche standard (mappa di karnaugh e tabella di verità) non arrivano. La tecnica migliore è imparare prima a sommare due bit (circuito half adder), poi imparare a sommarne 3 (circuito full adder); in seguito si implementano le stesse tecniche per sommare più cifre.

Il riporto è la AND, la somma è la XOR.

Half Adder → XOR tra a, b per fare la somma + OR tra a, b per fare il riporto

Aggiungendo all'half adder un altro half adder che somma all'output il terzo addendo, ho la somma.

I due carry finiscono in una or.

Uso dei control bit

$zx \rightarrow$ se è vero, $X = 0$ altrimenti lo lascia passare inalterato

$nx \rightarrow$ se è vero, X viene negato, altrimenti lo lascia passare inalterato.

zy e ny fanno la stessa cosa su y

$f \rightarrow$ se è vero fa la somma, altrimenti fa la and;

$no \rightarrow$ se è vero, nega l'output

X ed Y vanno in un circuito che o lascia passare inalterato, o azzerà (questo tramite un multiplexer); nx/ny seleziona se farli passare normali oppure negati.

f fa uscire la somma oppure la and; no decide di fare passare il risultato normale o quello negato.

I singoli operandi sono multiplexer.

Somma di X ed Y → f deve essere uguale a 1. Per questo a f devono arrivare in input due 0.

And di X e Y → a f arrivano in input un 1 e uno 0 (possono essere x o y indifferentemente)

Or di X e Y → se nego gli ingressi della AND, riesco a fare la Or: lascio quindi passare i due valori X e Y, poi li nego entrambi; esce una And negata, che devo negare di nuovo in corrispondenza di no .

Usando la codifica del complemento a 2, come abbiamo detto, è possibile rendere la ALU in grado di eseguire sottrazioni senza implementare la sottrazione vera e propria: infatti sottrarre in complemento a 2 è l'equivalente di fare la negazione bit-a-bit della somma fra il primo negato bit-a-bit ed il secondo:

$$x - y = -(-x + y) = -(!x + 1 + y) = -(!x + y) - 1 = !(!x + y) + 1 - 1 = !(!x + y)$$

Negazione dei numeri in complemento a 2:

$-x$ = prendi un numero, nega tutti i bit, e somma 1 (110111)

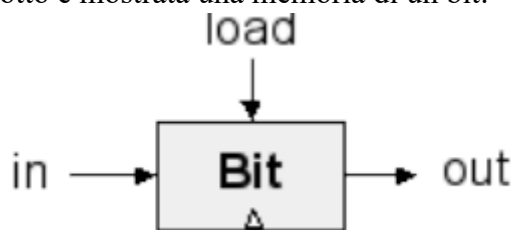
Deve quindi arrivarmi in input $!x$, y , poi quando $(!x + y)$ passa attraverso *no* la neghiamo ed esce $!(!x + y)$

La ALU ha due bit di stato:

- zr : se $out == 0$ allora $zr = 1$, altrimenti $zr = 0$
- ng : se $out < 0$ allora $ng = 1$, altrimenti $ng = 0$

Circuiti Sequenziali

Nei circuiti logici combinatori l'output in un certo istante dipende solamente dagli input nel medesimo istante. Esistono però componenti, come le memorie, il cui output dipende da input precedenti. Ad esempio, qui sotto è mostrata una memoria di un bit:



“out” avrà il valore che aveva “in” nell’ultimo istante in cui “load” è stato attivo. In altre parole, questo componente “memorizza” il valore che aveva “in” nell’ultimo momento in cui è stato attivato “load”.

Latch SR

Un esempio base di circuito non combinatorio è il latch SR.

Un latch SR (Set-Reset) è un tipo di circuito bistabile che può mantenere uno stato (0 o 1) finché non riceve un nuovo input. Il latch SR ha due ingressi, generalmente indicati come S (Set) e R (Reset), e due uscite, Q e Q' (dove Q' è il complemento di Q). Le uscite dipendono dagli ingressi S e R secondo la seguente logica:

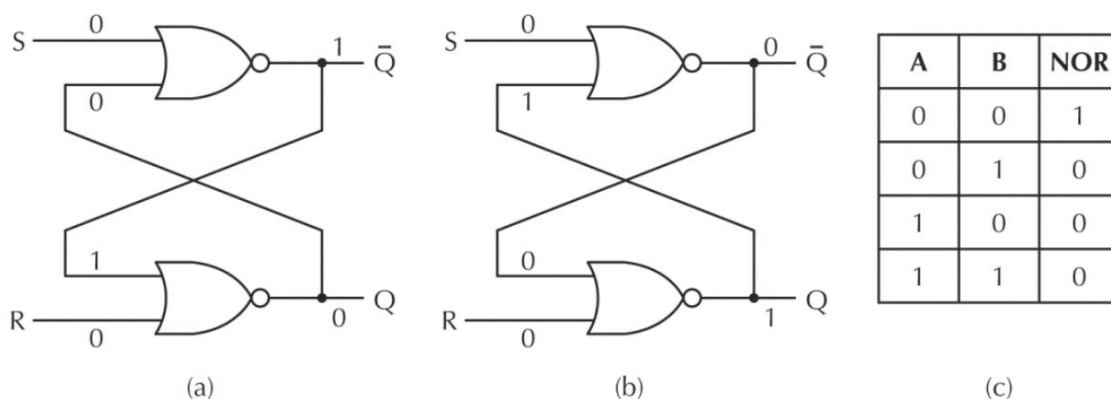


Figura 3.21 (a) Latch di tipo NOR nello stato 0. (b) Latch di tipo NOR nello stato 1. (c) Tabella di verità del NOR.

Le uscite dipendono dagli ingressi S e R secondo la seguente logica:

- Se $S=1$, $R=0$ allora $Q=1$, $Q'=0$
- Se $S=0$, $R=1$ allora $Q=0$, $Q'=1$
- Se $S=1$, $R=1$ allora $Q=0$, $Q'=0$
- Se $S=0$, $R=0$ allora ... non si sa!

Se i due input sono a 0, non si sa bene quale sarà l'output. Infatti l'output dipenderà da valori precedenti dell'input.

OSSERVAZIONE: tale circuito contiene un ciclo, cosa che non è permessa nei circuiti combinatori!

Tale circuito è usato in modo tale da non avere mai come input $S=1$, $R=1$. Quindi avremo che i due output sono sempre uno la negazione dell'altro.

Sotto questa ipotesi riconsideriamo il caso $S=0$, $R=0$. Vediamo che gli output rimangono esattamente gli stessi dell'istante in cui entrambi gli input sono scesi a 0. Il latch "memorizza" l'ultimo segnale: se è S allora $Q=1$, se è R allora $Q=0$.

Quando gli ingressi S e R sono entrambi uguali a 0, lo stato del latch dipende dal valore precedente che aveva assunto. Non c'è una variazione che "forza" un nuovo stato, e perciò il latch mantiene il valore precedente di Q e Q'. In altre parole, non cambia il suo stato e rimane nell'ultimo stato in cui si trovava.

Quindi, non è propriamente "imprevedibile" nel senso di essere casuale, ma piuttosto, **non può essere determinato solo sulla base degli input attuali S e R**. Serve conoscere lo stato precedente del latch per sapere cosa succederà a Q.

Latch SR temporizzato

Di solito, si evita che il latch cambi il proprio stato in particolari momenti, e lo si rende sensibile agli input in altri intervalli di tempo.

Un **latch temporizzato** è un tipo di latch che include un segnale di controllo, il Clock. Questo segnale di controllo determina quando il latch può accettare i valori di input e aggiornare il suo stato.

Mentre un latch SR standard reagisce immediatamente ai cambiamenti degli ingressi S e R, un latch temporizzato aggiorna il suo stato solo quando il segnale di abilitazione clock è attivo, cioè ha valore 1. Quando ha valore 0, invece, non può cambiare stato.

L'uso del segnale di abilitazione permette di controllare con maggiore precisione quando il latch deve cambiare stato. Questo è particolarmente utile nei circuiti digitali, perché consente di sincronizzare il cambiamento degli stati con altri segnali del sistema (come un segnale di clock, per esempio).

Questo tipo di latch è spesso utilizzato come componente base per flip-flop e altri circuiti di memoria, che a loro volta sono fondamentali per la realizzazione di contatori e registri nei computer.

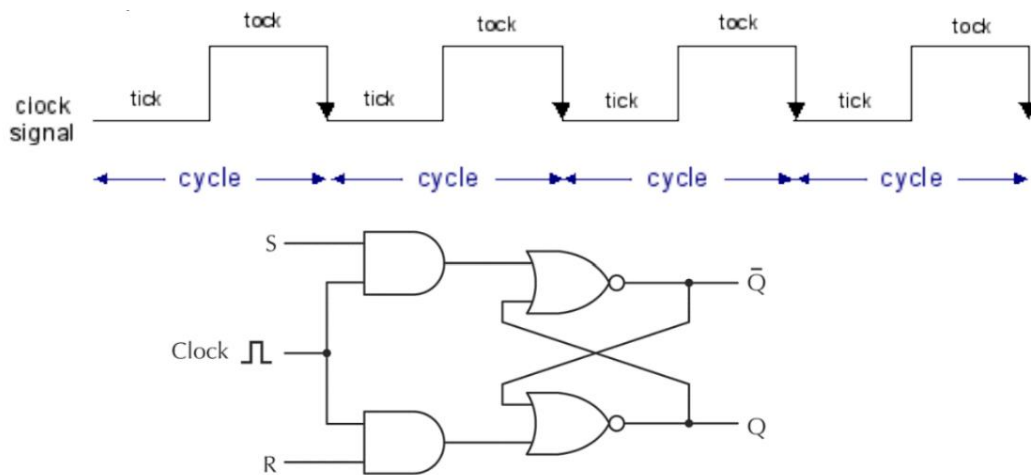


Figura 3.22 Latch SR temporizzato.

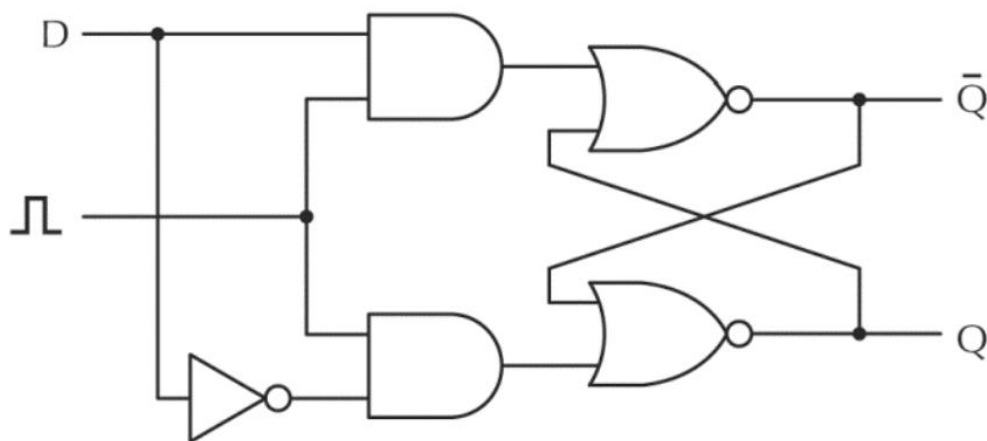
Latch D temporizzato

Per essere sicuri che la situazione indesiderata ($S=R=1$) non si verifichi, si usa il latch D che ha un solo input (detto appunto D) che viene collegato a S, e negato per l'input R.

Il latch D cambia il proprio stato mentre il clock è attivo. Durante la fase 0 del clock, il latch memorizza l'ultimo valore di D alla fine della fase 1, e non cambia stato.

Questo comportamento è detto “commutazione a livello” (livello 1 di clock permette il cambio di stato).

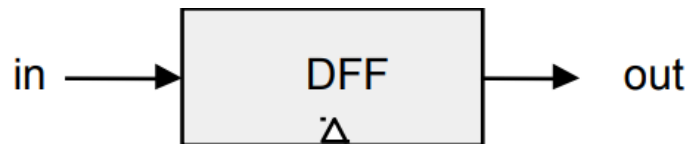
C'è quindi solo un input, che passa direttamente e negato: quando il clock è attivo, cambia stato, quando invece ha valore 0, memorizza l'ultimo input e non cambia stato. In questo modo, non ha nessuna condizione indeterminata, ed ha un comportamento prevedibile.



Flip-Flop (DFF)

Circuiti logici con due stati stabili, con “commutazione sul fronte” (cioè cambio di stato solo quando il clock sale oppure quando scende) sono detti flip-flop (il nome deriva dal suono che facevano i primi circuiti bi-stabili a relè durante il loro cambiamento di stato).

Esistono vari tipi di flip-flop, noi considereremo il flip-flop di tipo D (useremo la sigla DFF). “t-1” e “t” indicano cicli consecutivi del clock:



$$\text{out}(t) = \text{in}(t-1)$$

I flip-flop di tipo D sono tra i circuiti di memoria più utilizzati nei sistemi digitali, come registri, contatori e memorie. Rispetto ai latch, i flip-flop introducono una maggiore precisione nella gestione dei segnali grazie al segnale di clock che controlla esattamente quando lo stato del flip-flop deve essere aggiornato.

Un flip-flop D ha due ingressi principali:

- D: Data (dato)
- Clock (o CLK): Segnale di clock.

E due uscite:

- Q: L'uscita principale, che memorizza il dato.
- Q' (oppure $\bar{Q}$): Il complemento di Q, che è l'inverso di Q.

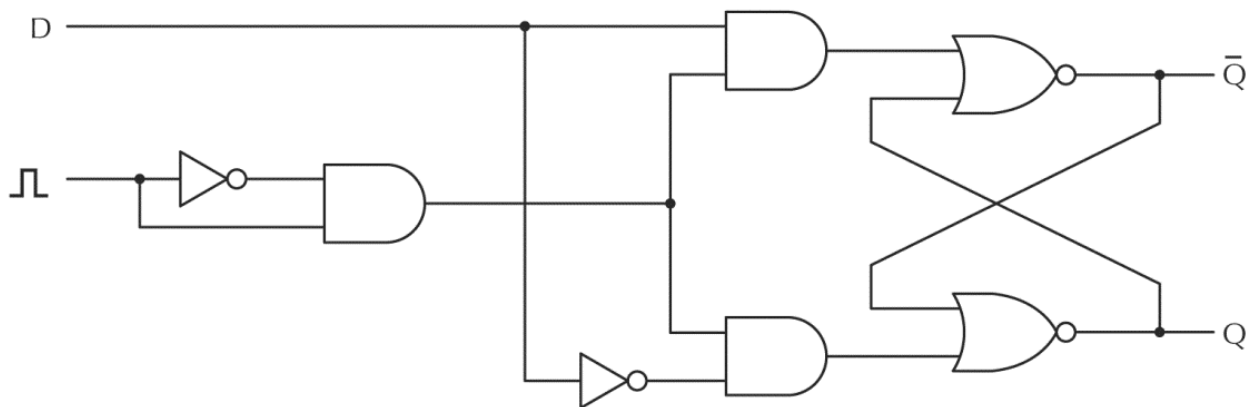
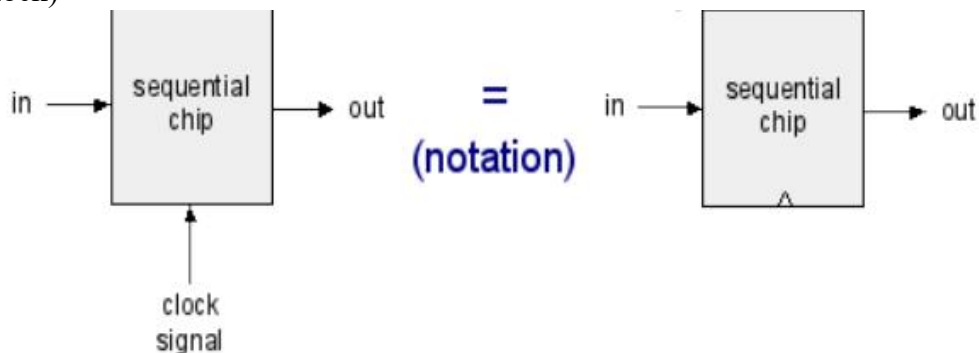


Figura 3.25 Flip-flop D.

Il flip-flop D funziona in modo simile al latch D, ma aggiorna il suo stato solo in corrispondenza di un cambiamento del segnale di clock.

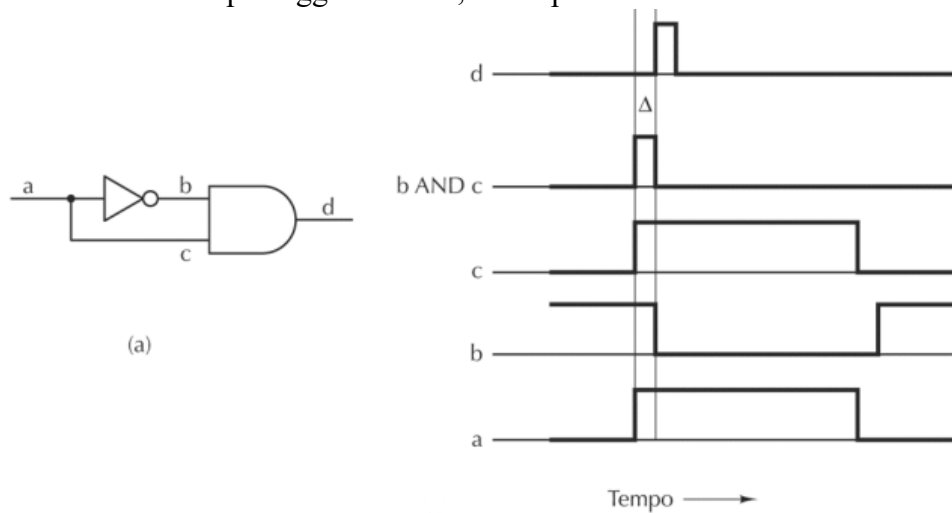
- Il flip-flop D aggiorna **Q** quando il segnale di clock passa da **0** a **1**.
- Il flip-flop D aggiorna **Q** quando il segnale di clock passa da **1** a **0**.

Il triangolo in basso indica che si tratta di circuito temporizzato (controllato dal fronte di salita di un segnale di clock)

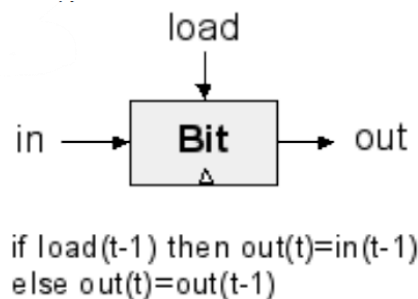


Il circuito sul clock crea un unico breve istante in cui il segnale in uscita è 1 (sul fronte di salita del clock).

Inserire una Not rallenta il passaggio dei dati; non è più considerabile istantaneo.



Memoria da 1-bit



Il 1-bit register “Bit” memorizza il valore di “in” nell’ultimo istante in cui “load” è stato attivo. Alla fine sono multiplexer.

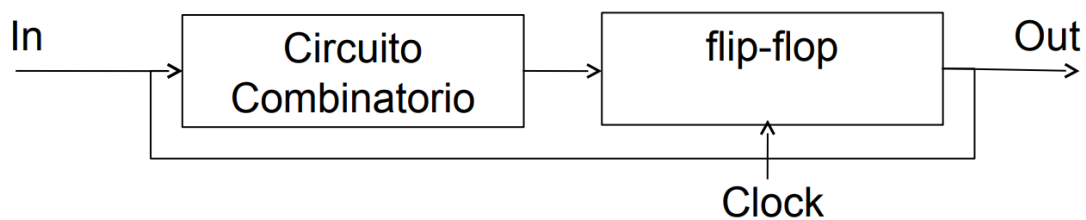
È possibile usare un DFF per riportare in uscita all’istante t il valore deciso all’istante t-1 tramite un usuale circuito combinatorio.

Per fare una memoria a 16 bit combino 16 memorie da 1-bit. Il load e quindi il clock sarà lo stesso per tutti.

Schema tipico per realizzare circuiti sequenziali:

L’implementazione del 1-bit register ricalca uno schema tipico nella realizzazione di circuiti sequenziali.

- Un circuito combinatorio calcola il valore di uscita.
- Tale valore viene riportato al ciclo di clock successivo tramite un flip-flop, in modo tale che l’uscita successiva dipenda dall’uscita precedente e dagli input precedenti: $out[t] = f(out[t-1], in[t-1])$



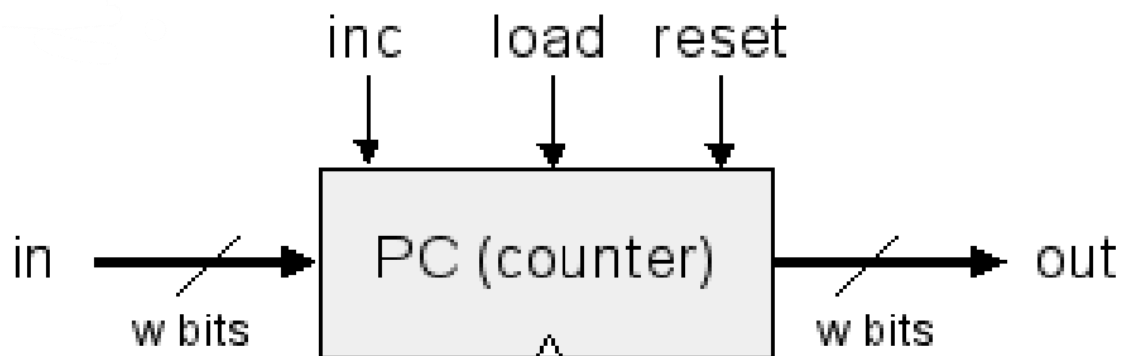
Contatore

Mentre i registri sono tutti uguali tra loro, il program counter è leggermente diverso:

Il program counter ha più funzionalità. Come i registri normali ha un output e un load; in più ha

- il reset che azzerava il valore, che serve ad eseguire la memoria dall'inizio. Tipicamente quando resettato il computer riparte da zero.
- L'inc, che quando eseguiamo un assembler esegue le istruzioni in sequenza una dopo l'altra, e lo va incrementando di uno il valore.

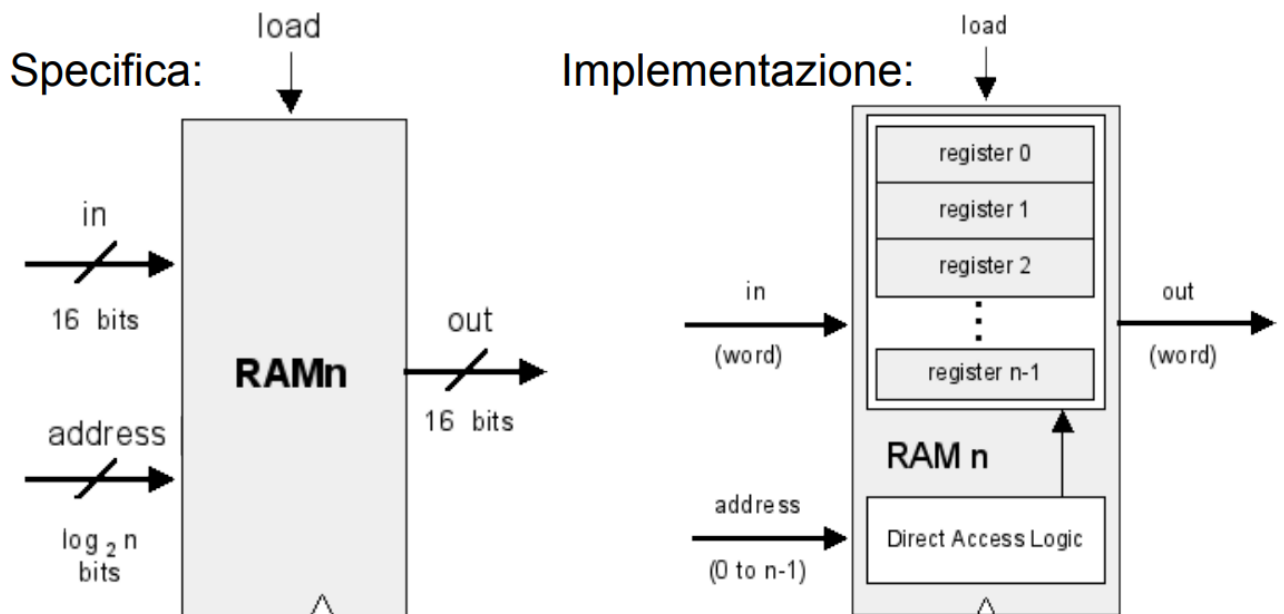
O si attiva reset, o si attiva load, o si attiva inc.



```

[true]
if reset(t-1) then out(t)=0 [true]
else if load(t-1) then out(t)=in(t-1)
else if inc(t-1) then out(t)=out(t-1)+1
[true] else out(t)=out(t-1)
  
```

Una memoria con n locazioni da w -bit, può essere realizzata con n “ w -bit register” controllati da uno specifico circuito (Direct Access Logic) che indica quale di questi registri deve essere letto (e scritto se “load” è attivo)



In una memoria da 1kb ci sono 1024 circuiti (2^{10})

Direct Access Logic.

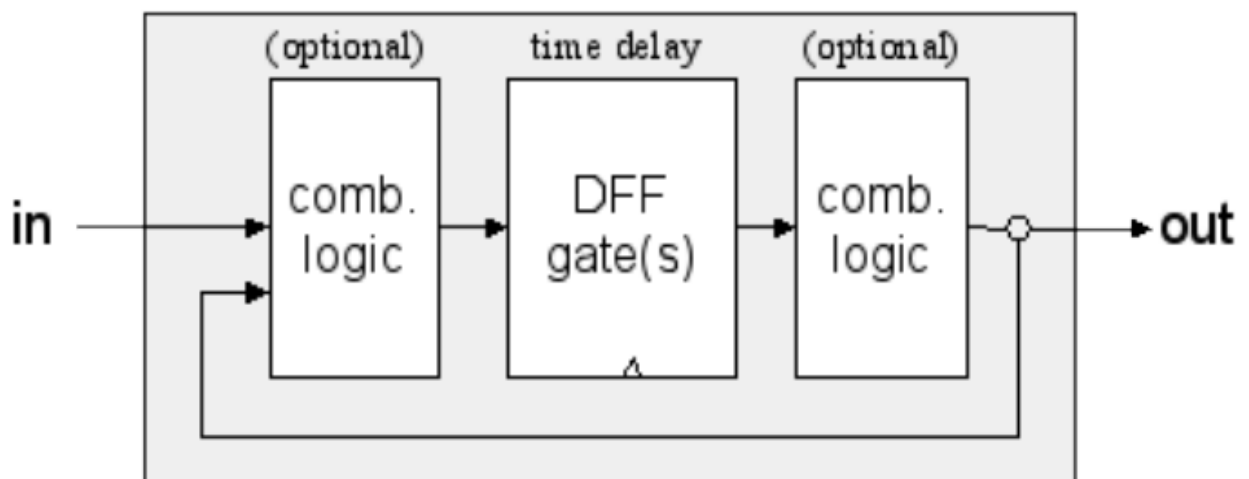
Dobbiamo prendere le uscite dei singoli, e prendere dei multiplexer per capire qual è l'uscita giusta.

il load può andare anche a tutti, ma deve essere a zero in quelli sbagliati e deve passare in quello giusto.

Nella implementazione della memoria della slide precedente, torna utile avere un circuito combinatorio anche dopo le uscite delle memorie.

Utile per decidere quale delle uscite dei singoli registri deve essere portata su "out". In generale, quindi possiamo avere (o no) circuiti combinatori prima e dopo i flip-flop.

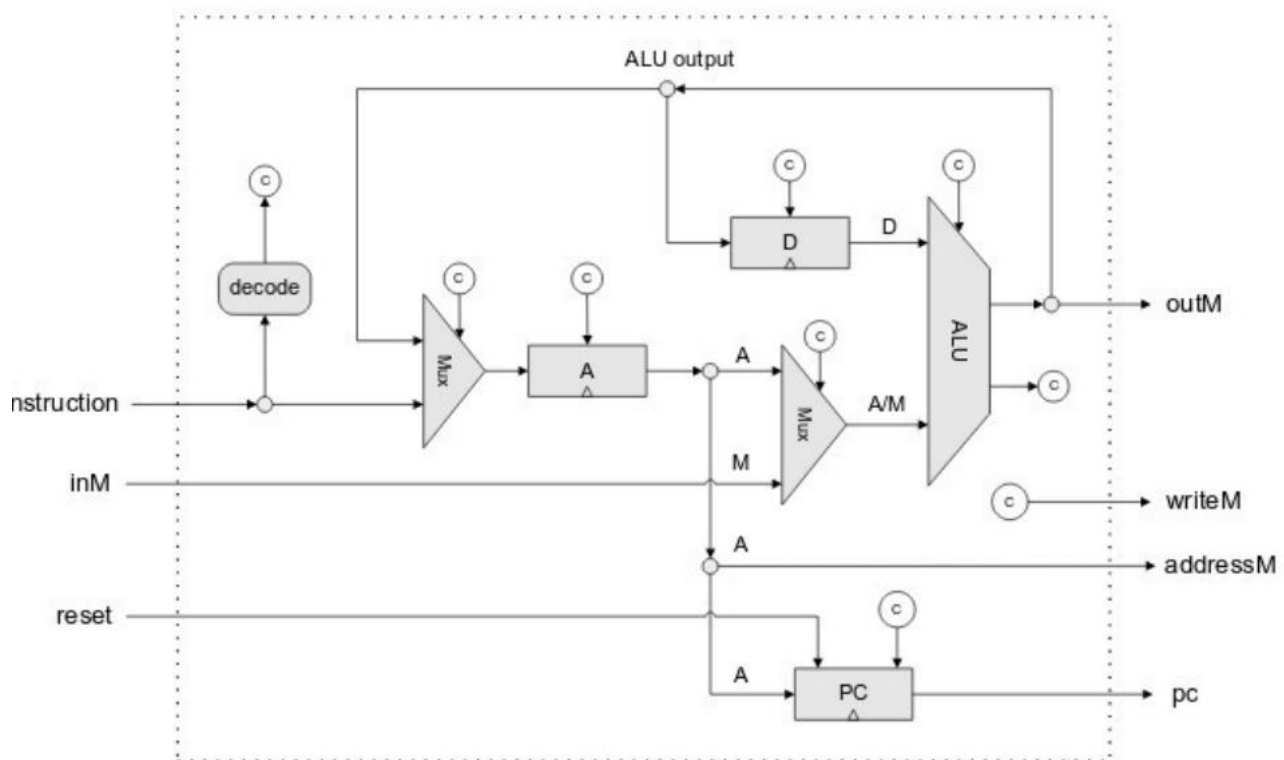
Sequential chip



Microarchitettura

Vedremo un esempio realistico per calcolatori molto vecchi o processori embedded in elettrodomestici ecc.

Abbiamo solo 2 registri, la Alu, il program counter; visto che il computer Hack fa tutto in un ciclo di clock, mancano alcuni registri, e i valori vanno direttamente al blocco decode (la CU) senza essere salvati.



inM = memoria

- L'istruzione viene letta dal decode, ed i suoi bit di output vengono usati come control bit (vedi i simboli "c") per tutti gli altri componenti.
- È presente la ALU
- Un registro D che contiene uno dei due operandi della ALU, e che può memorizzare un precedente output.
- Un registro A che può contenere un dato che fa parte delle istruzioni o un precedente output.
- Il secondo input della ALU può essere o il contenuto del registro A, oppure un dato proveniente dalla memoria.
- È presente anche il Program counter, che, per quanto riguarda i salti, può essere impostato tramite il registro A
- Il registro A può essere anche usato come puntatore alla memoria (per operazioni di lettura/scrittura)

Il flusso dei dati fra i vari componenti viene controllato tramite Mux.

I Mux ed i bit di controllo dei registri vengono gestiti da una microarchitettura composta da semplici circuiti combinatori. Infatti, l'intero ciclo Fetch-Decode-Execute del processore Hack viene eseguito in un solo ciclo di clock, ed i segnali di controllo sono funzione dell'istruzione corrente.

In altre parole, una istruzione in ingresso al tempo t viene completamente eseguita entro il tempo $t+1$.

Al tempo $t+1$ viene considerata l'istruzione successiva.

Accesso ai dati in memoria

Le memorie realizzate usando flip-flop come in Hack sono chiamate SRAM (sono tra le più veloci, ma anche fra quelle più costose per quanto riguarda il costo per bit). Sono troppo costose per realizzare una intera memoria con tale tecnologia, per questo motivo la memoria viene organizzata secondo una gerarchia (eccone un tipico esempio).

- SRAM (“static”), usate per le cache → funziona in maniera simile a una sequenza di registri come abbiamo visto. È molto costosa.
- DRAM (“dynamic”), usate per la memoria centrale → funziona allo stesso modo ma è fatta in modo diverso.
- Dischi

SRAM e DRAM

Le SRAM (Static RAM) sono realizzate tramite flip-flop come le memorie viste in precedenza. Hanno una decina di transistor

- Veloci (ordine del nanosecondo)
- Usate principalmente per le cache

Le DRAM (Dynamic RAM) o SDRAM (Synchronous DRAM), usate per le memorie centrali, hanno un solo transistor ed un condensatore, un piccolo accumulatore che mantiene (tramite carica elettrica, che viene rilasciata piano piano) un singolo bit

- Visto che il condensatore perde la propria carica, deve essere ricaricato per evitare di perdere la propria informazione
- Si rendono necessarie periodiche fasi di “refresh” (ad intervalli dell’ordine del millisecondo). Appena prima che si dimentichi dell’informazione, si fa il refresh (quindi si ricarica)
- A causa del refresh sono più lente (ordine della decina di nanosecondi)
- Richiedendo un solo transistor costano meno e possono essere maggiormente miniaturizzate

Cache

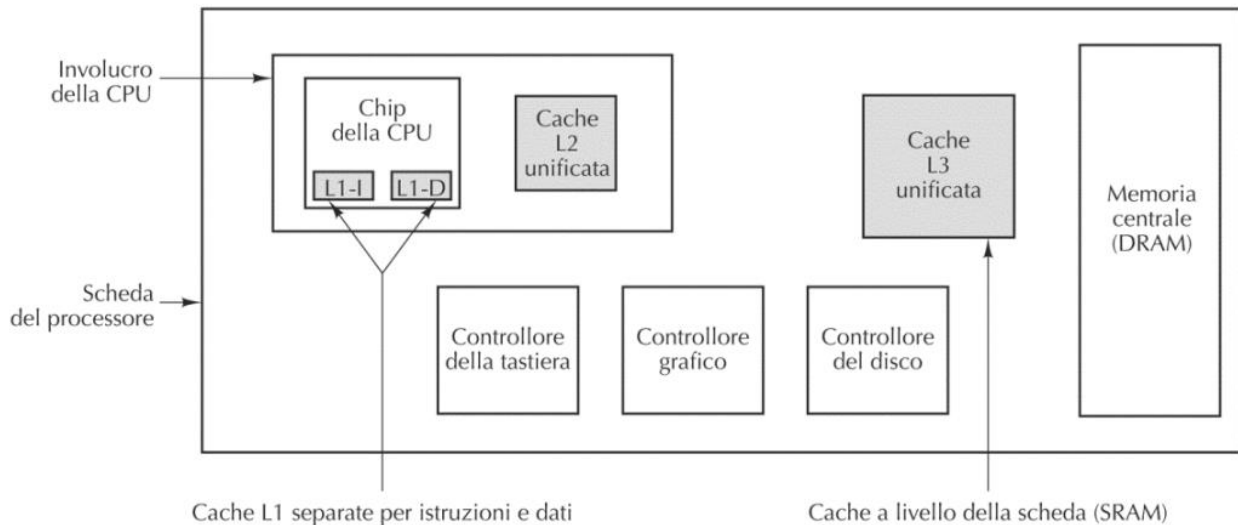
La Cache è una memoria più grande di un registro con velocità e dimensione intermedia; il calcolatore accede alla cache invece che accedere direttamente alla RAM

Le cache sono largamente utilizzate, ed organizzate in modo sofisticato. Riportiamo un tipico esempio di cache a tre livelli:

- Una prima piccola cache (livello 1: L1) è direttamente nel chip della CPU separata fra istruzioni e dati (dimensioni fra 16-64 KB). Ce ne sono quindi due. Sono molto veloci nell’accesso, anche perché sono le più vicine, ma molto piccole.
- Una seconda cache (livello 2: L2) un pochino più grande nel medesimo “involucro” della CPU “unificata” fra dati e istruzioni (fra 512 KB ed 1 MB)
- Una terza cache (livello 3: L3) esterna alla CPU (alcuni MB)

Si va a tentativi: la CPU va a cercare il dato nelle varie cache in ordine, e se non le trova va in memoria.

La cache programmando in assembler non viene vista; ha solo effetto sulla velocità e non su cosa possiamo eseguire.



Per località **temporale** intendiamo l'alta probabilità che la stessa cella di memoria venga acceduta più volte a breve distanza di tempo

- Mantenere in cache una cella acceduta di recente rende altamente probabile che una delle prossime operazioni si trovi in cache la cella di cui necessita, senza dover andare in memoria centrale
- Località temporale avviene ad esempio in esecuzioni basate su stack (last-in-first-out) che limitano l'accesso ai soli dati in cima allo stack, o nei loop

Per località **spaziale** intendiamo l'alta probabilità che celle di memoria vicine possano essere accedute a breve distanza di tempo

- Se inseriamo in cache blocchi contigui di celle di memoria è altamente probabile che una cella che verrà a breve acceduta verrà trovata in cache, senza dover andare in memoria centrale
- Località spaziale dovuta ad esempio a esecuzione sequenziale delle istruzioni e accesso sequenziale ad array

Organizzazione delle cache

A livello di microarchitettura si definisce il modo di organizzare la cache. Esistono diverse politiche di gestione: vediamo come esempio la "cache a corrispondenza diretta" (**direct mapped cache**)

- Si suddivide la memoria centrale in blocchi di dimensione m
- La cache è organizzata in un certo numero, diciamo n , di linee di cache di medesima dimensione m
 - Le linee di cache sono indicizzate da 0 a $n-1$
- I blocchi della memoria principale possono essere inseriti in cache secondo uno schema "ad orologio"
 - Il k -esimo blocco in memoria centrale può essere inserito nella linea di cache di indice " $k \bmod n$ "
 - In cache viene tenuta traccia di quale specifico blocco è attualmente presente in ogni linea di cache
- Ad ogni accesso in memoria, in base all'indirizzo della cella di memoria, si capisce in quale linea di cache andare a cercarlo

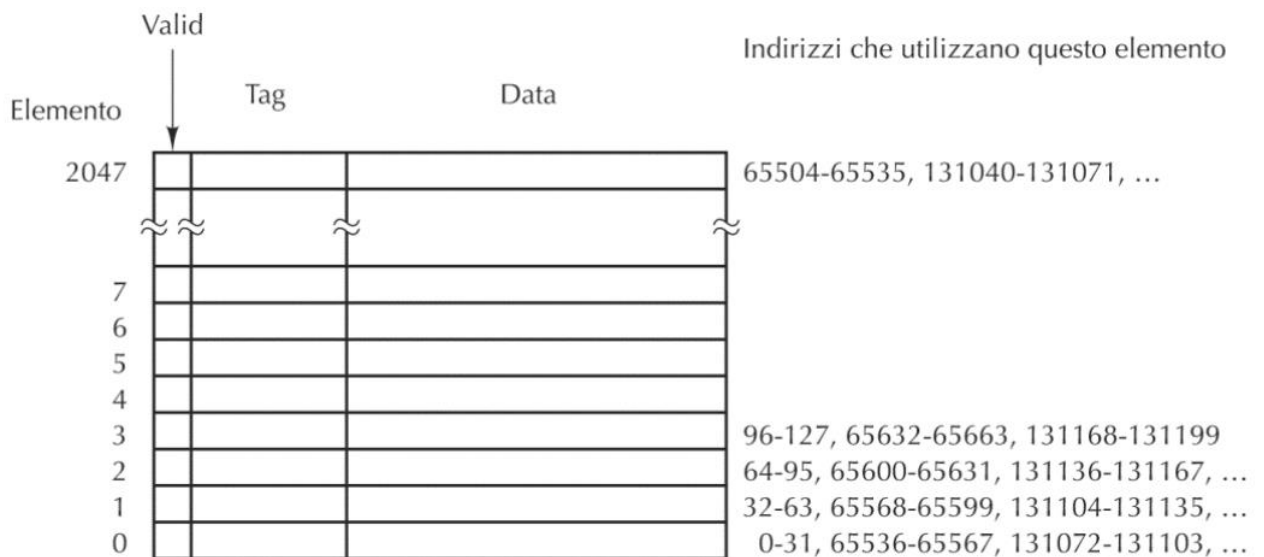
Direct Mapped Cache

Immaginiamo ora una cache con $n=2048$ linee di dimensione $m=32$ byte

- “Valid”: indica se la linea di cache contiene un blocco. (sarà a 0 inizialmente, a 1 quando viene scritto un dato)
- “Data”: contiene i 32 byte del blocco
- “Tag”: indica esattamente quale blocco è contenuto

All’inizio la cache contiene dati a caso, che pian piano vengono sovrascritti con i dati giusti.

Scrivo i dati a blocchi di 32 byte, e vado su fino a che non finisco la cache. Quando ho finito, riparto da capo da zero. Per capire a quale blocco di dati accedere in ogni riga, utilizzo il tag.



Immaginiamo di avere indirizzi di memoria a 32 bit. I 5 bit meno significativi indirizzano il byte dentro data. Gli 11 bit successivi indicano quale linea di cache usare. Gli altri 16 bit vanno confrontati con tag: se coincidono (e se valid è vero) allora il byte cercato è in cache, altrimenti è da cercare in memoria principale.

Esempio 1: vogliamo accedere a questo indirizzo.

... 011011011011000110

Per capire se la cache ha il dato relativo procede così:

- i 5 bit meno significativi individuano il byte nella linea
- gli 11 bit successivi indicano quale linea usare, e controlla che valid sia true.
- Se valid è true, vado a controllare il campo tag indicato dai restanti bit e confronto con il tag della linea di cache.

Se effettivamente il dato viene trovato, esso è mandato al processore; altrimenti, lo recuperiamo dalla memoria generale, e prima di rimandarlo al processore, sovrascriviamo quel dato nella cache salvandolo.

Istruzione successiva: sarà uguale ma con un 1 in fondo.

... 011011011011000111.

Il controllo rimane nella stessa linea, e il tag è uguale; prendo solo il bus successivo. In questo caso, se l’istruzione sarà uguale la troverà sicuramente, altrimenti è comunque probabile che lo trovi perché i dati vengono sovrascritti in blocco: la cache non sovrascrive solo la locazione desiderata ma anche quelle vicine. Se la cache avrà bisogno di recuperare una locazione vicina alla precedente nella stessa riga, la troverà sicuramente.

Esempio 2

Vogliamo accedere alla locazione 34 in una cache con campo data da 16 parole e 8 linee di cache. Assumo indirizzi di memoria su 10 bit.

34:

Lo scrivo in binario e lo spezzo come descritto sotto.

000 010 0010

campo data da 16 parole → 4 bit meno significativi per l'offset dentro data 0010

8 linee di cache → 3 bit successivi per indirizzare la linea 010

tutto il resto e' il tag: 000

Come accedo:

- Vado alla riga 010
- Se valid e' falso → cache miss, fine (vado in RAM)

Se valid e' vero, confronto il tag della riga 010 con quello dell'indirizzo (000)

- Se sono diversi => cache miss, fine (vado in RAM)
- Se sono uguali => cache hit, fine (mando il dato in posizione 0010 alla CPU)

In caso di cache miss prendo dalla RAM tutto il blocco che contiene l'indirizzo (indirizzi da 000 010 0000 a 000 010 1111) e lo scrivo nel campo data della riga considerata (riga 010)

Metto il tag della stessa riga al valore dell'indirizzo al quale sto accedendo (000 010 0010, quindi tag 000)

Metto valid della stessa riga a vero.

In contemporanea mando alla CPU il valore richiesto

Esempio 3

Si consideri una direct mapped cache con 8 linee di cache, ognuna da 16 parole, inizialmente con tutti i valid a 0. Si mostri come cambia il contenuto della cache con l'esecuzione di accessi alle seguenti locazioni:

locazione 12

0 **000** 1100 → valid false → sovrascrivo il bus da 0-15 (cache miss). Valid = 1, tag = 0.

locazione 14

0 **000** 1110 → valid true → tag = 0 → trovo la locazione 14 (cache hit)

locazione 22

0 **001** 0110 → valid false → sovrascrivo (cache miss) → Valid = 1, tag = 0

locazione 130

1 **000** 0010 → valid true → tag diverso da 1 (cache miss) → Valid = 1, il tag = 1.

locazione 24

0 **001** 1000 → valid true → tag uguale a 0 (cache hit) → la 24 è nello stesso blocco della 22

locazione 13

0 **000** 1101 → valid true → tag diverso da 1 (cache miss)

Quando l'accesso alla cache ha successo si dice che è avvenuta una "cache hit" (successo della cache). Quando l'accesso alla cache non ha successo si dice che è avvenuta una "cache miss"

- In questo caso il blocco di interesse deve essere portato in cache, ma prima il contenuto della corrispondente linea di cache deve essere riportato in memoria.
- Solo in questa fase una modifica ad una cella di memoria presente nella linea di cache diventa visibile anche in memoria centrale.

Esiste quindi un momento in cui non c'è coincidenza fra i dati in memoria e i corrispondenti dati in cache.

- Questo può generare problemi quando più processori o più dispositivi accedono alla memoria centrale! Bisogna quindi gestire questi casi di conflitto (ad esempio, vietando l'accesso per i blocchi attualmente in cache alla loro versione in memoria)

Paginazione (non c'entra nulla con microarchitettura ma è relativa a OS)

Il principio che regola la paginazione è molto simile al funzionamento della cache, nonostante sia più complesso.

La cache deve fare molto in fretta, non può fare cose troppo complicate: altrimenti non avrebbe senso usarla e andremmo direttamente in memoria.

La paginazione invece deve essere più veloce del disco; d'altro canto, un fallimento è più critico in questo caso. L'algoritmo sarà quindi più raffinato.

Moralmente, cache e paginazione fanno la stessa cosa; sono però diverse le condizioni di contorno.

Cache vs RAM → tempi velocissimi e lavora a livello esclusivamente hardware.

Paginazione vs disco → tempi più lunghi, e lavora anche a livello software.

La paginazione:

Fra memoria centrale e memoria di massa (dischi) esistono politiche simili di spostamento dei dati

- Solitamente si usa la tecnica della paginazione, gestita dal sistema operativo
- Blocchi di dati (solitamente chiamate "pagine") sono mantenute in memoria di massa, e vengono spostate in memoria centrale quando tali dati servono
- Quando i dati non servono più vengono riportati in memoria di massa
- Questi spostamenti sono gestiti da algoritmi, detti algoritmi di "paginazione", eseguiti dal sistema operativo

Studieremo queste cose durante l'analisi del livello "sistema operativo"

Pre-fetch istruzioni

Ci sono dei momenti morti nel fetch delle istruzioni: quando il processore manda le istruzioni la memoria aspetta, mentre le istruzioni vengono recuperate il processore aspetta. Per ottimizzare questo processo, quando la memoria ha recuperato istruzione 1, la memoria inizia subito a mandare istruzione 2 ecc.

Il processore ha due parti distinte: quella che manda le istruzioni e quella che gestisce la memoria.

Problematiche

- serve una cache di istruzioni che viene riempita automaticamente sia quando chiediamo una certa cella di memoria, sia riempiendola con ciò che ci servirà in futuro. Es. se sto chiedendo la cella da 0-15, mi porto avanti iniziando a chiedere 16-31, che è molto probabile mi servano in futuro.
- Possibilità di salti; se l'istruzione seguente non è quella immediatamente successiva, ho salvato delle istruzioni inutili, e il processo viene quindi rallentato. È quindi una questione probabilistica.

Nell'architettura del nostro calcolatore Hack, ci siamo semplificati la vita considerando due distinti ingressi per la CPU:

- "instruction": carica l'istruzione da eseguire da una specifica memoria programma
- "inM": carica i dati necessari da una distinta memoria dati

Nelle architetture usuali (Von Neumann) dati e programmi risiedono nella stessa memoria.

In ogni caso caricare un'istruzione e poi i suoi operandi richiede un ciclo di clock molto lungo.

Si usa hardware dedicato chiamato IFU: Instruction Fetch Unit

Pipeline

L'aggiunta di una unità indipendente di pre-fetch dell'istruzione è un primo esempio di pipeline

In un dato istante ci sono due istruzioni contemporaneamente in elaborazione nel processore, una istruzione nel primo stadio ed una al secondo stadio

1. fetch (F) → ci serve un unico program counter che si trova nel fetch
2. decode (D)
3. lettura operandi (L)
4. esecuzione sulla ALU (E)
5. scrittura risultato (S)

è un sistema a basso costo e molto efficiente, viene quindi largamente impiegato nei calcolatori.

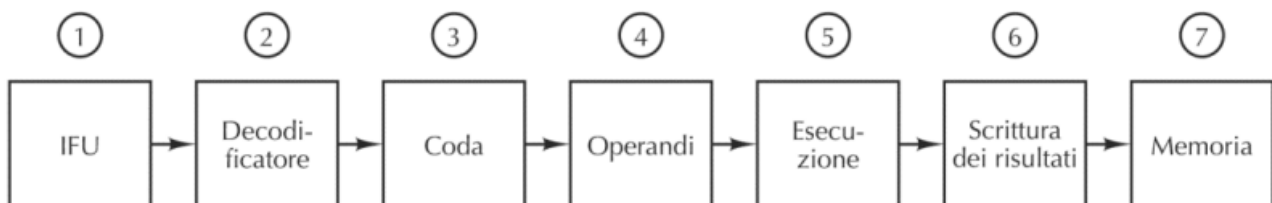
Con 5 fasi, assumiamo che il ciclo di clock del processore con pipeline sia $\frac{1}{4}$ del processore senza pipeline: è un'ipotesi ragionevole; infatti, se fosse lungo $\frac{1}{5}$ stiamo assumendo che il processo sia ottimale senza nessuna perdita di tempo.

Mi servono 5 cicli di clock per eseguire un'istruzione; con 34 cicli eseguo 30 istruzioni (34 deriva dal numero di istruzioni più 4).

Con 34 cicli di clock del processore con pipeline eseguo quindi 30 istruzioni:

$34/4 \rightarrow 8,5$ cicli di clock di un processore singolo $\rightarrow$ ottimo vantaggio

Esistono pipeline più complesse, ad esempio la seguente a 7 stadi:



Le istruzioni vengono inserite nel pipeline in modo sequenziale: Istruzione i-esima, (i+1)-esima, (i+2)-esima,...

Cosa succede quando l'istruzione i-esima è un salto (da istruzione i-esima a istruzione k-esima)? I salti possono essere molto frequenti!

if (i == 0)		CMP i,0	; confronta i con 0
k = 1;		BNE Else	; salta a Else se diversi
else	Then:	MOV k,1	; sposta 1 in k
k = 2;		BR Next	; salta a Next
	Else:	MOV k,2	; sposta 2 in k
	Next:		
(a)		(b)	

Se ci accorgiamo del salto allo stadio v della pipeline (ad esempio dopo l'“esecuzione”, stadio 5 nella pipeline della slide precedente) le istruzioni successive già negli stadi $1..(v-1)$ devono essere scartate

Le microarchitetture moderne usano meccanismi sofisticati per cercare di predire i salti già dai primi stadi. Per i salti incondizionati, si riescono a intercettare in fase di decode (solitamente stadio 2)

- per evitare di fare lavoro inutile, si può mettere sempre una istruzione nulla “nop” (no operation) dopo i salti incondizionati

Per predire salti condizionati si usano “euristiche”

- i salti all'indietro è più probabile che vengano eseguiti rispetto a salti in avanti
- i salti indietro sono usati alla fine dei cicli per tornare ad inizio ciclo (vedi “for” in C)

se una istruzione ha generato un salto nelle sue ultime due esecuzioni, mi aspetto che anche la prossima volta salti; se invece non ha saltato nelle ultime due esecuzioni mi aspetto che non salti nemmeno la prossima volta

- Statisticamente, se il salto che viene fatto all'indietro verrà eseguito, poiché in assembler abbiamo molti loop, e quindi è probabile che si riuscirà ad eseguirlo.
- Altrimenti, mi tengo un paio di bit per memorizzare se il salto l'ho fatto di recente o meno.

Può succedere che istruzioni in stadi diversi accedano ai medesimi registri (bisogna fare attenzione all'ordine in cui eseguire tali operazioni “concorrenti”)

Esempio di ordine sbagliato: istruzione in stadio 6 modifica i registri che una istruzione allo stadio 4 sta leggendo per caricare i propri operandi (dipendenza RAW=Read After Write)

In questi casi l'istruzione allo stadio 4 deve aspettare, bloccando anche quelle dietro (stall)

Si possono usare tecniche di riordino delle istruzioni per limitare queste situazioni

Se le istruzioni vengono riordinate dalla CPU la situazione si complica ulteriormente.

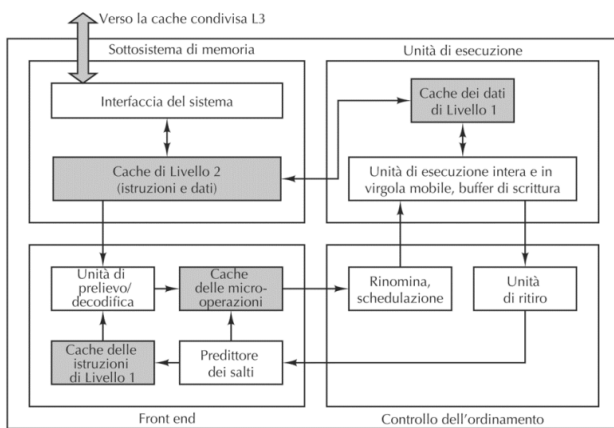


Figura 4.46 Diagramma a blocchi della microarchitettura Sandy Bridge del Core i7.

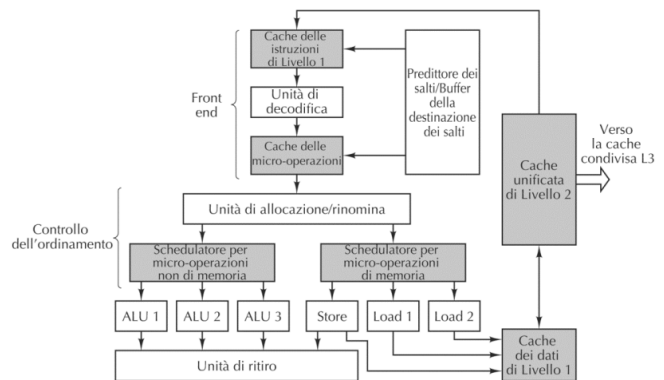


Figura 4.47 Vista semplificata del percorso dati del Core i7.

L'unità di “ritiro” ha lo scopo di rendere definitivi i risultati delle istruzioni (memorizzati temporaneamente in registri “invisibili”) nell'ordine giusto

Un dato in un registro “invisibile” non viene spostato fino a che non sono state ritirate tutte le istruzioni precedenti.

La microarchitettura descritta si replica per vari core disponibili all'interno del processore.

ISA HACK

L'ISA in termini generici rappresenta l'interfaccia fra hardware e software. Abbiamo studiato i principali componenti di un processore (circuiti logici combinatori e sequenziali, memorie, microarchitettura,..). Ora dobbiamo studiare come tali componenti vengono effettivamente usati. Per usare un processore, lo si deve programmare tramite un suo specifico linguaggio, costituito da un insieme di istruzioni che solitamente viene detto "instruction set" che dipende dalla cosiddetta "instruction set architecture" (ISA). Non ci basiamo su un linguaggio di alto livello, ma su linguaggi di basso livello direttamente eseguibili dall'architettura. Possiamo pensare ai linguaggi macchina in una versione "simbolica" interpretabile dall'essere umano.

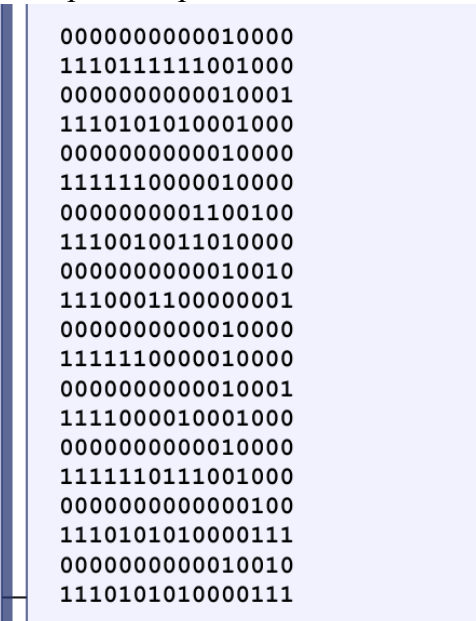
L'assembly è un linguaggio di alto livello in cui ogni istruzione di un programma corrisponde ad una istruzione macchina eseguibile dal processore.

In particolare studieremo l'assembly Hack, che è specifico per l'architettura Hack.

```
// Adds 1+...+100.
@i      // i refers to some RAM location
M=1     // i=1
@sum    // sum refers to some RAM location
M=0     // sum=0
(LOOP)

@i
D=M     // D = i
@100
D=D-A   // D = i - 100
@END
D;JGT   // If (i-100) > 0 goto END
@i
D=M     // D = i
@sum
M=D+M   // sum += i
@i
M=M+1   // i++
@LOOP
0;JMP   // Goto LOOP

(END)
@END
0;JMP   // Infinite loop
```



L'assembly corrisponde direttamente alla serie di numeri binari a sinistra, riga per riga.

Il linguaggio assembly dipende dalla instruction set architecture, quindi dipende dall'architettura specifica.

Cos'è la ISA? L'insieme degli elementi dell'architettura visibili al compilatore, cioè importanti quando si converte il codice sorgente in codice macchina. Non tutto è rilevante per poter compilare.

Instruction set → solo le istruzioni

Instruction set architecture → istruzioni + dettagli sull'architettura.

Storicamente, gli sforzi per migliorare le prestazioni di una architettura hanno portato a due generali approcci di progettazione dell'hardware. Complex Instruction Set Computing (CISC), con cui si cerca di ottenere prestazioni migliori sfruttando un instruction set più elaborato (che quindi permette di scrivere singole istruzioni che potenzialmente richiedono computazioni più complesse), e Reduced Instruction Set Computing (RISC), che utilizzano instruction set in cui ogni singola istruzione è più semplice, con lo scopo di consentire una implementazione hardware ottimizzata. Il computer Hack non è facilmente classificabile, dal momento che usa un instruction set semplice (vedi A-instruction e C-instruction più avanti) ma non utilizza tecniche di accelerazione hardware come quelle già studiate.

Tuttavia, come menzionato nel primo modulo, in Hack l'esecuzione di ogni istruzione viene completata in un singolo ciclo di clock.

Computer HACK → è una macchina a 16 bit

- Memoria ROM → contiene il programma.
- CPU → Nel computer Hack ogni istruzione è eseguibile in un solo ciclo di clock. Viene presa l'istruzione indicata dal program counter, che viene letta e codificata, e i risultati sono scritti in memoria.
- Memoria RAM
- Registri D, A → due registri interni alla CPU (A sta per address, D sta per Data)
Corrispondono all'incirca ai memory address register, e memory data register. In uno conteniamo tutti gli indirizzi, nell'altro salviamo temporaneamente i dati
Registro M: → uno qualunque dei registri di memoria in cui stiamo leggendo o scrivendo. È un registro dentro la memoria che corrisponde all'indirizzo scritto nel registro A.
- ALU
- Program Counter: il registro che contiene l'indirizzo del registro contenente la prossima istruzione da eseguire (ovvero in ROM[PC]). Inizialmente il valore di PC è 0.
- Alla CPU l'istruzione viene dalla ROM; l'outM viene dalla ALU e possiamo scriverlo in memoria. L'output PC torna indietro alla ROM dando l'istruzione successiva.

Segue il modello Von Neumann ma il programma è memorizzato in una memoria a parte.

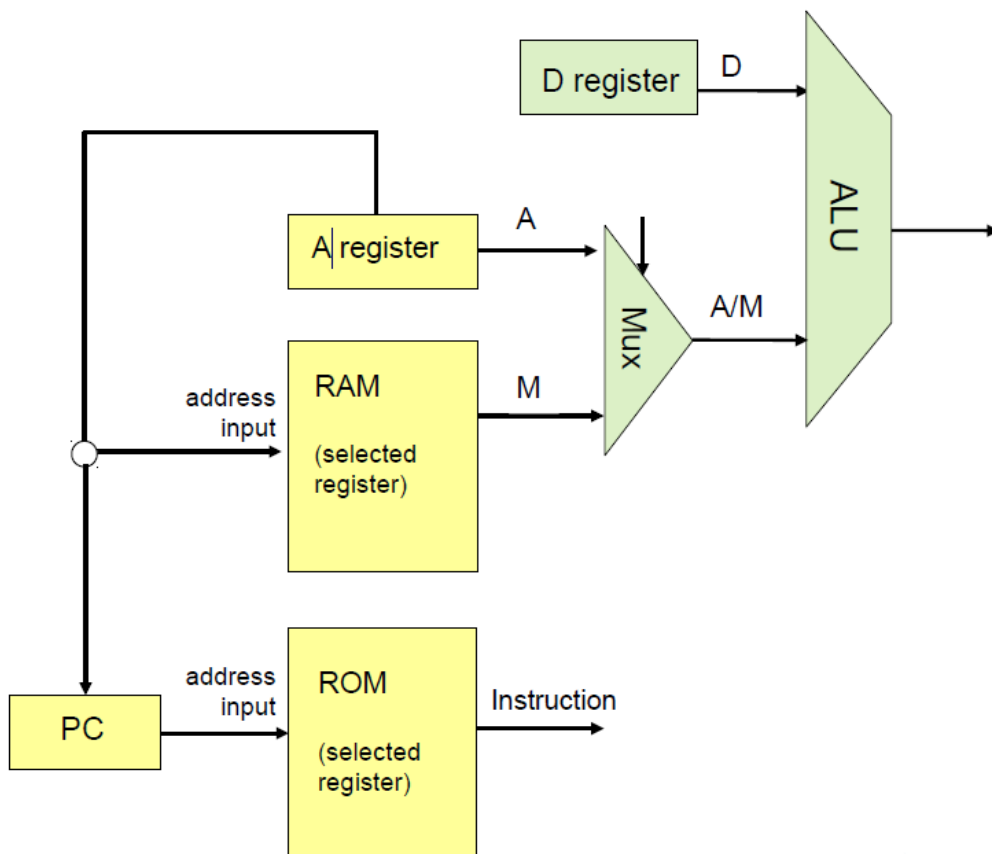
Nel registro A, possiamo salvare l'istruzione che arriva dalla ROM, oppure l'output della ALU.

Nel registro D posso salvare l'output della ALU;

Alla Alu posso dare in input D oppure A o M, in base all'indirizzo scritto in A.

Decode → decodifica l'istruzione, e genera un segnale di controllo, che entra in tutti i pezzi della CPU.

Nel program counter entrano l'adresa, il reset e ©



Esistono due tipi di istruzioni in Assembly Hack:

- A-instruction: istruzioni che si limitano a caricare un valore all'interno del registro A. Useremo queste istruzioni per memorizzare in A valori numerici.
- C-instruction: istruzioni che eseguono computazione tramite la ALU, prelevando i due operandi dai registri A, D, M. Possiamo eseguire il not, l'inversione del valore, l'and, la somma e sottrazione, passare il valore così com'è, oppure passare true e false.
- 0, 1, -1, D, A, !D, !A, -D, -A, D+1, A+1, D-1, A-1, D+A, D-A, A-D, D&A, D|A, M, !M, -M, M+1, M-1, D+M, D-M, M-D, D&M, D|M

C-instruction

Forma generale:

`dest = comp ; jump`

("dest =" e ";" jump" sono opzionali → se sono nulli, non li scriviamo)

- **comp** specifica quale computazione eseguire
- **dest** specifica dove memorizzare il risultato di tale computazione
- **jump** specifica se, dopo aver eseguito `dest = comp` occorre fare un salto nel programma, ovvero aggiornare il contenuto del PC al valore attualmente salvato nel registro A, così che la prossima istruzione da eseguire non sia la successiva nel programma (ovvero all'indirizzo ROM immediatamente successivo a quello correntemente puntato da PC, cioè eseguendo $PC=PC+1$), ma sia invece l'istruzione salvata nella locazione ROM puntata dal registro A, cioè eseguendo $PC=A$.

I possibili valori di questi tre elementi:

comp = 0, 1, -1, D, A, !D, !A, -D, -A, D+1, A+1, D-1, A-1, D+A, D-A, A-D, D&A, D|A, M, !M, -M, M+1, M-1, D+M, D-M, M-D, D&M, D|M

dest = M, D, MD, A, AM, AD, AMD o nullo (in questo caso dest viene omissso)

jump = prendi comp, ma confrontala con 0 usando i bit di stato zr e ng della alu

- JGT (greater than) comp > 0
- JEQ (equal) comp = 0
- JGE (greater or equal) comp >= 0
- JLT (less than) comp < 0
- JNE (not equal) comp ≠ 0
- JLE (less or equal) comp <=0
- JMP (jump always)

o nullo (omesso)

Esempio

D=D+1; JLT

Questa istruzione corrisponde ad incrementare di 1 il contenuto del registro D, ovvero calcolare D=D+1 e, successivamente, se (D+1)<0 saltare nel programma all'istruzione presente all'indirizzo ROM pari al corrente valore di A.

Se jump non è nullo, di solito si testa il valore di D e non M/A

Un salto incondizionato si codifica con 0;JMP.

A-instruction

Forma generale

@value //A=value Dove value è un numero

Possiamo usarlo per caricare una costante da utilizzare in altre istruzioni.

Esempi

```
@17
D=A // ho salvato in D la costante 17
```

Possiamo usarlo per leggere un dato della RAM

```
@17 // A= 17
D = M // D = RAM[17]
```

Selezionare una locazione ROM per effettuare un salto nel programma: PC = A

```
@17 // A = 17
0;JMP // salva 17 in PC, quindi la prossima istruzione sarà ROM[17]; il salto è incondizionato
```

Esercizi semplici

Nota: nei commenti usiamo l'abbreviazione X+=Y al posto di X=X+Y, e similmente X-=Y per X=X-Y.

1. D=0

D=0

2. A=0

A=0 oppure @0

3. A=1 D=1

A=1 D=1 oppure AD = 1

4. D=3

@3

D=A

5. D=RAM[3]+3

@3

D = M

D = D + A

Non posso fare M + A perché la ALU oltre a D può prendere solo A o solo M.

6. D= 3+4

@3

D = A

@4

D=D+A

7. RAM[3]=7

@7

D=A

@3

M=D

8. D=RAM[A]+3

@3

D=A

D=M+D

1. D=D-1

D=D-1

2. D=D-3

@3

D=D-A

3. D=10-RAM[5]

@10

D=A

@5

D=D-M

4. RAM[0]=RAM[0]+2

@2

D=A

@0

M=M+D

Tipicamente prima settiamo il dato da scrivere e poi dove scriverlo.

5. D=RAM[RAM[0]]

@0

A=M

D=M

Fare AD=M non è equivalente: fa

A=RAM[0] e D=RAM[0]

6. RAM[RAM[0]] = RAM[0]

@0

A=M

M=A

Inizializzazione: non possiamo assumere che la memoria RAM sia vuota. Bisogna inizializzare a zero

1. RAM[2]=RAM[0]+RAM[1]

@2 // in caso RAM[2] non sia 0:

M=0 // RAM[2]=0

@0

D=M // D=RAM[0]

@2

M=D+M // RAM[2] += D

@1

D=M // D=RAM[1]

@2

M=D+M // RAM[2] += D

Oppure

D=0 // in caso D non sia uguale a 0

@0

D=D+M // D += RAM[0]

@1

D=D+M // D += RAM[1]

@2

M=D // RAM[2]=D

1. Se RAM[0]>0, goto istruzione con indirizzo memorizzato in RAM[1]

@0

D=M //D=RAM[0]

@1

A=M → **setto la destinazione del salto**

(che si riferisce ad A) !!! non scordare!!!

D;JGT → compara con D e poi va

all'indirizzo salvato in A. Scrivere

A;JGT non ha molto senso perché A è

sia il dato che viene comparato con zero,

sia la destinazione del salto.

2. D=D+1; successivamente, se D=0 goto istruzione 3

<pre> @3 D=D+1; JEQ 3. RAM[0] = RAM[5] e se RAM[5] <> 0 goto istruzione 3 @5 D=M //D=RAM[5] </pre>	<pre> @0 M=D //RAM[0]=RAM[5] @3 D;JNE </pre>
--	--

Etichette

Oltre alle A/C-instruction, possiamo definire etichette, ed usarle nelle A instruction come fossero valori

<pre> 0: @2 1: D=A (BACK) 2: @4 3: D=M 4: D=D+M 5: @2 6: D=D+A 7: @BACK 8: D;JGE </pre>	Dichiarazione di etichetta: dichiariamo un nome a piacimento per far riferimento all'indirizzo dell'istruzione <u>successiva</u> al punto di dichiarazione. In questo caso "BACK" vale 2 e quindi fa riferimento all'istruzione in ROM[2], ovvero "@4". Uso di etichetta: il simbolo "BACK" sarà sostituito (studieremo poi da chi e quando) con il valore dell'etichetta, cioè 2. Quindi nell'istruzione successiva, se il salto avverrà, sarà un salto all'istruzione in ROM[2] cioè "@4". Quindi invece di @BACK potremmo scrivere direttamente @2, ma le etichette ci evitano di dover gestire esplicitamente i numeri di riga (quindi indirizzi ROM) nei nostri programmi.
---	--

L'indentazione del programma è irrilevante.

L'etichetta non è scritta in ROM. Non viene codificata in binario.

If-then-else

Possiamo utilizzare il costrutto if-then-else, solitamente disponibile in linguaggi di alto livello, sfruttando i salti delle C-instruction e le etichette.

```

D=D-1 // o qualunque sia il dato da testare if (D-1>0) {
@CASO_FALSE // preparo destinazione salto
D;JLE //successo nel test: eseguo il blocco 1 e poi skippo il
blocco 2. test fallito: vado in else ed eseguo il blocco 2.
... blocco di codice 1...
@END
0;JMP // salto incondizionato: skippo il blocco 2 per andare fuori
dall'if }
(CASO_FALSE) else {
... blocco di codice 2.... }
(END)
... blocco di codice 3....

```

While

Idem per il costrutto while. Da notare che possiamo implementarlo sia come while-do sia come do-while. Qui vediamo il while-do. Per esercizio, implementa il do-while (il test va alla fine del corpo del ciclo, non all'inizio)

while (es. $D > 0$)

(LOOP)

@EXIT

D;JLE //testo la condizione

...blocco di codice 1...

@LOOP

0;JMP //ricomincia il ciclo

(EXIT)

... blocco di codice 2...

Esercizio stile esame

RAM[2] = RAM[0] × RAM[1]

Implementiamolo come $RAM[2] = RAM[0] + \dots + RAM[0]$ (RAM[1] volte)

NOTA: se moltiplico per una costante molto piccola, non c'è bisogno di un ciclo

@2 //inizializzo:

M=0 //RAM[2]=0

(LOOP)

@0

D=M //D=RAM[0]

@2

M=D+M //RAM[2]+= D, salvo in RAM[2]+=RAM[0] e ogni volta lo sommo di nuovo

@1

MD=M-1 //RAM[1]=RAM[1]-1 : usiamo RAM[1] come contatore dei cicli. Lo scrivo anche in D perché è D che posso testare, non M!

@LOOP

D;JGT //ricomincio il ciclo se RAM[1]>0

MD=M-1

Ricorda: queste assegnazioni sono concorrenti: $M=M-1$ e allo stesso tempo $D=M-1$. Avremmo potuto usare due istruzioni (in quale ordine?)

Esercizio difficile: $RAM[0..9] = [10, 9, 8, \dots, 1]$

Fine dei programmi

Di default, non esiste una “fine” del programma. Se non facessimo nulla per impedirlo, il PC verrebbe incrementato per sempre ogni volta che il processore ha finito di eseguire l'istruzione corrente (ovviamente, “per sempre” qui significa fino all'ultimo indirizzo della ROM...).

Per questo motivo è buona norma terminare ogni programma con un ciclo dummy che “previene” l'esecuzione di istruzioni scritte in locazioni ROM successive (e che in generale potrebbero appartenere ad un altro programma).


```

...
(END) // ciclo dummy finale
@END
0;JMP

```

Osservazione: comparazioni NON per 0

Nelle C-instruction scritte negli esempi fatti finora, abbiamo sempre avuto bisogno di comparare per 0 per stabilire se eseguire un salto. E, come visto, il linguaggio Assembly Hack considera sempre test per 0.

Ovviamente, questo non vuol dire che non possiamo comparare con valori arbitrari diversi da 0: prima di eseguire il test del salto, usiamo addizioni/sottrazioni nel modo opportuno.

Pseudocodice di esempio: // if RAM[2]>3 then RAM[5]=1 else RAM[5]=0

Un possibile programma (ne possiamo immaginare svariati):

```

@2
D=M
@3
D=D-A      // D=RAM[2]-3
@ELSE
D;JLE      // salta se D<=0, ovvero RAM[2]<=3
@5
M=1

```

```

@END
0;JMP
(ELSE)
@5
M=0
(END)
@END
0;JMP

```

In Hack si usano due tipi di simboli:

- **Etichette:** Già viste, usate per fare riferimento ad indirizzi della ROM. Dichiarate tramite direttiva (XXX) che definisce il simbolo XXX che farà riferimento all'indirizzo di ROM dell'istruzione successiva alla dichiarazione
- **Variabili:** Usate per far riferimento ad indirizzi in memoria (RAM, anche se non per forza).
 - pre-definite: ad esempio SCREEN e KBD che fanno riferimento agli indirizzi RAM 16384 e 24576 (indicano rispettivamente l'inizio della memoria per gestire lo schermo e la locazione dove viene inserito il codice di un eventuale tasto premuto su tastiera).
 - definite dall'utente: un simbolo non predefinito xxx usato in un programma Hack senza essere dichiarato usando (xxx) viene trattato come variabile, e gli viene assegnato in automatico un valore (a partire dal valore 16: indirizzi da 0 a 15 sono usati dalle variabili predefinite R0..R15).

Chi assegna i valori ai simboli? L'**assemblatore**, cioè il programma che traduce un programma Hack simbolico in binario. Posso salvare una variabile tramite una A-instruction. La variabile diventa semplicemente un riferimento a un indirizzo di memoria e l'assemblatore sceglierà dove salvarla da RAM[16] in poi.

Le variabili vanno sempre in minuscolo, le etichette sempre in maiuscolo.

Usare la variabili non aggiunge potere espressivo: la differenza è che non scelgo io la locazione di memoria in cui salvo un numero, ma lascio la decisione all'assemblatore. Se non so dove la

variabile è scritta, se devo sovrascrivere una locazione in RAM potrei finire per sovrascrivere una locazione con un dato salvato. Quindi, o uso sempre variabili, o non le uso mai.

Esercitarsi a fondo sulla scrittura di programmi in Assembly Hack. Solo dopo, considerare ciò: Abbiamo detto che esistono variabili predefinite e che queste fanno riferimento a (cioè il loro valore viene memorizzato in) locazioni di memoria predefinite. Per esempio le variabili R0..R15 fanno riferimento a RAM[0]...RAM[15]. Infatti scrivere @R0 equivale a scrivere @0. In termini pratici, le locazioni di memoria in questo intervallo sono riservate per essere utilizzate a piacimento dal programma in esecuzione, come memoria di lavoro. Non possiamo sapere invece in quali locazioni di memoria verranno memorizzate le variabili non predefinite ma definite da noi: l'assemblatore, durante la conversione da codice simbolico a codice binario, fisserà questa scelta. Per ora sappiamo solo che le variabili non predefinite fanno riferimento a locazioni dal 16 in poi. Per questo motivo, se per risolvere un esercizio ci troviamo a dover scrivere su RAM[16], RAM[17], etc..., ovvero includiamo istruzioni come ... @16 M=0 ... allora non dovremmo usare variabili nel nostro programma, perché corriamo il rischio di sovrascriverle accidentalmente. Per esempio, considera l'esercizio 3 nella slide "Ulteriori esercizi".

Esercizi Assembly

RAM[2] = max(RAM[0],RAM[1])

@2

M=0

@0

D=M

@1

D=D-M // se RAM[0]<RAM[1] allora compara la loro sottrazione con 0 e vedi se è minore di 0

@TRUE

D; JLT

@0

D=M

@2

M=D

@END // ciclo dummy per concludere il programma

0; JMP

(TRUE) //metto una sola etichetta per eseguire il ramo true. Se la condizione non si verifica, il codice semplicemente continua

@1

D=M

@2

M=D

(END)

@END

0; JMP

➔ con variabile:

@R0

D=M

@1

D=D-M // se RAM[0]<RAM1 allora compara la loro sottrazione con 0 e vedi se è minore di 0

@TRUE

D; JLT

@0

D=M

@max

M=D

@END // ciclo dummy per concludere il programma

0; JMP

(TRUE) //metto una sola etichetta per eseguire il ramo true. Se la condizione non si verifica, il codice semplicemente continua

@1

D=M

@max

M=D

(END)

@max

D=M

@2

M=D

(DUMMY)

@DUMMY

0;JMP

Note su CPU emulator.

- Le C-instructions operano sempre con la ALU.
- Quando assegno un valore al registro A, per la ALU sarà disponibile a partire dal ciclo di clock successivo (quindi dall'istruzione successiva).

- Quando faccio delle operazioni di assegnamento come

@0, M=D oppure @3, D= A

Sto dicendo alla ALU di far passare il valore così com'è, con input il registro A, D oppure M

- Se io scrivo 10 righe di codice e non metto il loop per concludere, il pc aumenta fino a che finisce di leggere tutti gli indirizzi ROM, anche se a vuoto, e alla fine mi dà errore.

// Scrivere il codice Assembly Hack che scriva in RAM[0] il minimo valore correntemente
// memorizzato nelle locazioni di memoria comprese tra RAM[5] e RAM[10]

// Notare che RAM[1..4] non vengono usati. Quindi se lo vogliamo possiamo usarli
// per memorizzare contatori o altri dati parziali. Per esempio, usiamo:

```
// -- RAM[3] per l'indirizzo correntemente considerato ("variabile indirizzo")
// -- RAM[0] per il minimo corrente (tra quelli visti finora)

// Quindi inizializziamo RAM[3]=5, copiamo RAM[0]=RAM[RAM[3]] (il primo numero è il
// minimo corrente), quindi cominciamo un ciclo in cui RAM[3] avrà valori 6,7,8,9,10.
// Ad ogni iterazione, leggiamo RAM[RAM[3]] e confrontiamo con RAM[0].
// Se RAM[RAM[3]]<RAM[0], aggiorniamo il minimo corrente.
// Terminiamo uscendo dal ciclo quando RAM[3]>10.

// Nota: in questa soluzione non si usano variabili, ma essendo certi di non dover scrivere
// la locazione RAM[16] e successive, potremmo anche usarle.
// Nota: se non vogliamo usare RAM[3], possiamo incrementare direttamente RAM[1] come
// contatore. Questo è anche l'approccio seguito nella soluzione all'esercizio v2.

// testare settando a mano RAM[5..10] nell'interfaccia dell'emulatore
```

```
@5
D=A          //5 è il primo indirizzo da considerare
@3
AM=D         //lo memorizziamo in RAM[3], ed anche in A
D=M          //D=RAM[5]
@0
M=D          //minimo corrente (cioè RAM[5]) copiato in RAM[0]
```

(LOOP)

```
@3
D=M
@10
D=D-A        //D=RAM[3]-10
@END
D;JGE        //abbiamo finito quando RAM[3]>=10, ovvero D>=0. altrimenti si continua
```

```
@3
M=M+1        //RAM[3]++ incrementiamo l'indirizzo all'inizio del ciclo LOOP
```

```
@3
A=M
D=M          //D=RAM[RAM[3]]
```

```
@0
D=D-M        //D= RAM[RAM[3]] - RAM[0]
@LOOP
D;JGE        //RAM[RAM[3]]>=RAM[0] se D>=0: in questo caso non aggiorniamo il minimo
```

```

@3
A=M
D=M //D=RAM[RAM[3]]
@0
M=D      //altrimenti aggiorniamo il minimo corrente

```

```

@LOOP
0;JMP    //e ricominciamo il ciclo

```

(END)

```

@END
0;JMP

```

//Si scriva il programma in Assembly Hack che scrive nelle celle di memoria da MEM[10] all'indirizzo salvato in //MEM[1] i numeri dispari crescenti a partire da 2MEM[0]+1. Assumere MEM[1]>=10.

//Ad esempio, se MEM[0] vale 5 scriverà i numeri 11, 13, 15...

```

MEM[2] = 10
MEM[2] = MEM[2] + 1
MEM[10] = 2MEM[0]+1
MEM[11] = MEM[10] + 1
MEM[11] = MEM[11] + 1
...
MEM[MEM[1]] = MEM

```

```

@10
D=A //D= 10
@2
M=D //RAM[2] = 10
A=M //A = RAM[2]

```

//MEM[0] = 2*MEM[0]+1

```

@0
D=M //D=RAM[0]
D= D+M //D= 2RAM[0]
D= D+1 // 2RAM[0] + 1
M=D

```

(WHILE)

```

@R1
D=M //D= RAM[1]
@R2
D=D-M // D= RAM[1] - RAM[2]
@END
0;JLT

```

```

//MEM[MEM2]=MEM[0]
@0
D=M
@2
A=M
M=D

//MEM[0]+=2
@2
D=A
@0
M=M+D

//MEM[2]++
@2
M=M+1
@WHILE
0; JMP

(End)

```

Tutti gli esercizi Assembly Hack

- D=0
- A=0
- A=1, D=1
- D=3
- D=RAM[3]+3
- D=3+4
- RAM[3]=7
- D=RAM[A]+3
- D=D-1
- D=D-3
- D=10-RAM5
- RAM[0]=RAM[0]+2
- D=RAM[RAM[0]]
- RAM[RAM[0]]=RAM[0]
- RAM[2]=RAM[0]+RAM[1]
- Se RAM[0]>0, goto istruzione con indirizzo memorizzato in RAM[1]
- D=D+1; successivamente, se D=0 goto istruzione 3
- RAM[0] = RAM[5] e se RAM[5]≠0 goto istruzione 3
- RAM[2] = RAM[0] × RAM[1]
- RAM[0..9] = [10, 9, 8,..., 1] ovvero RAM[0]=10, RAM[1]=9, ...
- RAM[2] = max(RAM[0],RAM[1])
- RAM[10] = max(RAM[9],RAM[8],...,RAM[1],RAM[0])

- $RAM[2] = RAM[1] - RAM[0] - 2$
- $RAM[2] = RAM[1] \text{ nand } RAM[0]$ //nand bit a bit
- Scrivere il valore 1 in tutte le celle di memoria con indirizzo compreso tra $RAM[0]$ e $RAM[1]$, assumendo $RAM[1] > RAM[0] > 1$
- $RAM[2] = 2 \times RAM[2]$ // qualche slide più su abbiamo già visto la moltiplicazione arbitraria. In questo caso, basta qualcosa di più semplice?
- $RAM[2] = RAM[1] \times (2^{RAM[0]})$
- $RAM[2] = RAM[1] / RAM[0]$, $RAM[3] = RAM[1] \bmod RAM[0]$ //divisione intera
(Dei prossimi, soluzione su virtuale [es. 20/11])
- $RAM[2] = \max(RAM[0], RAM[1])$
- scrivere soli 1 (cioè 16 1) dall'indirizzo in $R0$ all'indirizzo in $R1$, assumendo $R1 \geq R0$.
assunzione: $R0 > 1$
- Scrivere il codice Assembly Hack che scriva in $RAM[0]$ il minimo valore correntemente memorizzato nelle locazioni di memoria comprese tra $RAM[5]$ e $RAM[10]$

Notare che $RAM[1..4]$ non vengono usati. Quindi se lo vogliamo possiamo usarli per memorizzare contatori o altri dati parziali. Per esempio, usiamo:

- $RAM[3]$ per l'indirizzo correntemente considerato ("variabile indirizzo")
- $RAM[0]$ per il minimo corrente (tra quelli visti finora)

Quindi inizializziamo $RAM[3]=5$, copiamo $RAM[0]=RAM[RAM[3]]$ (il primo numero è il minimo corrente), quindi cominciamo un ciclo in cui $RAM[3]$ avrà valori 6,7,8,9,10.

Ad ogni iterazione, leggiamo $RAM[RAM[3]]$ e confrontiamo con $RAM[0]$.

- Se $RAM[RAM[3]] < RAM[0]$, aggiorniamo il minimo corrente. Terminiamo uscendo dal ciclo quando $RAM[3] > 10$.
- Scrivere il codice Assembly Hack che scriva in $RAM[0]$ il minimo valore correntemente memorizzato nelle locazioni di memoria comprese dall'indirizzo memorizzato in $RAM[1]$ fino all'indirizzo in $RAM[2]$, ovvero: $RAM[0] = \min(RAM[RAM[1]], \dots, RAM[RAM[2]])$.
Assumere $RAM[2] \geq RAM[1] > 1$
- i, j, k sono interi memorizzati in $RAM[0]$, $RAM[1]$, $RAM[2]$, rispettivamente. considera il seguente pseudocodice:

```

j=1
while( j <= i ) {
    k = k + 1
    j = j + 3
}
k = j + i

```

Scrivi il codice HACK corrispondente. esegui, per esempio, con gli indirizzi $RAM[0-2]$ inizializzati a 9,2,3. l'algoritmo termina con valori 9,10,6. non è obbligatorio in questo caso usare variabili i, j, k : è possibile mantenere il loro valore semplicemente in $RAM[0..2]$. Usare variabili rende il codice più facilmente confrontabile con lo pseudocodice, ma richiede di copiare in $RAM[0..2]$ il loro valore prima che il programma termini.

- Dividere $RAM[0]$ per $RAM[1]$ e scrivere il quoziente in $RAM[2]$ ed il resto in $RAM[3]$.
Assumere $RAM[1] > 0$, $RAM[0] > 0$

Nota: nello pseudo-codice, scriviamo per brevità " $R1$ " invece di $RAM[1]$, etc. Da non confondere con le variabili predefinite $R0..R15$, che valgono $0..15$

inizializzazione: R2, R3 = 0

```
while( R0 > 0 ) {  
    R0 = R0 - R1  
    if(R0 < 0)  
        R3 = R1 + R0  
    exit (fine programma)  
    R2++  
}
```

- Si scriva un programma in assembly HACK che scrive nelle celle di memoria da MEM[10] all'indirizzo salvato in MEM[1] i numeri dispari crescenti a partire da 2MEM[0]+1. Assumere MEM[1]>=10.

Ad esempio, se MEM[0] vale 5 scriverà i numeri 11,13,15,...

MEM[2]=10. indirizzo dove scrivere
MEM[0] = 2 * MEM[0]+1. dato da scrivere

```
while(MEM[2]<=MEM[1])  
{  
    MEM[MEM[2]] = MEM[0]  
    MEM[0]+=2  
    MEM[2]++  
}
```

- moltiplicazione di x,y usando somme. input x in RAM[1], y in RAM[2], il risultato va in RAM[3]. usiamo x (ma si poteva usare anche y) come contatore da decrementare. per esempio $2 \times 3 = 3 + 3$

```
x=RAM[1]  
y=RAM[2]  
RAM[3]=0  
while(x > 0) {  
    RAM[3] += y  
    x--  
}
```

Realizzazione del Computer Hack

Componenti principali

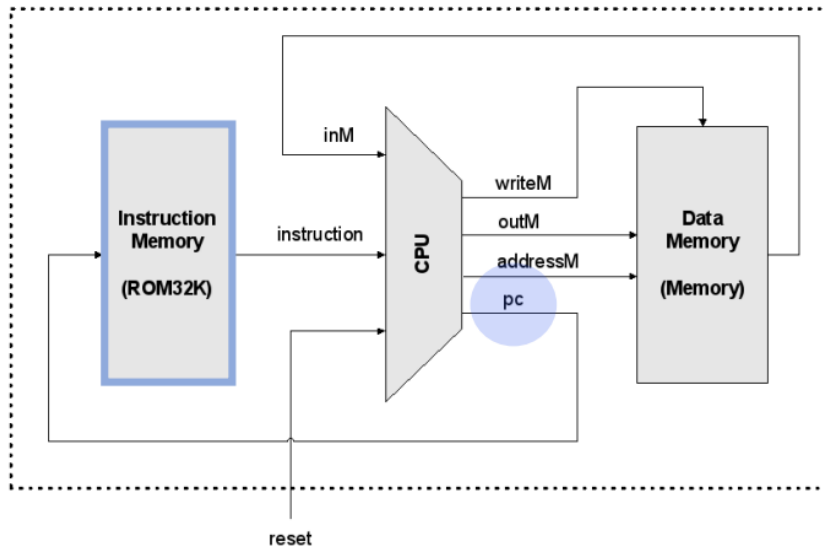
- ROM
- Memoria Dati
 - Memory
 - Dati (RAM)
 - Tastiera
- CPU

La separazione tra memoria dati e memoria programma semplifica di molto il processo, permettendo di eseguire in un solo ciclo di clock il tutto.

Il display è 256x512. È progettato per eseguire programmi scritti nel linguaggio macchina Assembly Hack (binario).

Memoria per le istruzioni (ROM)

- La ROM (un chip built-in) è precaricata con un programma scritto nel linguaggio Assembly Hack
- Dato un indirizzo ROM address a 15-bit (proveniente dal PC), emette un numero a 16-bit che codifica in binario l'istruzione corrente da eseguire, ovvero **out** = ROM32K[address]



Chip Memory

Una precisazione: finora, per semplicità, ci siamo sempre riferiti a questo componente come memoria RAM (ed infatti, anche programmando in Assembly, ci siamo sempre riferiti ad un indirizzo di memoria k come RAM[k]).

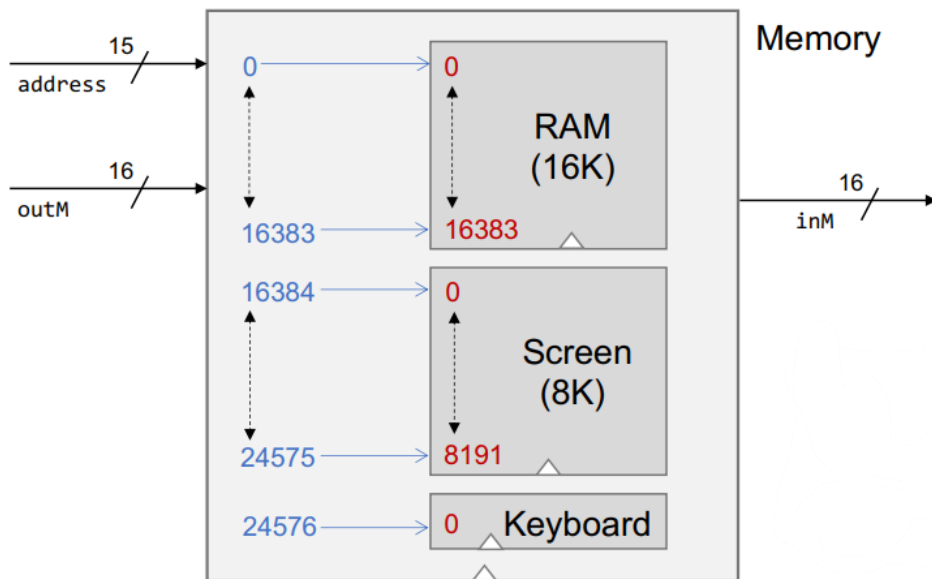
In realtà, come vedremo ora, il chip Memory è composto da 3 componenti per una dimensione totale di 24K:

- Il chip RAM (16K): la memoria RAM di lavoro vera e propria
- Il chip Screen (8K): memoria tipo RAM usata per la mappa dello schermo
- Un registro Keyboard: un singolo registro per leggere il tasto premuto

Per questo motivo, quando necessario, possiamo riferirci al registro Memory con indirizzo k come MEM[k], senza specificare se si tratti di memoria dati RAM, di indirizzo relativo allo schermo, o alla tastiera.

In blu: lo spazio degli indirizzi del chip Memory (univoco: da 0 a 24576).

In rosso: gli spazi di indirizzamento di ciascun chip interno.



Ricordare le due variabili predefinite in Hack: SCREEN=16384, KBD=24576. Screen ha quel valore perché corrisponde al primo indirizzo di screen, idem per keyboard.

Domanda: un indirizzo a 15 bit può indirizzare fino a $2^{15}-1=32767$. Cosa accade agli indirizzi mancanti dalla figura (esprimibili in binario, ma non corrispondenti a chip fisici)? Sono rappresentabili, ma non esiste un chip fisico che corrisponde ad essi, quindi non si possono utilizzare. Non si implementa per semplificare l'hardware; se utilizzassimo 14 bit, non sarebbero abbastanza.

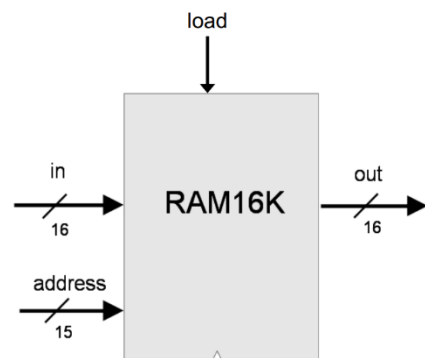
Memoria per i dati (RAM)

Per leggere RAM[k] :

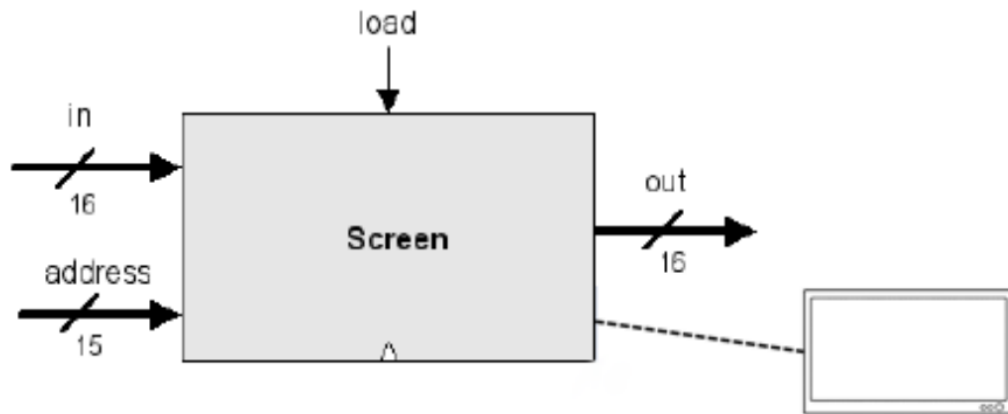
- portare k sul bus indirizzi
- leggere out

Per scrivere RAM[k] = value :

- portare k sul bus address
- portare value sul bus in
- portare 1 su load
- attendere un ciclo di clock



Screen



Lo schermo ha una funzionalità di tipo RAM:

Logica di lettura:

- $\text{Out} = \text{Screen}[\text{address}]$

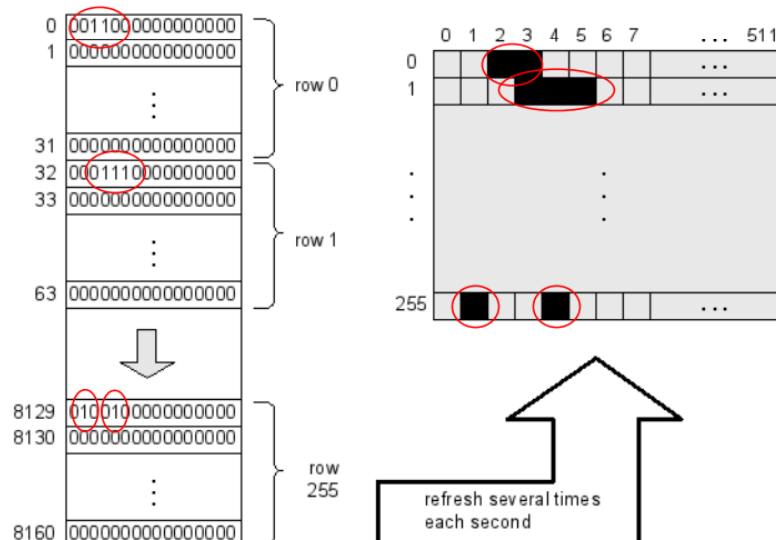
Logica di scrittura

- If load then $\text{Screen}[\text{address}] = \text{in}$

Esiste una corrispondenza diretta fra ogni pixel del monitor e i corrispondenti bit memorizzati ad opportuni indirizzi all'interno del chip Screen.

Il chip Screen contiene 8K parole da 16 bit, che corrispondono a 16 pixel. Un bit a 1 indica che il corrispondente pixel deve essere nero, 0 indica invece bianco.

$512/16 = 32 \rightarrow$ le prime 32 parole (da 0 a 31, poi da 32 a 63 ecc) corrispondono alla prima riga dello schermo.



Per impostare il pixel (row,col) dello schermo per essere nero o bianco:

Settare il bit $\text{col} \% 16$ della parola alla locazione $\text{Screen}[\text{row} * 32 + \text{col} / 16]$, dove $\text{col} / 16$ è una divisione intera, a 1 (nero) oppure 0 (bianco)

Una scheda grafica invece è un coprocessore. Ha un'architettura e tipi di calcoli diversi (vettori, traslazioni ecc).

Disegna sullo schermo un rettangolo largo 16 pixel e alto MEM[0] pixel

<pre> @0 D=M // D=MEM[0] @INFINITE_LOOP D;JLE // if D>0 then @counter M=D // counter=D @SCREEN // predefinita D=A @address M=D // address=SCREEN (LOOP) @address A=M // A=address M=-1 // MEM[A]=111...1 @32 D=A @address M=M+D // address+=32 </pre>	<pre> @counter MD=M-1 // counter--, scrivo anche D per test @LOOP D;JGT // ciclo se counter<=0 (END) @END 0;JMP </pre>
---	---

vedi file rect.asm

MEM[0] corrisponde a RAM[0]

Vai nel chip Screen, e negli indirizzi corrispondenti alla prima parola scrivi a tutti 1, con adress corrispondente alla variabile screen che è il primo indirizzo del chip → M=-1 mette tutti i bit di quella parola a 1. Per settare la prima colonna, aggiungo 32 alla variabile screen per saltare alla riga successiva e fare la stessa identica cosa.

Keyboard

Chip Keyboard: un unico registro da 16-bit

Out: la codifica ASCII (estesa a 16-bit) del tasto correntemente premuto, oppure 0 se non si preme alcun tasto (esistono alcuni tasti speciali, vedi sotto)

Key pressed	Keyboard output	Key pressed	Keyboard output
newline	128	end	135
backspace	129	page up	136
left arrow	130	page down	137
up arrow	131	insert	138
right arrow	132	delete	139
down arrow	133	esc	140
home	134	f1-f12	141-152

Per leggere dalla tastiera: leggere il contenuto del chip Keyboard.

Nota: il registro non accetta valori in scrittura: possiamo solo leggere

Esempio di uso della tastiera in linguaggio Hack

Colora progressivamente lo schermo di nero se si tiene premuto un tasto. Logica: iteriamo (LOOP) su tutte le locazioni della mappa dello schermo, dalla prima all'ultima; se durante l'iterazione un tasto è premuto allora scriviamo -1 (16 '1') all'indirizzo corrente, altrimenti 0 (16 '0'). Quando il ciclo raggiunge l'ultima locazione della mappa (**) ricominciamo il programma puntando di nuovo alla prima locazione.

<pre>(START) @SCREEN // 16348 D=A @i M=D // i punta a 1a word della mappa (LOOP) // while(true) @i D=M @24575 // ultima word della mappa D=A-D // D=24575-i @START D;JLT // (**) se D<0 ricomincia @KBD D=M @COLORANERO D;JNE // salta se un tasto è premuto</pre>	<pre>@i D=M A=D M=0 // MEM[i]=00...0 (bianco) @CONTINUE 0;JMP (COLORANERO) @i D=M A=D M=-1 // MEM[i]=11...1 (nero) (CONTINUE) @i M=M+1 // i++: prossimo indirizzo @LOOP 0;JMP // fine while</pre>
---	--

C'è un ciclo infinito che itera lungo le parole nella mappa dello schermo.

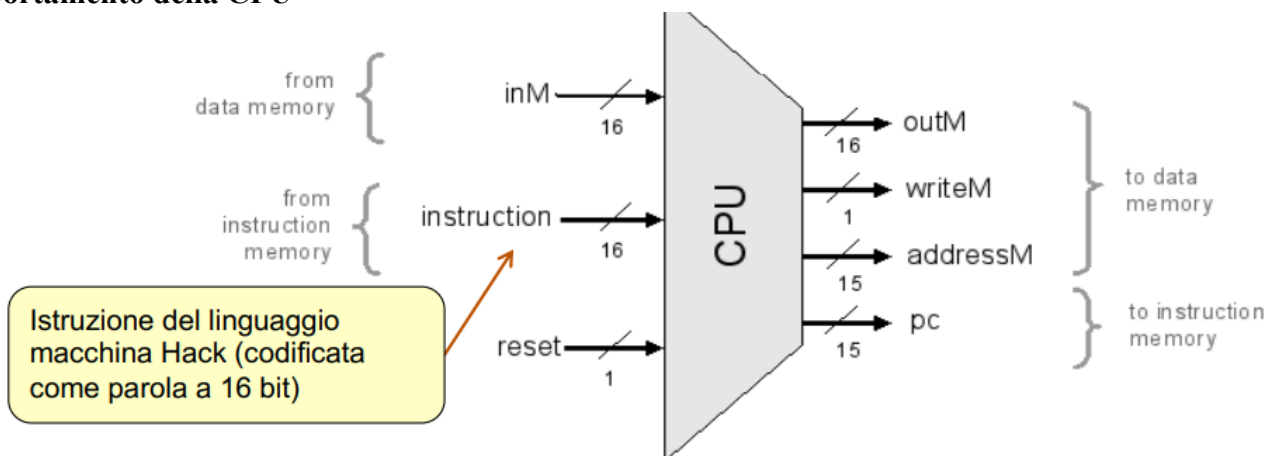
Se un tasto viene premuto, scrive 1 sulla parola corrente. Se non premo non colora.

Il ciclo parte da screen e arriva fino all'ultima word della mappa.

Se il tasto è premuto, salta e mette la locazione corrente a -1.

Altrimenti, continua e va all'indirizzo successivo.

Comportamento della CPU



Componenti interni della CPU: ALU e 3 registri: A, D, PC La CPU esegue la A/C-instruction seguendo la specifica del linguaggio Hack:

inM viene dalla RAM, instruction viene dalla ROM

write M dice di scrivere il valore outM all'indirizzo adressM. writeM diventa il load di data memory

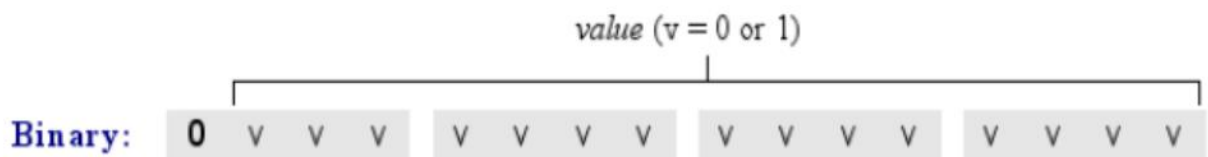
pc va a ROM

- I valori D e A, se presenti nell'istruzione, sono letti/scritti da/in A, M
- Il valore M, se presente nella parte destra dell'istruzione, viene letto da inM
- Se la parte sinistra dell'istruzione contiene M, l'output della ALU viene posto su outM, il valore di A viene posto su adressM, e writeM viene settato ad 1

Politiche di caricamento delle istruzioni: Se si tratta di una C-instruction (dest=comp;jump), instruction può includere un salto; nel caso di salto condizionato la condizione dipende dai due bit di controllo della ALU (zr e ng), indicanti se l'output della ALU è zero o negativo.

Quindi: Se reset==0: la CPU controlla se va fatto il jump. In caso di jump si carica il contenuto di A in PC (PC=A); altrimenti, si incrementa il registro (PC=PC+1) Se reset==1: si mette 0 in PC (si fa ripartire il computer)

Symbolic: @value // Where value is either a non-negative decimal number
// or a symbol referring to such number.

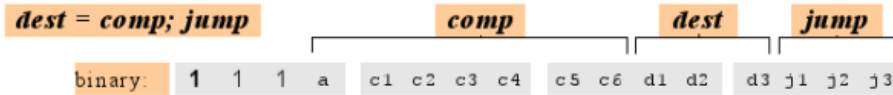


Il primo bit, uguale a 0, indica che si tratta di una A-instruction: quindi il numero “value” viene codificato in binario dai successivi 15 bit.

Questo spiega perché gli indirizzi sono a 15 bit e non 16 bit (vedi le interfacce dei chip nelle slide precedenti). Range di valori codificabile in value: i numeri non negativi che si possono codificare in complemento a 2 con 16 bit (i numeri negativi hanno infatti il bit più significativo ad 1).

Quando usiamo A come indirizzo, il primo bit non può essere 1 perché non esistono indirizzi negativi. Con 16 bit, abbiamo un bit sempre a 0 e 15 bit che codificano; ma siccome il primo bit è sempre a 0, possiamo anche non includerlo.

Le C-instruction invece hanno come bit più significativo a 1.



(when a=0) <i>comp</i>	c1	c2	c3	c4	c5	c6	(when a=1) <i>comp</i>	d1	d2	d3	Mnemonic	Destination (where to store the computed value)
0	1	0	1	0	1	0		0	0	0	null	The value is not stored anywhere
1	1	1	1	1	1	1		0	0	1	M	Memory[A] (memory register addressed by A)
-1	1	1	1	0	1	0		0	1	0	D	D register
D	0	0	1	1	0	0		0	1	1	MD	Memory[A] and D register
A	1	1	0	0	0	0	M	1	0	0	A	A register
!D	0	0	1	1	0	1		1	0	1	AM	A register and Memory[A]
!A	1	1	0	0	0	1	!M	1	1	0	AD	A register and D register
-D	0	0	1	1	1	1		1	1	1	AMD	A register, Memory[A], and D register
-A	1	1	0	0	1	1	-M					
D+1	0	1	1	1	1	1						
A+1	1	1	0	1	1	1	M+1					
D-1	0	0	1	1	1	0						
A-1	1	1	0	0	1	0	M-1					
D+A	0	0	0	0	1	0	D+M					
D-A	0	1	0	0	1	1	D-M					
A-D	0	0	0	1	1	1	M-D					
D&A	0	0	0	0	0	0	D&M					
D A	0	1	0	1	0	1	D M					

j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	Mnemonic	Effect
0	0	0	null	No jump
0	0	1	JGT	If out > 0 jump
0	1	0	JEQ	If out = 0 jump
0	1	1	JGE	If out ≥ 0 jump
1	0	0	JLT	If out < 0 jump
1	0	1	JNE	If out ≠ 0 jump
1	1	0	JLE	If out ≤ 0 jump
1	1	1	JMP	Jump

Ogni bit in realtà ha un suo ruolo. Facendo un circuito combinatorio generiamo dei segnali di controllo.

Es. se il bit d3 è 1, stiamo scrivendo in memoria.

Assemblatore

L'assembler si occupa della traduzione del linguaggio assembly (simbolico) al relativo codice in linguaggio macchina (binario). Nel caso del linguaggio Hack, ogni istruzione in linguaggio assembly viene tradotta in un codice a 16 bit, ad eccezione delle direttive di dichiarazione di etichette, che semplicemente definiscono dei simboli e che non vengono quindi tradotte.

- In input all'assemblatore forniamo un file in linguaggio assembly Hack (estensione .asm)
- in output l'assemblatore genera un file contenente per ogni riga un numero binario a 16 bit scritto in ASCII (estensione .hack)

Nota: In un sistema reale, l'assemblatore non scriverebbe in ASCII ma in binario. Qui invece viene usato un formato testuale per permetterci comunque di aprire ed ispezionare i file codificati, e di caricarli nel CPUEmulator.

Programma assembly (o "programma ISA") composto da:

- A-instructions
- C- instructions
- dichiarazioni ed etichette

Ulteriori righe possono essere commenti, spazi e righe vuoti, simboli di tabulazione '\t', o simboli di fine linea come '\r' o '\n':

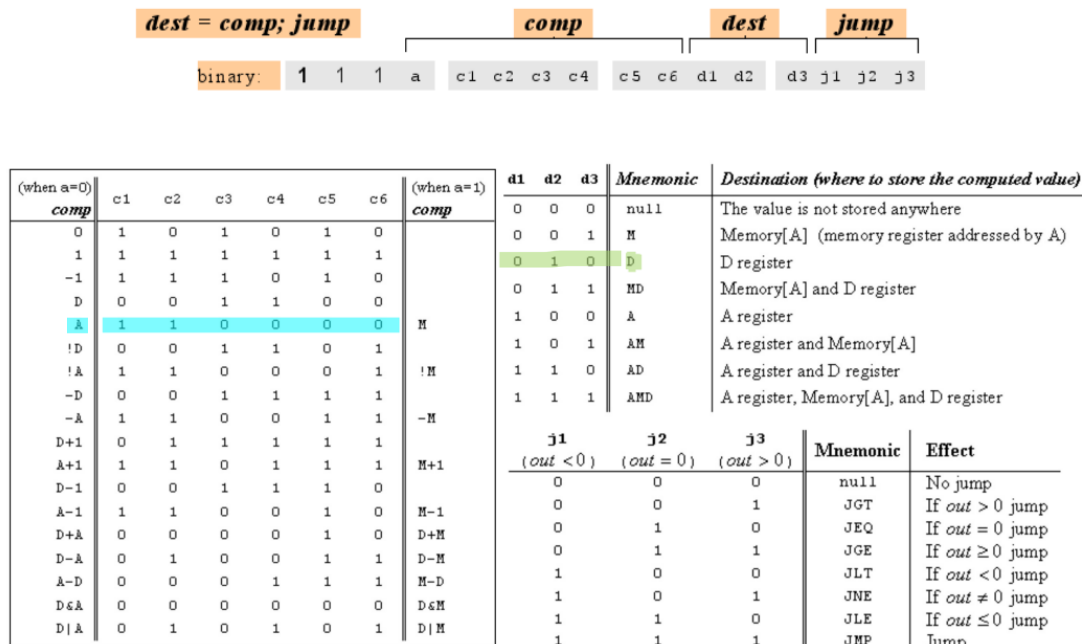
Il nostro obiettivo è programmare l'assemblatore.

Approccio di traduzione in codice macchina:

- Se value è un numero decimale, basta considerare la rappresentazione binaria di value a 15 bit: è un indirizzo sempre non negativo.
Ad esempio @3 → 0000000000000011

- Se value è un simbolo: vedi sotto.

Generare il codice assembly relativo alle A/C-instructions mostrate (non è es. di esame, solo per progetto):



@7 → 0000000000000111

D=A → 111011000010000

I primi tre bit sono sempre 1. Il jump è nullo (ultimi 3 bit). Il bit a (il quarto) è 0, perché il bit A ci dice se parliamo di A oppure di M. Siccome parliamo di A, lo mettiamo a 0.

A=-1 → 111011101010000

M=1 → 111011111001000

D=D+A → 1110000010010000

D=D&A → 111000000010000

@11

D=-D; JGT → 1110001111010001

I simboli sono usati per identificare indirizzi di istruzioni in ROM (destinazione dei salti) oppure indirizzi in memoria RAM (variabili).

Sono costituiti da questi possibili caratteri:

- Lettere minuscole (comprese fra 'a' e 'z')
- Lettere maiuscole (comprese fra 'A' e 'Z')
- Cifre decimali (comprese fra '0' e '9')
- Simboli particolari: '_', '.', '\$'

Il primo carattere di un simbolo non può essere una cifra decimale

In pratica: se in un programma ISA in input leggiamo una istruzione che inizia con "@" seguito da una cifra, allora sappiamo che si tratta di una normale A-instruction, altrimenti di un simbolo. Per capire di quale simbolo si tratti (etichetta/variabile), vedremo fra poco.

Alcuni simboli sono predefiniti, e hanno già un valore assegnato di default. Ad esempio:

R0 → 0

R1 → 1

Fino a R15 → 15

SCREEN → 16384

KBD → 24576

Altri sono predefiniti

SP → 0

LCL → 1

ARG → 2

THIS → 3

THAT → 4

Registri virtuali:

I simboli R0,..., R15 sono automaticamente predefiniti per far riferimento agli indirizzi di memoria RAM 0,...,15

Puntatori per I/O:

I simboli SCREEN and KBD fanno riferimento agli indirizzi RAM 16384 e 24576, rispettivamente (indirizzi base della mappa in memoria dello schermo, e della tastiera)

Puntatori di controllo della Virtual Machine:

I simboli SP, LCL, ARG, THIS, and THAT sono automaticamente predefiniti per far riferimento agli indirizzi di RAM 0, 1, 2, 3, e 4, rispettivamente

Etichette:

Usate per identificare i punti di destinazione delle istruzioni di salto (vedi LOOP e END nell'esempio). Definite tramite la direttiva (YYY) che definisce l'etichetta YYY a cui verrà assegnato come valore l'indirizzo di memoria ROM in cui verrà caricata la prima istruzione successiva alla direttiva (YYY)

Variabili:

Usate nelle A-instruction per identificare celle di memoria RAM da dedicare a specifiche variabili (vedi counter e x nell'esempio). Riceveranno un valore che va dal 16 in poi. I valori vengono assegnati seguendo l'ordine in cui tali variabili appaiono. Nell'esempio counter varrà 16, mentre x varrà 17

Per la gestione e traduzione dei simboli, creiamo una tabella dei simboli (symbol table), che contiene per ogni simbolo (predefinito o no) il valore corrispondente. Quanto detto finora comporta che:

Primo passo:

Possiamo creare una tabella di simboli inizializzandola con tutte le variabili predefinite.

Secondo passo:

Poi si scansiona tutto il codice per completare la tabella calcolandone il valore. Si scorre l'input per inserire in symbol table le etichette incontrate, tenendo un contatore del numero di A/C-instruction viste finora (iniziando dalla numero 0). Quando si incontra una definizione di etichetta le si assegna il valore di tale contatore (più 1) in quel momento (e non si incrementa il contatore: non è una A/C-instruction!).

Appena incontro un'etichetta, devo assegnare il valore del contatore delle istruzioni prima e assegnarglielo (+1). → (solo quelle dichiarate, non quelle con @!!)

Terzo passo:

Si apre in scrittura il file di output .hack e si scorre di nuovo l'input:

per A- e C-instructions si scrive in output il relativo codice. In particolare:

- se una A-instruction usa un simbolo (vedi prima su come riconoscerli): si cerca il valore in symbol table; se il simbolo è assente (quindi non è una etichetta, ma una variabile) si assegna un valore progressivo, partendo da 16, e lo si memorizza in symbol table.

- Se presente (o se appena inserito), si usa questo valore per generare in output il codice dell'istruzione.

```

0      @i
1      M=1    // i = 1
2      @sum
3      M=0    // sum = 0
      (LOOP)
④      @i    // se i>RAM[0] goto WRITE
5      D=M
6      @R0
7      D=D-M
8      @WRITE
9      D;JGT
10     @i    // sum += I
11     D=M
12     @sum
13     M=D+M
14     @i    // i++
15     M=M+1
16     @LOOP // ricomincia ciclo
17     0;JMP
      (WRITE)
⑱      @sum
19     D=M
20     @R1
21     M=D    // RAM[1] = sum
      (END)
⑳      @END
23     0;JMP

```

SYMBOL TABLE

//inizializzazione

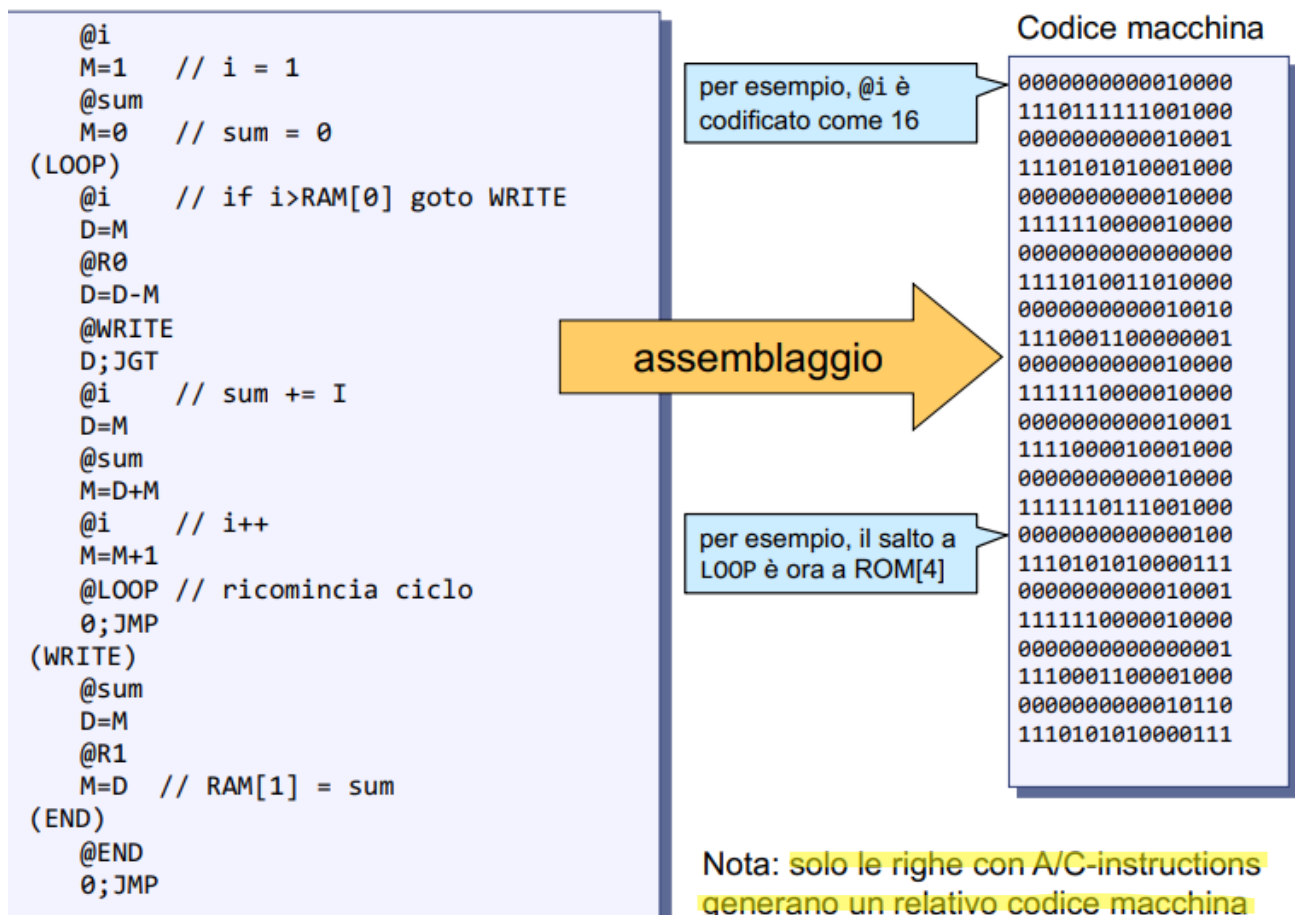
R0	1
...	
R15	15
SCREEN	
16384	
KBD	24576
SP	0
LCL	1
ARG	2
THIS	3
THAT	4
//primo giro	
LOOP	4
WRITE	18
END	22
//Seconda passata	
i	16
sum	17

i → uso il primo valore a

partire dal 16 compreso

sum → gli assegno il valore

dopo, 17



Altri tipi di ISA

Parliamo ora di altri tipi di ISA più avanzate, a livello più generico, considerando anche gestioni della memoria più avanzate.

Il livello ISA rappresenta l'interfaccia fra l'hardware ed il software.

Comprende non solo l'istruzione set, ma anche tutti i dettagli e componenti dell'architettura che sono "visibili" al compilatore, quindi necessari per generare codice macchina eseguibile dal computer in uso.

Possiamo in breve dire che un programma ISA è l'output della compilazione di un programma di alto livello.

È fondamentale conoscere l'ISA perché determina il tipo di istruzioni eseguibili in base al tipo di macchina in cui devono essere compilate.

Noi abbiamo già studiato il formato delle istruzioni per l'ISA di Hack, che sono codificati con 16 bit.

Nel caso di Hack, le due semplici tipologie di istruzione sono pensate per facilitare l'implementazione (vedi progetto 4): A-istruzione (formato semplice) e C-istruzione (formato "strutturato", organizzato in modo tale da semplificare l'implementazione logico-digitale, ad esempio, per controllare la ALU direttamente con i bit dell'istruzione).

In un generico computer, le istruzioni ISA contengono:

- Codice operativo, detto opcode: parte che indica il tipo di istruzione da eseguire (somma, and, etc..).
- Gli indirizzi (più di uno, nei linguaggi che lo supportano) che indirizzano, in vario modo (vedi dopo), gli operandi da usare

A differenza della rappresentazione “strutturata” usata per Hack:

- istruzioni diverse potrebbero dedicare più o meno bit alla parte dedicata alla codifica in binario del codice operativo.
- istruzioni diverse potrebbero avere un numero diverso di indirizzi (cioè operandi), a seconda della computazione che specificano.

Più bit di opcode consentono di codificare un numero maggiore di operazioni possibili, limitando il possibile indirizzamento degli operandi.

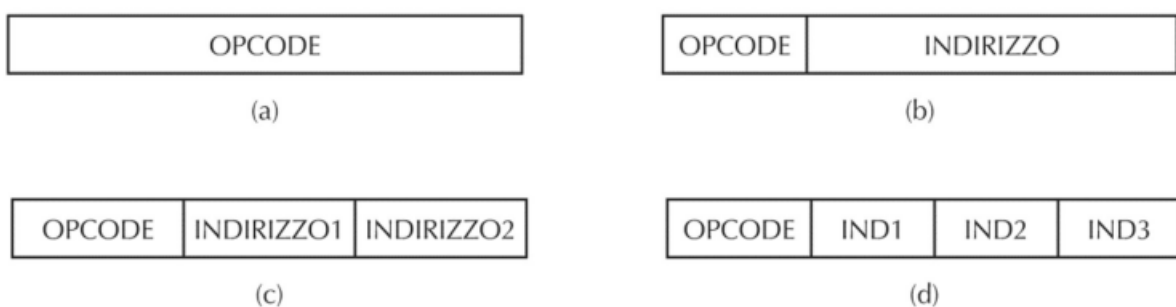
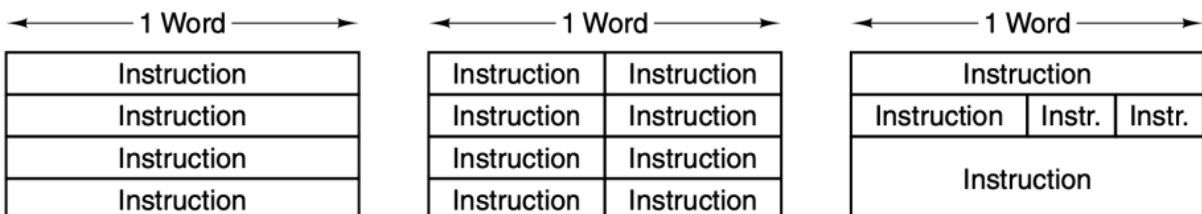


Figura 5.9 Quattro formati d'istruzione: (a) Istruzione senza indirizzi. (b) Istruzione a un indirizzo. (c) Istruzione a due indirizzi. (d) Istruzione a tre indirizzi.

Se mantengo pochi bit per l'indirizzo, lo costringo ad essere ridotto. Un opcode troppo piccolo codifica poche istruzioni. Questa cosa deve essere calibrata con la grandezza della memoria. Possiamo anche fare scelte diverse relative alla dimensione dell'intera istruzione, in relazione alla dimensione della word.



Istruzioni più piccole occupano meno memoria, ma hanno opcode (quindi instruction set) e indirizzi ridotti, e possono richiedere una fase di decodifica più complessa.

In Hack l' instruction set è molto ridotto, ma utilizzare 16 bit per la codifica strutturata (ma meno sintetica) delle C-instructions ci ha consentito una decodifica ed implementazione più facile (e veloce, in esecuzione).

In un architettura come i-7, tutte le parti hanno utilizzi diversi e scopi diversi, dunque anche strutture diverse.

La parte “indirizzi” indica dove reperire gli operandi da usare. Esistono diversi modi per indicare come reperire tali operandi

- Indirizzamento **immediato**: L'istruzione include già l'operando da usare (ad esempio le A-instruction in Hack: @value specifica già il valore da memorizzare in A)

- Indirizzamento **diretto**: L'istruzione contiene l'indirizzo completo di memoria della cella in cui reperire l'operando (questo tipo non è possibile in Hack in una sola istruzione: equivarrebbe a scrivere per esempio $D = \text{RAM}[1024]$. Nella stessa istruzione deve andare a recuperare il valore della RAM e scrivere in D)
- Indirizzamento **a registro**: L'operando viene prelevato da un registro (possibile in Hack, per A/D)
- Indirizzamento **a registro indiretto**: L'operando viene prelevato dalla memoria, all'indirizzo puntato da un registro (simile ad Hack, che però usa solo il registro A → vai a prendere il dato all'indirizzo puntato da A)
- Indirizzamento **indicizzato**: L'istruzione include uno "spiazzamento" (offset) che viene sommato al contenuto di un registro per ottenere l'indirizzo di memoria da cui prelevare l'operando. Leggo il contenuto di un indirizzo in un registro, e poi aggiungo un valore (detto offset). Il valore cercato ad esempio è il valore contenuto nel registro A + 10.
- Indirizzamento **a stack**: Si considera una parte di memoria da gestire secondo un modalità LIFO (last in-first out), ovvero tramite uno stack (in italiano: pila). Si usa un registro interno dedicato, solitamente chiamato "Stack Pointer". Le operazioni di lettura/scrittura vengono effettuate sulle celle puntate dallo stack pointer tramite operazioni push/pop (per impilare / rimuovere dati sullo / dallo stack). Anche questa modalità è usata nella gestione degli "activation record".
Possiamo salvare i dati correnti e impilare nuovi dati su cui lavoreremo. Essi potranno essere cancellati alla fine dell'operazione e ripristinare i dati allo stadio originale (per le chiamate a procedure)

Tipiche operazioni

Studiando l'ISA del processore Hack, abbiamo già avuto modo di vedere le tipiche istruzioni che vengono messe a disposizione:

- Operazioni di trasferimento dati: È possibile trasferire dati da memoria a registri, da registri a memoria, e da registri a registri
- Operazioni aritmetico-logiche binarie: Noi abbiamo visto solo operazioni su interi, ma solitamente esistono anche istruzioni per numeri floating-point
- Salti (condizionati e incondizionati)

Inoltre :

- Operazioni unarie: A volte sono presenti istruzioni di "shift" o rotazione, per velocizzare moltiplicazioni o divisioni per potenze di 2
- Operazioni molto frequenti possono avere una propria istruzione, vedi INC (incremento) caso particolare di somma (anche in Hack)

Invocazione di procedura

Solitamente esistono istruzioni per l'invocazione di procedura (non presenti in Hack).

Esistono almeno:

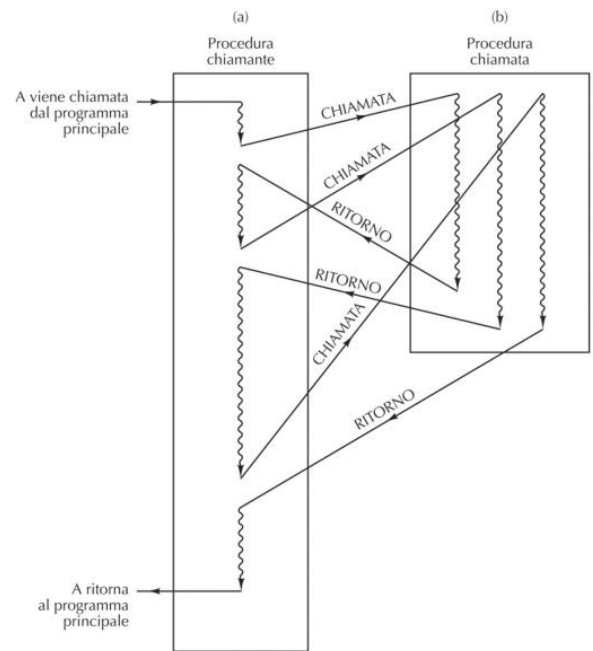
- Una istruzione di chiamata a procedura:

Si interrompe l'esecuzione sequenziale, si salta ad un nuovo programma (o parte di un programma), ma si tiene traccia del punto da dove si è effettuata l'invocazione (salvando l'«indirizzo di ritorno») → si salta all'interno di diverse parti di memoria.

In pratica, dovrò ricordare i parametri degli argomenti del chiamante e l'indirizzo da cui parto per fare il salto e recuperare le informazioni.

- Una istruzione di ritorno: Si torna ad eseguire dalla istruzione successiva a quella di invocazione: intuitivamente, caricando l'indirizzo di ritorno nel program counter.

Nelle chiamate a procedure il passaggio del controllo tra il chiamante e la procedura chiamata è “asimmetrico”



Implementare le invocazioni di procedura richiede di allocare in memoria i dati necessari per eseguire la procedura invocata. Questi includono: gli argomenti, le variabili locali, l'indirizzo di ritorno. Tale spazio in memoria viene chiamato “record di attivazione” (activation record). → si tratta di qualunque porzione di memoria in cui abbiamo salvato i dati che servono per la chiamata a procedura, e sono organizzati a stack.

- I record di attivazione sono organizzati a stack (pila), ovvero vengono salvati in memoria in maniera sequenziale, con modalità LIFO (last-in, first-out). Vengono salvati i dati di partenza, e in cima allo stack viene messo il risultato della chiamata ricorsiva.
- Quando termina l'esecuzione di una procedura, si riattiva la precedente (cioè la penultima invocata).

Intuitivamente, dobbiamo “ricaricare” il precedente record di attivazione. In pratica, teniamo invece un semplice puntatore (FP) che punta al record ‘corrente’, e quando vogliamo ripristinare il precedente record, aggiorniamo solo il puntatore a quello.

Record di attivazione

Per fare questo, utilizziamo un registro specializzato chiamato Frame Pointer (FP), che punta all'inizio (al primo indirizzo di memoria) del record di attivazione della funzione attualmente in esecuzione (per sapere dove inizia il record corrente sullo stack). Nel record di attivazione vengono inseriti:

- **I parametri di invocazione**
- **L'indirizzo di ritorno:** punto di rientro per rimettere in esecuzione il chiamante, cioè l'indirizzo dell'istruzione successiva nel chiamante
- **Il vecchio frame pointer** serve per ripristinare FP al momento del ritorno al chiamante (quando la procedura chiamata termina), in modo che FP punti di nuovo al precedente record di attivazione
- **Variabili locali:** variabili locali alla procedura chiamata

Esempio: Torre di Hanoi

Vediamo un esempio di implementazione a stack delle chiamate a procedura, tramite un programma ricorsivo che risolve il problema della Torre di Hanoi

Nel problema della torre di Hanoi è necessario spostare dei dischi da “Piolo 1” a “Piolo 3”, spostando un disco alla volta e mettendo i dischi sopra a dischi più grandi.

Il programma è ricorsivo, viene eseguito varie volte fino a che non viene esaurito, e il controllo torna al chiamante.

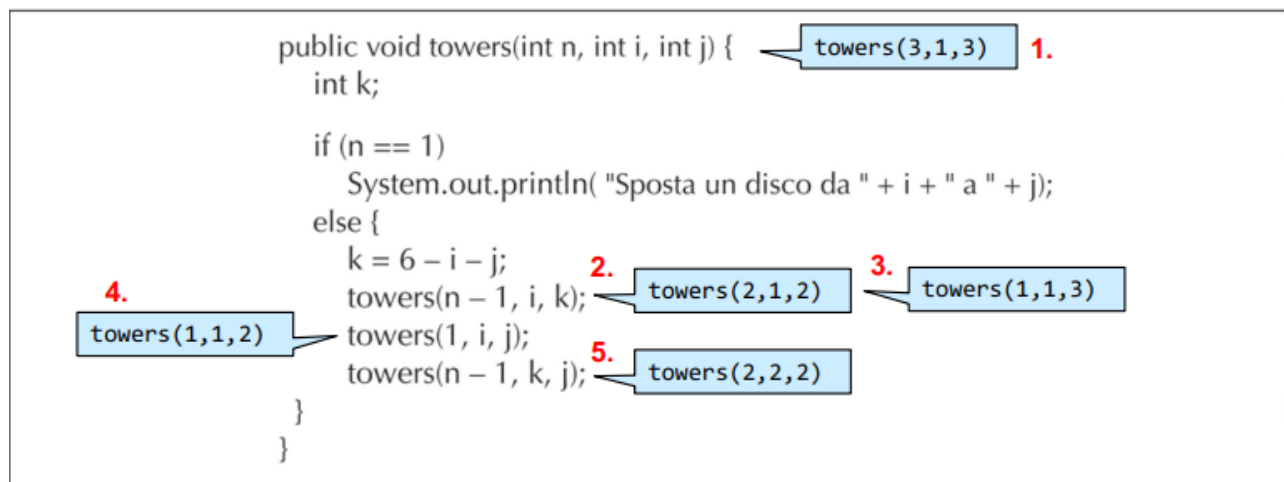
Quindi con “lo stack” si intende un’area di memoria che manipoliamo con logica LIFO (push/pop), e in particolare parliamo qui di come usare lo stack per mantenere i record di attivazione di chiamate a procedure.

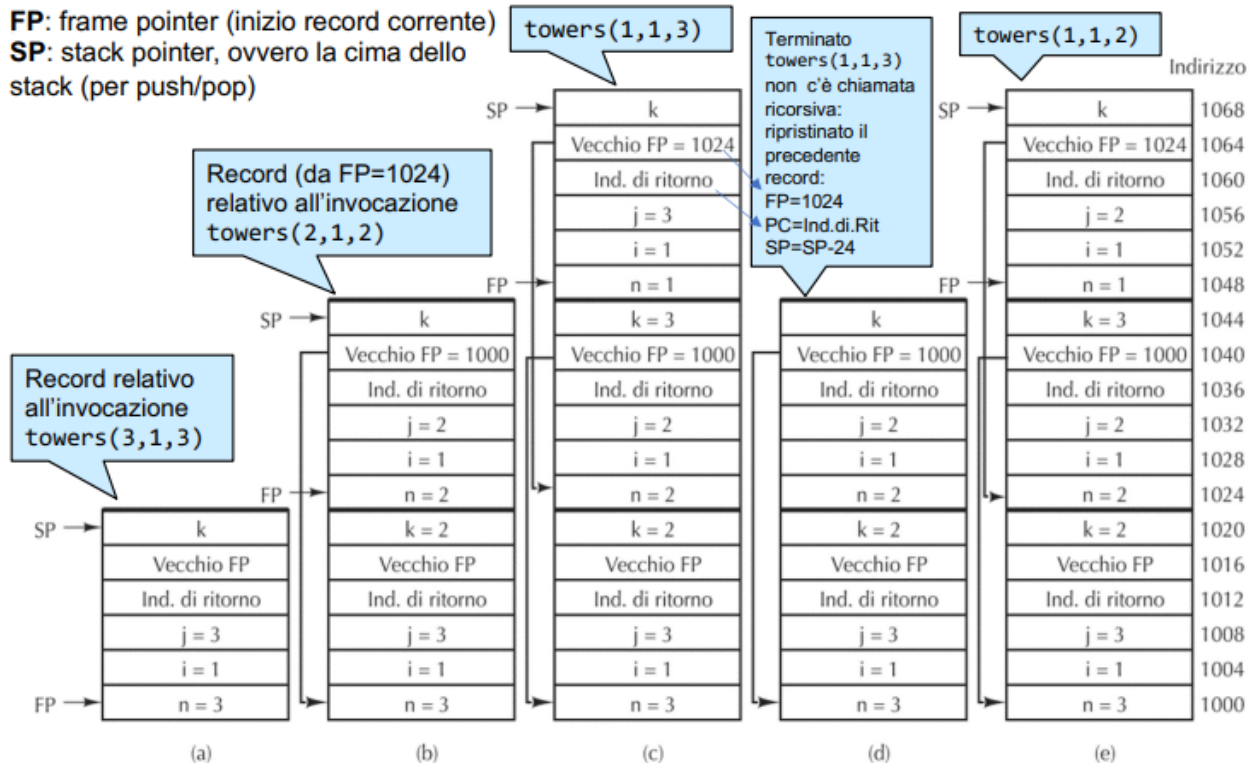
Il vantaggio di usare una memoria organizzata a stack per i record di attivazione è che, invece di dover assegnare indirizzi assoluti alle variabili locali (e argomenti, valori ritornati, valore dell’indirizzo di ritorno, etc.) delle diverse chiamate a procedura, possiamo utilizzare uno indirizzamento indicizzato (con offset) relativamente al FP corrente.

Questo è molto vantaggioso e pratico in generale, ma soprattutto è necessario nel caso di chiamate ricorsive (che quindi richiedono “copie” delle stesse variabili (e argomenti, etc.), per ogni livello della ricorsione).

Eseguiamo il programma invocando: `towers(3,1,3)`: 3 dischi, da piolo 1 a piolo 3
Quando `n==1` un disco viene spostato, altrimenti eseguiamo chiamate ricorsive

Raffiguriamo le prime 5 chiamate:





Trap

Una trap è una chiamata di procedura automatica effettuata quando si verificano condizioni rilevanti di errore, ad esempio:

- Opcode non definito (istruzione sconosciuta)
- Accesso ad area di memoria non consentita (per esempio segmenti di memoria riservati ad altri processi o activation records)
- Overflow

Se si verifica un evento simile, il controllo è trasferito ad una procedura detta “gestore di trap”. Ad esempio, il gestore di trap potrebbe comunicare l’errore e non restituire più il controllo al programma che ha generato l’errore.

Il computer salva lo stato del programma, va al gestore che fa ciò che deve fare e poi torna allo stato iniziale. Questa cosa funziona usando le chiamate ricorsive.

Essenzialmente, una trap è scatenata da una condizione causata da programma stesso, e catturata dall’hardware o dal microprogramma.

Quindi le trap sono “sincrone” (al programma): la trap viene scatenata da un’istruzione del programma in esecuzione che causa una condizione di errore.

Il vantaggio di un sistema con trap, è che il programma(tore) non deve occuparsi esplicitamente di catturare queste condizioni, che vengono automaticamente catturate e gestite.

Interrupt

Similmente alle trap, le “interruzioni” (interrupt) sono condizioni che interrompono il programma in esecuzione e trasferiscono il controllo ad un gestore deputato (detto Interrupt Service Routine - ISR).

Sono tipicamente legati a dispositivi di I/O (ma non solo: anche la CPU e programmi), e non si riferiscono solamente a condizioni di errore, ma anche a notifiche/eventi.

Esempio: un dispositivo di I/O indica la fine di una specifica operazione, per esempio la ricezione di un messaggio, o il fatto che è pronto a ricevere dati dalla CPU.

Esempio di errore: bit di parità errato.

Esistono diverse implementazioni. In generale, nel caso di dispositivi, si usano dedicate linee per la notifica di un interrupt (interrupt requests), che vengono ricevute da un circuito di controllo il quale invia un segnale alla CPU. Quando la CPU è pronta (restituisce un acknowledgement della richiesta), il circuito scrive sul bus dati l'indice (interrupt vector) dell'ISR specifica per l'interrupt/dispositivo. Questo indice viene usato per prelevare l'indirizzo dell'ISR (dove il programma relativo è memorizzato) da una apposita tabella statica salvata in memoria (Interrupt Description Vector, a sua volta puntato da un registro specializzato), così che la CPU può salvare lo stato corrente e settare il program counter a quell'indirizzo (effettuando così una chiamata di procedura all'ISR specifico).

La tecnica dell'interrupt è fondamentale per evitare il cosiddetto “busy waiting”, (la pratica di eseguire continuamente istruzioni per andare a verificare se è accaduto un certo evento, invece di ricevere notifiche mentre si continua l'esecuzione).

Mentre le trap sono “sincrone”, gli interrupt sono “asincroni”: sono scatenati da altri dispositivi indipendentemente dal programma in esecuzione.

Mascherabilità e priorità

Non sempre un interrupt deve interrompere l'esecuzione attuale. Per esempio, il gestore stesso (ISR) non dovrebbe sempre essere anch'esso interrotto da nuovi interrupt.

- Il sistema operativo potrebbe quindi decidere che in alcuni momenti alcuni interrupt non devono essere in grado di interrompere il programma in esecuzione. In questo caso si parla di interrupt che vengono “mascherati”. In certi momenti, gli interrupt potrebbero anche essere completamente disabilitati. Alcuni interrupt (tipicamente relativi ad eccezioni o errori gravi) possono essere dichiarati non-mascherabili.
- In altri casi, conviene definire una gerarchia di interrupt: alcuni sono più importanti di altri (assegnandogli una priorità numerica). Un interrupt può interrompere il gestore solo se ha priorità maggiore di quella attualmente in gestione. Altrimenti dovrà attendere per essere gestito.

Di solito un dispositivo I/O ha una priorità prestabilita, e tutti gli interrupt generati dal dispositivo ricevono quella priorità.

Da ChatGPT:

Trap

Un **trap** è un'interruzione generata dal processore stesso in risposta a un evento interno al programma in esecuzione. È spesso usato per segnalare condizioni particolari o per richiedere specifici servizi al sistema operativo.

Caratteristiche principali dei trap:

1. **Origine interna:** I trap sono causati dal programma in esecuzione, per esempio:
 - o **Chiamate di sistema:** Il programma richiede al sistema operativo di svolgere un'operazione (ad esempio, leggere un file o allocare memoria).
 - o **Eccezioni:** Errori come la divisione per zero, accesso a memoria non valida o istruzioni illegali.
2. **Gestione:** Quando si verifica un trap, il controllo passa al sistema operativo, che esegue il codice del gestore corrispondente (il "trap handler").
3. **Esempio di uso:** Supponiamo che un programma voglia aprire un file. Questo effettuerà una chiamata di sistema (tramite un trap), trasferendo il controllo al sistema operativo, che gestirà l'operazione.

Obiettivo principale:

I trap sono progettati per consentire la comunicazione tra il programma utente e il kernel del sistema operativo, oppure per gestire condizioni eccezionali interne al programma.

Interrupt

Un **interrupt** è un segnale generato da una fonte esterna al processore o dal processore stesso, che indica la necessità di attenzione immediata da parte della CPU.

Caratteristiche principali degli interrupt:

1. **Origine esterna:** Gli interrupt sono spesso generati da dispositivi hardware, come:
 - La pressione di un tasto sulla tastiera.
 - La ricezione di dati da una rete.
 - La fine di un'operazione di I/O (Input/Output), ad esempio il completamento della lettura di un disco.
2. **Priorità:** Gli interrupt possono avere priorità diverse. Ad esempio, un segnale di emergenza da un sistema critico potrebbe avere priorità più alta rispetto a un semplice timer.
3. **Gestione:** Quando si verifica un interrupt:
 - La CPU salva il contesto dell'esecuzione corrente.
 - Esegue un gestore dell'interrupt (interrupt handler) per risolvere l'evento.
 - Dopo aver gestito l'interrupt, riprende l'esecuzione del programma interrotto.

Obiettivo principale:

Gli interrupt permettono al processore di rispondere rapidamente agli eventi esterni senza dover continuamente controllare lo stato dei dispositivi (polling), migliorando così l'efficienza.

Differenze principali tra trap e interrupt

Caratteristica Trap		Interrupt
Origine	Interna al programma	Esterna o interna (spesso hardware)
Causa	Chiamata di sistema o eccezioni	Eventi hardware o segnali esterni
Finalità	Richiesta di servizi al sistema operativo o gestione di errori	Risposta a eventi hardware esterni

Un interrupt è detto **mascherabile** se può essere ignorato (o "mascherato") dal processore per un certo periodo, generalmente disabilitando temporaneamente quel tipo di interrupt attraverso configurazioni software o hardware. In altre parole, il processore può scegliere di non rispondere immediatamente a un interrupt mascherabile.

Un interrupt **non mascherabile**, invece, è un tipo di interrupt che **non può essere disabilitato**, ed è gestito immediatamente dal processore, poiché di solito segnala eventi critici o di emergenza.

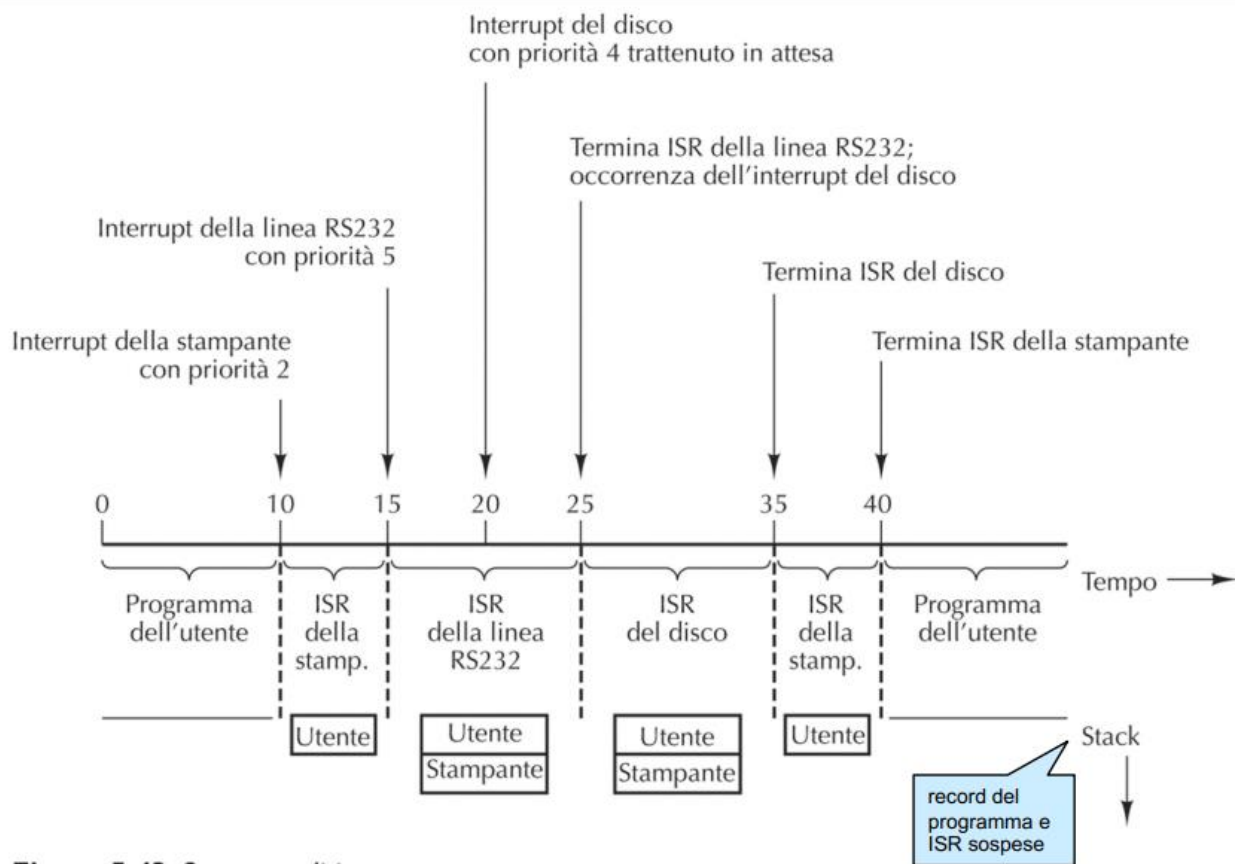


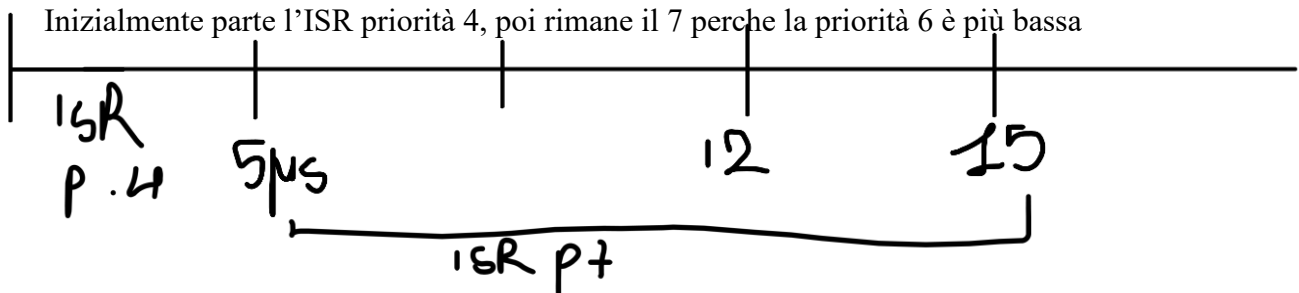
Figura 5.43 Sequenza di interrupt.

Esercizio

Si descriva cosa succede se ad un processore arrivano i seguenti interrupt, assumendo un sistema di gestione degli interrupt con priorità (numeri più grandi corrispondono a priorità maggiore) e nessun ISR in esecuzione al momento:

- interrupt a priorità 4 a $t=0\mu s$
 - interrupt a priorità 7 a $t=5\mu s$
 - interrupt a priorità 6 a $t=12\mu s$
- Si assuma che ogni routine di gestione dell'interrupt richieda $10\mu s$. (in una variante di questo esercizio, si potrebbe specificare la durata effettiva dell'esecuzione di ciascun ISR).

Inizialmente parte l'ISR priorità 4, poi rimane il 7 perche la priorità 6 è più bassa



Si descriva cosa succede se ad un processore arrivano i seguenti interrupt, assumendo un sistema di gestione degli interrupt con priorità (numeri più grandi corrispondono a priorità maggiore):

- interrupt a priorità 3 a $t=0\mu s$
- interrupt a priorità 5 a $t=4\mu s$
- interrupt a priorità 4 a $t=8\mu s$
- interrupt a priorità 6 a $t=13\mu s$

Si assuma che ogni routine di gestione dell'interrupt richieda $8\mu s$.

Programmazione in C

La compilazione C comprende le seguenti fasi:

Preprocessore: trasforma i file sorgenti in base alle “preprocessor directives” specificate nel file

Le principali preprocessor directives sono le seguenti:

- **#include “[headerName]”:** includere un file esterno.
- **#define [token] [code]:** definire una macro.
- **#ifdef [token] ... #endif:** se una macro è definita esegue il codice dopo l'if, altrimenti esegue l'else.
- **#ifndef [token] ... #endif:** se una macro non è definita esegue il codice dopo l'if, altrimenti esegue l'else.

Per terminare la compilazione dopo il preprocessore e ottenere il risultato in output occorre eseguire il seguente comando da terminale: `gcc -E [files]`

Con il seguente comando invece si ottiene in output un file assembly human readable per ogni file .c passato nel comando: `gcc -S [files]`

Compilatore: Traduce in assembly il programma in C. Di default si genera direttamente il “codice oggetto” (object file), cioè un file già in linguaggio macchina (ancora con riferimenti ancora da instanziare, per esempio invocazione di funzioni di libreria: vedi linking). Quindi fa uso di un assemblatore.

Per terminare la compilazione dopo il compilatore e ottenere un file oggetto per ogni file passato nel comando, occorre eseguire il seguente comando da terminale: `gcc -c [files]`

Linking: Collega vari object files e librerie (statiche) generando così un file eseguibile completo

Per ottenere un file eseguibile ottenuto dopo il linking occorre eseguire da terminale il seguente comando: `gcc -o [outputFile] [inputFiles]`

Si esegue il file:

```
./main
```

“./” specifica alla shell che l'eseguibile è nella directory corrente

Un modo per velocizzare (e razionalizzare) la compilazione in casi di programma scritto in più file è usare la utility make Richiede la preparazione di un file chiamato Makefile. Esempio:

```
main: main.o somma.o
    gcc -o main main.o somma.o
main.o: main.c somma.h
    gcc -c main.c
somma.o: somma.c somma.h
    gcc -c somma.c
```

```
clean: rm *.o main
```

Dopo aver creato il makefile è sufficiente lanciare il comando make o MinGW32-make (se si utilizza MinGW) da terminale, oppure make [commandName] per eseguire un singolo comando del makefile. Tra i benefici dell'utilizzo del make, oltre alla maggiore velocità di compilazione vi è il fatto che, grazie alla specifica delle dipendenze per ogni comando, ricompila solo le parti che sono state modificate.

Output formattato

int printf(char *format, arg1, arg2, arg3, ...)

Format è la stringa da stampare, la quale può contenere elementi (chiamati format specifiers) come quelli elencati sotto. La stringa stampata avrà questi specifiers sostituiti con gli argomenti arg1, arg2, ... passati, in ordine:

Ad esempio

- %d: stampa un intero.
- %f: stampa un float.
- %lf: stampa un double.
- %s: stampa una stringa.
- %c: stampa un singolo carattere.

```
char s[5] = "somma"; int a=3; int b=4;
printf("La %s di %d e %d è %d\n", s, a, b, a+b);
>> La somma di 3 e 4 è 7
```

Input formattato

int scanf(char *format, *arg1, *arg2, *arg3, ...);

Viene letto l'input (da tastiera) e copiato nelle variabili agli indirizzi *arg1, *arg2,... come specificato da format specifiers nel format:

```
int a, b;
printf("Scrivi due interi:\n");
scanf("%d %d", &a, &b);
printf("Letti: %d e %d\n", a, b);
```

L'operatore sizeof prende in input o una variabile o un identificatore di tipo, e restituisce la dimensione in byte. Esempio con un tipo:

```
printf("dimensione del float: %lu byte\n", sizeof(float) );
>> dimensione del float: 4 byte
```

sizeof() ritorna un risultato di tipo size_t, che è un alias per il tipo unsigned opportuno (dipendentemente dal sistema). Per stamparlo, possiamo usare unsigned long (format specifier: %lu) per portabilità.

Anche in C abbiamo la libreria string.h. strncpy non controlla la dimensione di dest, il controllo va fatto prima per evitare overflow.

Malloc e free

malloc() alloca dinamicamente la memoria (chiede al S.O. di riservare un'area di memoria) e restituisce un puntatore a void che punta all'inizio di quest'area di memoria allocata. Se non è disponibile sufficiente memoria restituisce NULL, ad indicare che l'allocazione non è stata effettuata.

```
char *s = (char *) malloc(10*sizeof(char));
```

Nota: malloc ritorna un puntatore void*. Il cast non è sempre necessario in C (a differenza di C++). Fare attenzione a non oscurare potenziali errori sui tipi facendo cast espliciti che non sono necessari.

free() invece libera la memoria allocata, rendendola di nuovo disponibile per una nuova allocazione con malloc

Per utilizzare le seguenti funzioni occorre includere la libreria <stdlib.h>

Casting

La conversione di tipo type-casting in qualche caso viene effettuata implicitamente dal compilatore C, ma può anche essere forzata dal programmatore mediante un operatore unario detto cast:

```
int i=5, j=2;
double f;
f = (double)i / (double)j;
printf("%1.2f", f);
```

Le strutture si costruiscono in maniera simile a C++.

```
typedef struct luogo {
char nome[30];
int x;
int y; int z;
} // CITTA// ;
```

Una volta definito il tipo di dato, posso dichiarare una variabile di quel tipo.

```
struct luogo bologna;
//CITTA bologna;
```

Puntatori a strutture

E' possibile utilizzare puntatori che puntano a dati di tipo strutturato come le struct. La logica è la stessa.

L'allocazione dinamica per strutture funziona come per altri tipi di variabile. Nel caso di campi di tipo puntatore, non puntano a memoria allocata.

```
persona *p2 = malloc( sizeof(persona) );
```

Sistema Operativo

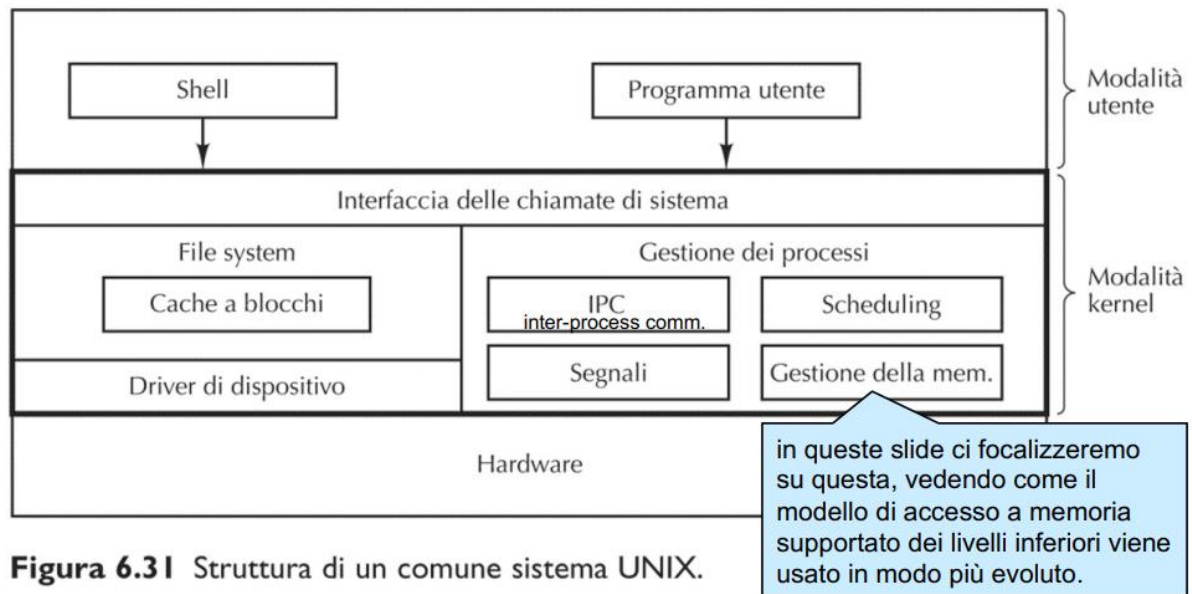
Le risorse messe a disposizione dall'architettura del calcolatore vengono gestite da una componente software: il sistema operativo. Possiamo vedere il sistema operativo come l'interfaccia fra il calcolatore ed i programmi applicativi (programmi utente: i sw installati sulla nostra macchina, ma anche i nostri programmi che eseguiamo).

Il sistema operativo è un componente software trasversale, responsabile dell'esecuzione dei programmi applicativi, e che offre funzionalità necessarie alla loro esecuzione (funzionalità che non sono parte del programma stesso).

Tra i compiti principali del sistema operativo:

- Gestione della memoria
- Gestione dei processi (programmi in esecuzione)

- Gestione delle periferiche di I/O



Un tipico OS è composto da 2 strati: la **modalità kernel**, che si interfaccia direttamente con l'hardware e gestisce i driver, i processi, il file system e le chiamate di sistema; e la **modalità utente**, che si interfaccia con la modalità kernel e gestisce la shell e i programmi utente.

- La Shell è una specifica interfaccia per interagire con l'OS (si utilizza tramite il terminale).
- Sia la Shell che i programmi utente interagiscono con l'OS tramite le *chiamate di sistema*.

Paginazione

Negli anni '60 i calcolatori avevano una memoria estremamente limitata. Pensiamo per esempio ad un computer a 16 bit con una memoria di 4K locazioni/indirizzi (non 4KB) contenenti parole da 16 bit e indirizzi a 16 bit. Ovviamente, vorremmo avere una memoria superiore a 4K locazioni per salvare i programmi in esecuzione ed i loro dati e per utilizzarla come memoria di lavoro.

Notare: Gli indirizzi fisici di memoria vanno quindi da 0 a 4095.

Ma $2^{16}=65536$, quindi gli indirizzi da 4096 a 65535 non vengono usati.

L'idea è di sfruttare appieno la "capacità di indirizzamento" dei 16 bit, assumendo una memoria virtuale (non fisica!) da 64K locazioni, quindi usando tutti gli indirizzi da 0 a 65535.

Tali indirizzi vengono detti indirizzi virtuali. Come è possibile assumere di avere a disposizione una memoria virtuale da 64K usando una memoria fisica (un chip) da 4K locazioni?

Nota:

1. Un kilobyte (KB) nel sistema binario:

- Nella convenzione binaria, utilizzata per la memoria nei computer, 1 kilobyte è definito come 2^{10} byte, che equivale a **1024 byte**.
- Quindi: 1 KB=1024 byte.

2. 4 kilobyte (KB):

- Se 1 KB è 1024 byte, allora 4 KB è semplicemente: $4 \times 1024 = 4096$ byte.

Idea: in ogni istante, al massimo 4K dati vengono posti in memoria primaria, usando per il resto la memoria secondaria. La difficoltà diventa scambiare continuamente il contenuto della memoria primaria, a seconda della necessità.

Tutto ciò in maniera trasparente al resto del sistema ed ai programmi: per quanto riguarda l'accesso a memoria, questa appare davvero da 64K locazioni.

Pro: ovviamente, abbiamo "a disposizione" più memoria di lavoro senza dover aumentare la memoria fisica (non pensiamo solo al costo e dimensioni, ma anche alla complessità della microarchitettura).

Contro: la memoria secondaria è meno performante della memoria primaria, quindi stiamo creando un collo di bottiglia.

(le memorie RAM sono più veloci degli HDD sia per differenze tecnologiche dei componenti usati, sia per l'interfaccia di accesso, per esempio parallela vs seriale: possiamo avere più linee dati verso la RAM).

Aggiungere più memoria (RAM) fisica migliora le prestazioni

Primo problema: bisogna "mappare" (o "risolvere") gli indirizzi virtuali a cui si accede negli indirizzi fisici dove il dato è effettivamente salvato (assumendo che sia in memoria al momento).

Secondo problema: bisogna gestire la sostituzione continua dei dati richiesti dai programmi copiandoli davvero in memoria primaria, tenendo il resto in memoria secondaria.

L'approccio per risolvere entrambi è basato sulla **paginazione**.

Da ChatGPT:

La **memoria virtuale** è una tecnologia utilizzata nei computer per gestire e ottimizzare l'uso della memoria fisica (RAM). Serve per consentire l'esecuzione di programmi che richiedono più memoria di quella effettivamente disponibile nel sistema.

Come funziona:

1. **Espansione della memoria:** La memoria virtuale utilizza una porzione del disco rigido o SSD come se fosse un'estensione della RAM. Questo spazio del disco viene chiamato **file di paging** (su Windows) o **swap space** (su Linux/macOS).
2. **Paginazione e segmentazione:**
 - o La memoria virtuale divide i programmi e i dati in **pagine** di dimensioni fisse o in **segmenti** di dimensione variabile.
 - o Solo le pagine necessarie in un dato momento vengono caricate nella RAM, mentre le altre rimangono sul disco fino a quando non sono richieste.
3. **Mapping degli indirizzi:**
 - o Ogni programma "pensa" di avere accesso a uno spazio di memoria continuo e privato (gli **indirizzi virtuali**).
 - o Il sistema operativo traduce questi indirizzi virtuali in **indirizzi fisici** reali della RAM. Questa traduzione avviene grazie a una componente hardware chiamata **MMU (Memory Management Unit)**.
4. **Swapping:**
 - o Se la RAM è piena e un'applicazione richiede più memoria, il sistema operativo "sposta" temporaneamente alcune pagine di dati dalla RAM al disco rigido per fare spazio. Questo processo è chiamato **swapping**.

Vantaggi della memoria virtuale:

- **Gestione efficiente:** Permette a più programmi di funzionare contemporaneamente, anche se la RAM è limitata.
- **Protezione della memoria:** I programmi non accedono direttamente agli indirizzi fisici, prevenendo conflitti e crash.
- Fornisce l'illusione di avere più memoria fisica di quanta ne sia effettivamente installata.

Svantaggi:

- **Prestazioni:** L'accesso al disco rigido (o SSD) è molto più lento rispetto alla RAM, quindi un uso eccessivo della memoria virtuale può rallentare il sistema

Paginazione

- La memoria virtuale viene divisa in pagine, tutte di uguale dimensione (una potenza di due, ovviamente).
- Inoltre, la memoria primaria (fisica) viene suddivisa in parti di uguale dimensione chiamate blocchi, della stessa dimensione delle pagine.

Idea: salvare le pagine di memoria (virtuale) nei blocchi di memoria (fisica).

Se la dimensione di pagina è 2k, si usano k bit per indirizzare una locazione all'interno di una pagina (offset). Nell'esempio consideriamo pagine da 4K locazioni (quindi servono 12 bit per l'offset).

- (a) Primi 64KB degli indirizzi virtuali suddivisi in 16 pagine da 4KB ciascuno, assumendo quindi 8 bit di parola.
- (b) Memoria principale di 32KB suddivisa in 8 blocchi di memoria da 4KB.

Pagina	Indirizzi virtuali
15	61440 – 65535
14	57344 – 61439
13	53248 – 57343
12	49152 – 53247
11	45056 – 49151
10	40960 – 45055
9	36864 – 40959
8	32768 – 36863
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

memoria virtuale molto più grande della memoria fisica

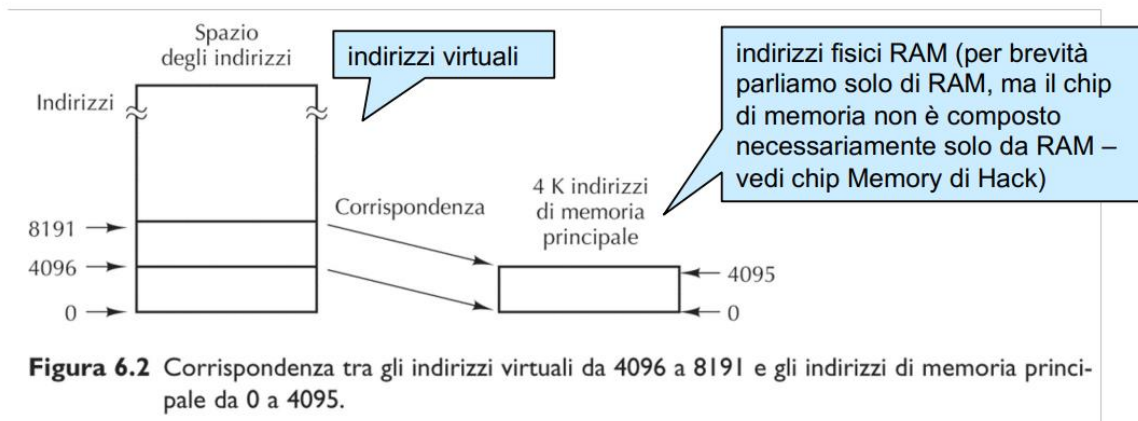
Blocco di memoria	Indirizzi fisici
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

Primi 32 K di memoria principale

(a) (b)

Tradurre indirizzi virtuali in indirizzi fisici

Bisogna “mappare” (o “risolvere”) gli indirizzi virtuali a cui si accede negli indirizzi fisici dove il dato è effettivamente salvato (assumendo che sia in memoria al momento).



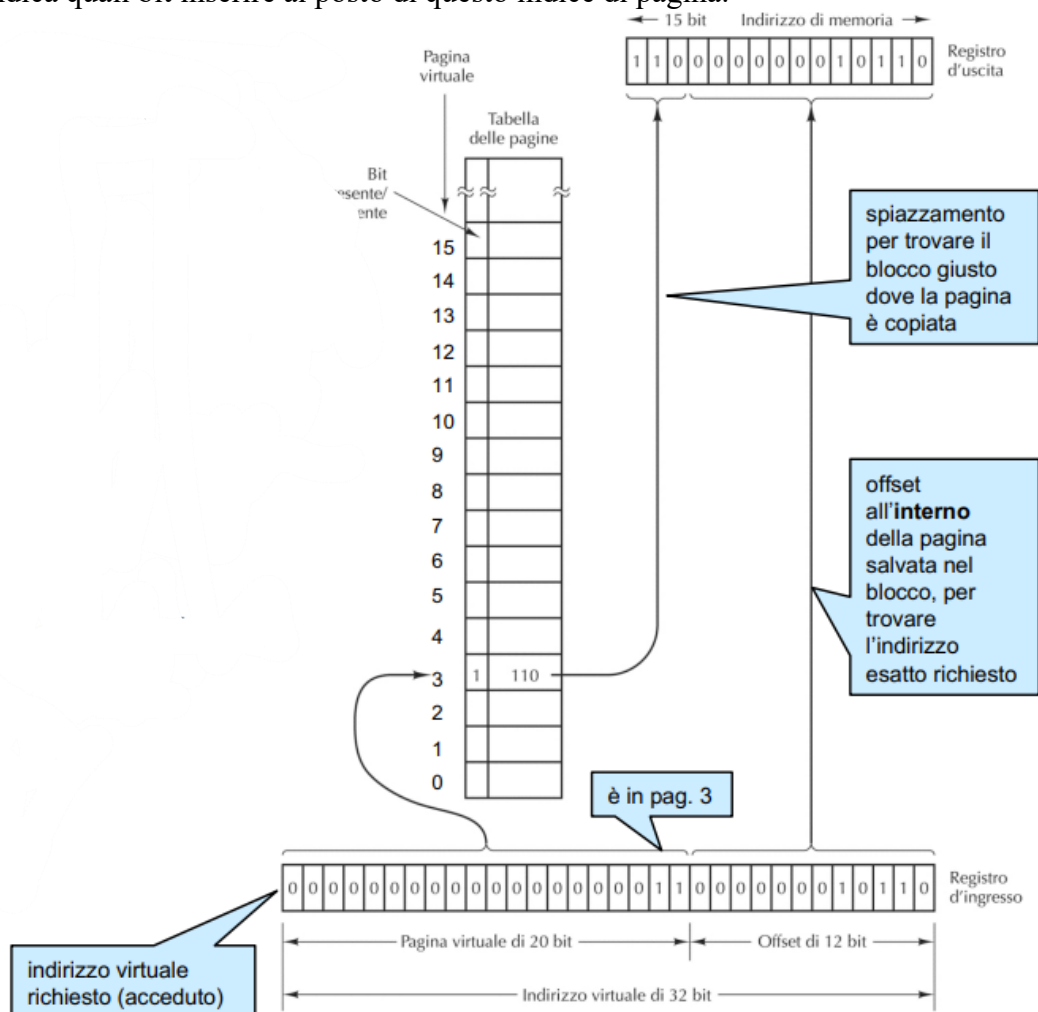
In figura, accedere per esempio all'indirizzo virtuale $4098 = 4096 + \text{offset } 2$ deve determinare un accesso in memoria primaria all'indirizzo fisico $2 = 0 + \text{offset } 2$ (offset=spiazzamento, cf. la modalità di indirizzamento indicizzato).

Questo mapping viene fatto dal S.O. in modo trasparente al programma.

Come tradurre: la tabella delle pagine

Esiste hardware dedicato (chiamato Memory Management Unit – MMU) che si occupa di tradurre un indirizzo virtuale nel corrispondente indirizzo fisico. Si usa una “tabella delle pagine” che indica, per ogni pagina, se questa è al momento in memoria, e in caso positivo indica in quale blocco. L'indice

di pagina si ottiene guardando l'indirizzo virtuale, esclusi i k bit usati per identificare l'offset. La tabella indica quali bit inserire al posto di questo indice di pagina.



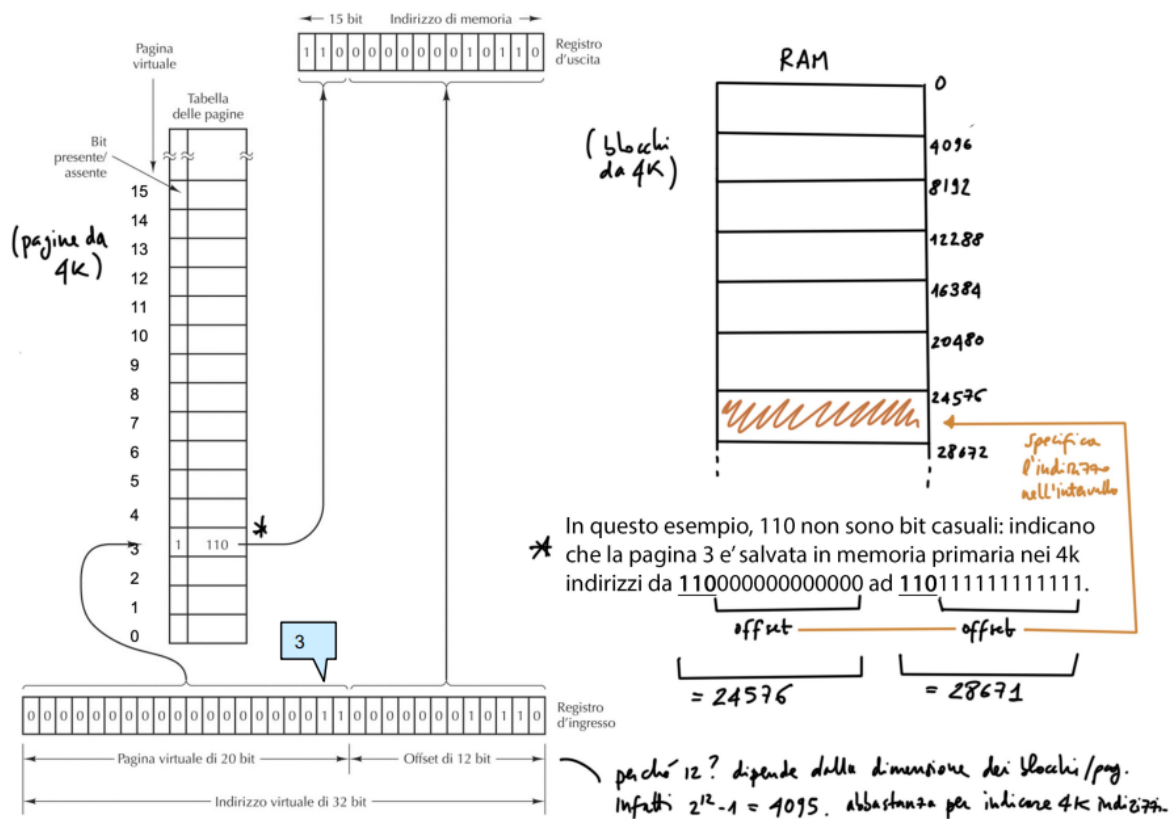


Figura 6.4 Formazione di un indirizzo fisico a partire da un indirizzo virtuale.

Page faults

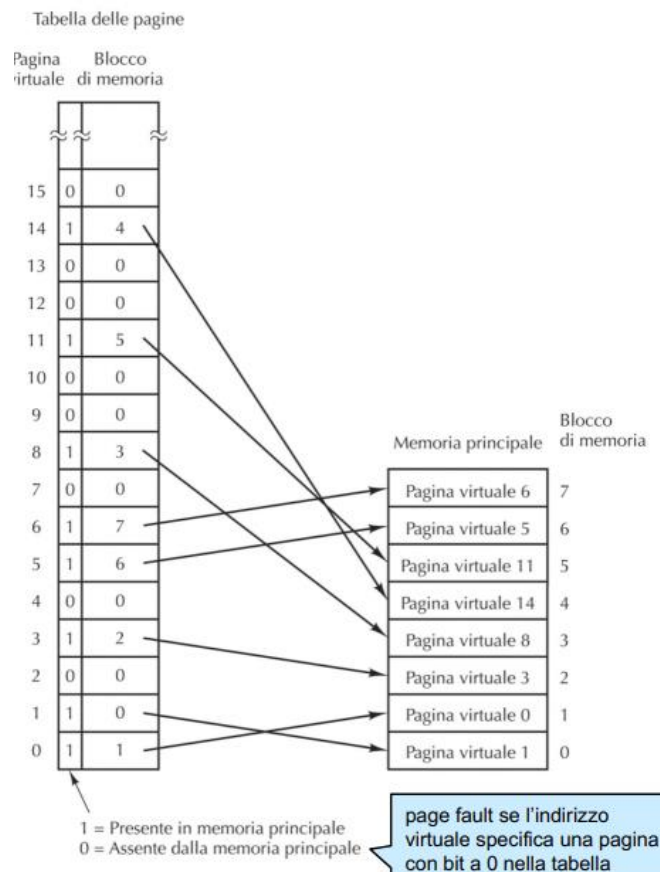
Come determinare le pagine da tenere in memoria? L'insieme delle pagine alle quali un programma deve accedere non è noto all'inizio (in generale).

Le pagine al momento in memoria vengono dette **working set**.

Se un programma prova ad accedere ad una pagina fuori dal working set avviene un trap detto **page fault**. Il gestore di tale trap deve rimuovere dalla memoria una pagina, per far posto per la pagina richiesta (che viene quindi copiata in memoria). Questa "sostituzione" di pagina in un blocco si chiama **page swapping**. La scelta di quale blocco liberare (quindi quale pagina rimuovere) viene presa dai cosiddetti algoritmi di paginazione.

In generale, possiamo:

- Attendere che il programma faccia richieste di accesso a memoria prima di copiare la pagina richiesta. Le pagine in memoria saranno solo pagine effettivamente necessarie al programma (o un sottoinsieme, se non ci sono abbastanza blocchi).

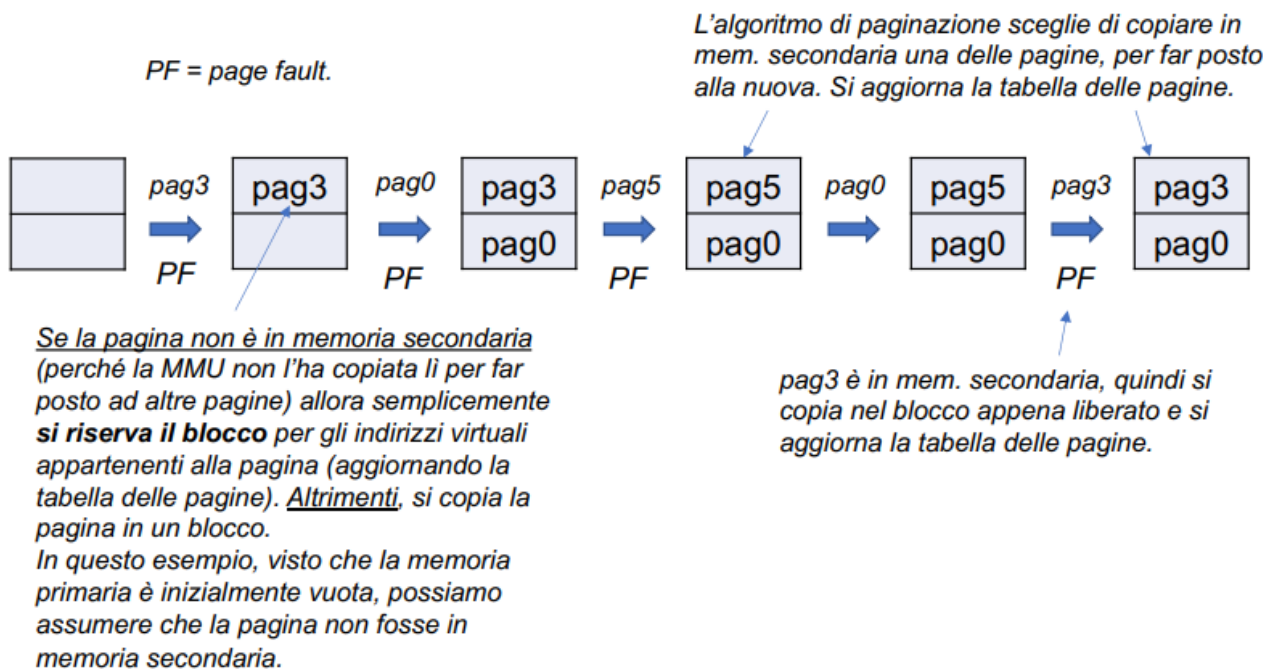


Oppure:

- Ad una richiesta, utilizzare il principio di località (ne avete parlato nel modulo 1 – vedi) per copiare in memoria anche altre pagine adiacenti. In questo caso potremmo avere in memoria pagine non realmente necessarie. Similmente, si potrebbe tentare di predire le pagine che serviranno, basandosi sul programma.

In quanto segue (e negli esercizi), ci focalizziamo sul primo approccio. Ma algoritmi assolutamente analoghi si possono definire facilmente anche per il secondo approccio.

Assumiamo una memoria primaria con solo 2 blocchi, per semplicità. Inizialmente la memoria è vuota, e successivamente vengono acceduti (in lettura o scrittura) indirizzi virtuali che appartengono alle pagine: pagina 3, poi pagina 0, poi pagina 5, poi pagina 0, poi pagina 3.



Algoritmi di paginazione: LRU e FIFO

Due tipici algoritmi di paginazione (~ tipi/famiglie di algoritmi) per determinare quale pagina rimuovere da un blocco quando si verifica un page fault:

LRU (Least Recently Used)

- Si rimuove dalla memoria la pagina che da più tempo non viene acceduta (in lettura/scrittura);
- Può essere implementato tramite una “lista”: quando si accede una pagina la si re-inserisce in coda, e quando serve rimuovere una pagina, si sceglie quella in testa.

FIFO (First-In First-Out)

- Si rimuove la pagina in memoria da più tempo;
- Può essere implementato da una coda. Oppure nel seguente modo: ogni blocco ha un contatore, inizializzato a 0 quando il blocco riceve una pagina, ed incrementato ad ogni page fault. Quando serve liberare un blocco si sceglie quello con il contatore più grande (intuizione: si “contano” i page fault a cui la pagina ha sopravvissuto).

Quando si rende necessario liberare un blocco (page fault), se la pagina non è stata modificata non serve ricopiarla in memoria secondaria!

Per tenere traccia di questo, si può usare la tecnica del “dirty bit”:

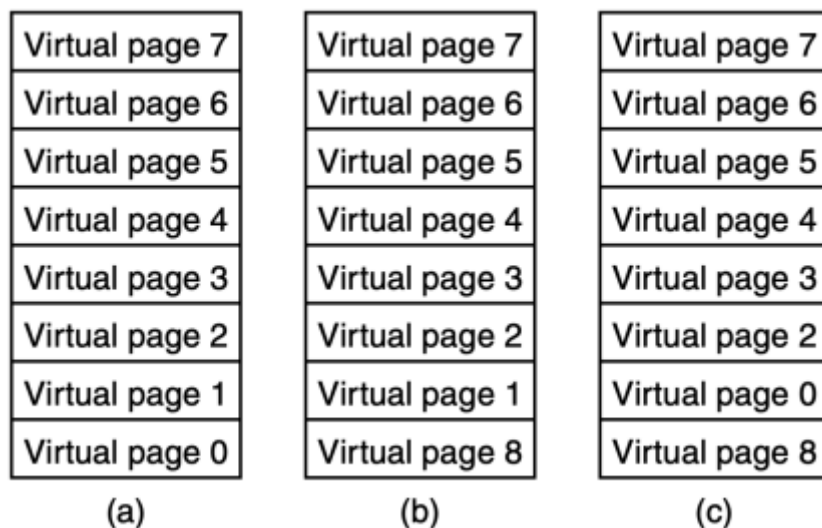
- Ogni blocco in memoria primaria ha un suo bit associato

- Quando si carica una nuova pagina nel blocco, si pone a 0 il bit corrispondente
- Quando la pagina in un blocco viene acceduta in scrittura, il bit viene posto a 1
- Quando si deve liberare il blocco, si salva la corrispondente pagina in memoria secondaria solo se il bit è a 1.

Osservazione: caso critico per LRU

Immaginiamo un programma che esegue un ciclo, il cui corpo accede in progressione ad indirizzi virtuali appartenenti a 9 pagine distinte (per semplicità, pagine dalla 0 alla 8, in ordine), ma la memoria fisica ha solo 8 blocchi: ci sarà sempre una pagina necessaria al programma ma che non è in memoria.

Durante l'esecuzione di un ciclo, si arriverà ad un certo punto ad avere le pagine 0- 7 in memoria (fig. a – in quale blocco salviamo ogni pagina non è rilevante). Cercando di accedere poi alla pagina 8, un algoritmo LRU rimuove la pagina 0 per far posto (fig. b). Ora il corpo del ciclo termina, ed occorre eseguirlo da capo, quindi la prima pagina ad essere acceduta sarà la pagina 0. Per far posto, LRU rimuove la pagina 1 (fig. c). Ma questa sarà anche la prossima pagina acceduta, etc.



Non confondere la paginazione (paging) della memoria virtuale con il caching (in particolare il page caching): il caching non ha a che fare con virtualizzazione della memoria (utilizzare un indirizzamento virtuale più vasto), ha come obiettivo le prestazioni, e riguarda l'accesso ai dati da/tra CPU a memoria primaria.

La virtualizzazione permette di avere a disposizione una memoria virtuale maggiore di quella fisica, così che possiamo caricare in memoria più dati e programmi (anche aumentando il grado di multiprogrammazione), ovviamente al costo di far lavorare la MMU.

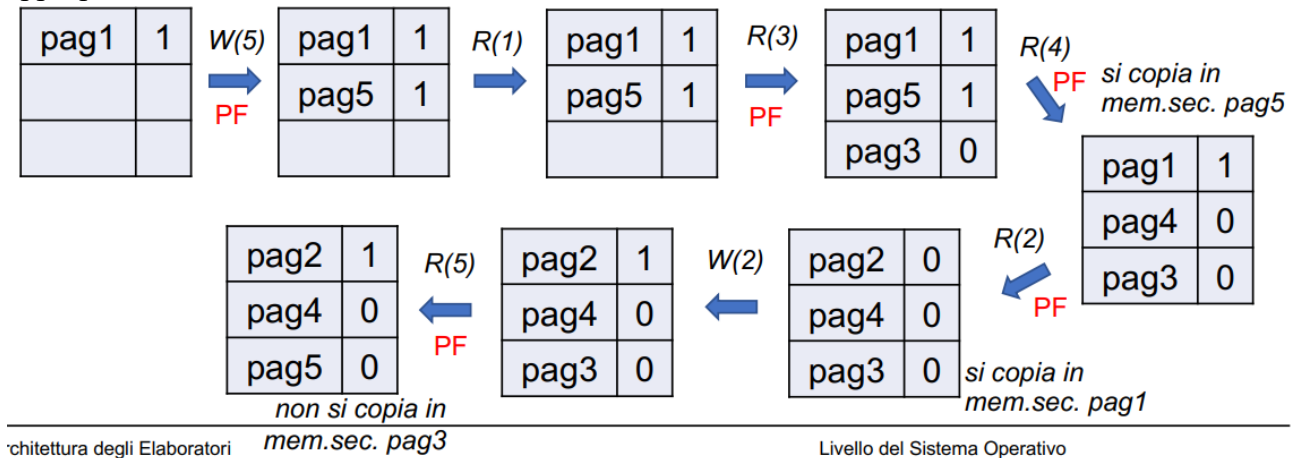
Le pagine di memoria virtuale vengono copiate in memoria secondaria solo per far spazio ad altre pagine in memoria primaria, ma non «sono» dati di memoria disco (secondaria), quindi la RAM non è «una specie di cache» tra CPU e disco.

Ovviamente possiamo fare una analogia tra queste due tecniche, ed in modo indiretto anche la paginazione aiuta le prestazioni, ma concettualmente ed in termini di obiettivi queste sono ortogonali e distinte.

Esercizio 1.

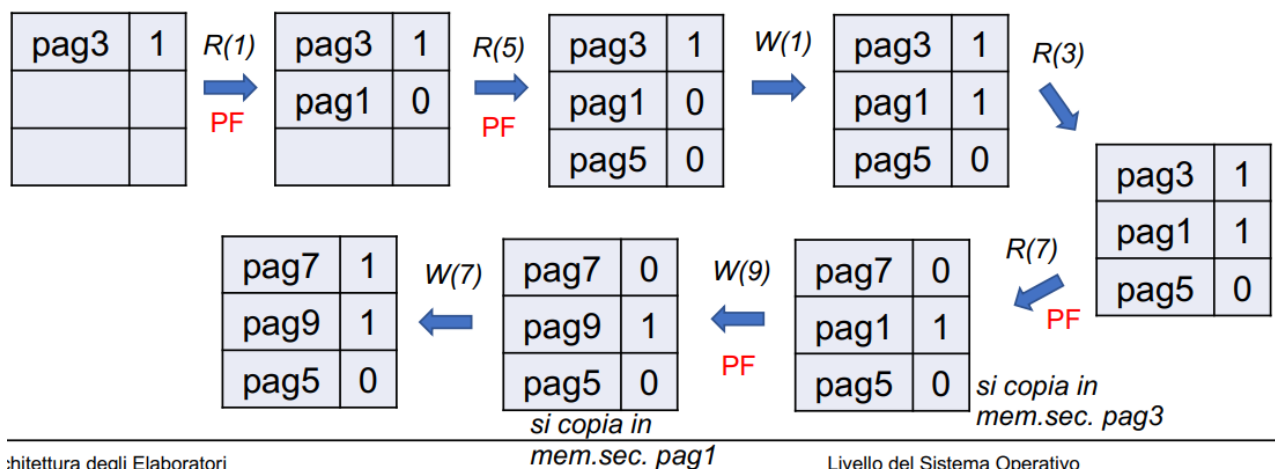
Si consideri un sistema di virtualizzazione della memoria basato su paginazione con algoritmo LRU e dirty bit. Si assuma che inizialmente sia caricata solo pagina 1 (su cui sono stati fatti accessi in scrittura), e che la memoria principale sia di 3 blocchi. Illustrare cosa succede (a parole e

diagramma) se vengono fatti i seguenti accessi: scrittura pagina 5, lettura pagina 1, lettura pagina 3, lettura pagina 4, lettura pagina 2, scrittura pagina 2, lettura pagina 5. Utilizzare la terminologia appropriata.



Esercizio 2.

Si consideri un sistema di virtualizzazione della memoria basato su paginazione con algoritmo FIFO e dirty bit. Si assuma che inizialmente sia caricata solo pagina 3 (su cui sono stati già fatti accessi in scrittura), e che la memoria principale sia di 3 blocchi. Si descriva (a parole e diagramma) cosa succede se vengono fatti i seguenti accessi: lettura pagina 1, lettura pagina 5, scrittura pagina 1, lettura pagina 3, lettura pagina 7, scrittura pagina 9, scrittura pagina 7. Utilizzare la terminologia appropriata.



Frammentazione interna

Supponiamo che ad un programma serva una quantità di memoria che non sia un multiplo della dimensione delle pagine: l'ultima pagina non verrà completamente utilizzata.

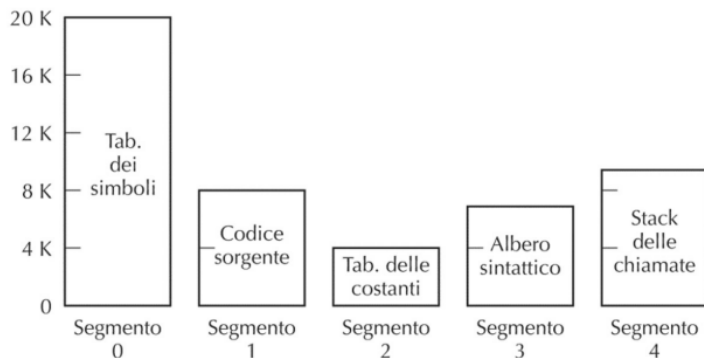
Esempio:

- Consideriamo un programma che richiede 26.000 byte
- In esecuzione su una macchina con pagine da 4KB (4096 byte)
- Il programma userà in modo completo 6 pagine ($6 \times 4.096 = 24.576$)
- Nell'ultima pagina, la settima, si useranno solamente $26.000 - 24.576 = 1.424$ byte
- I restanti $4.096 - 1.424 = 2.672$ byte non verranno utilizzati

Questo fenomeno viene chiamato frammentazione interna. Esiste anche la frammentazione esterna (che richiede però di introdurre prima la nozione di segmento).

Segmentazione

Finora abbiamo pensato a una memoria (virtuale o reale) di k locazioni come a un unico spazio di indirizzamento con indirizzi da 0 a $k-1$. Tuttavia una memoria è comunemente organizzata logicamente in porzioni distinte utilizzate con distinti scopi/modalità, diritti di accesso, protezioni in scrittura, etc, detti segmenti. Ciascun segmento ha il proprio spazio di indirizzamento. E' simile ad avere disponibili contemporaneamente diverse memorie. Esempio relativo a un compilatore:



Nota: dimensione non uguale né costante

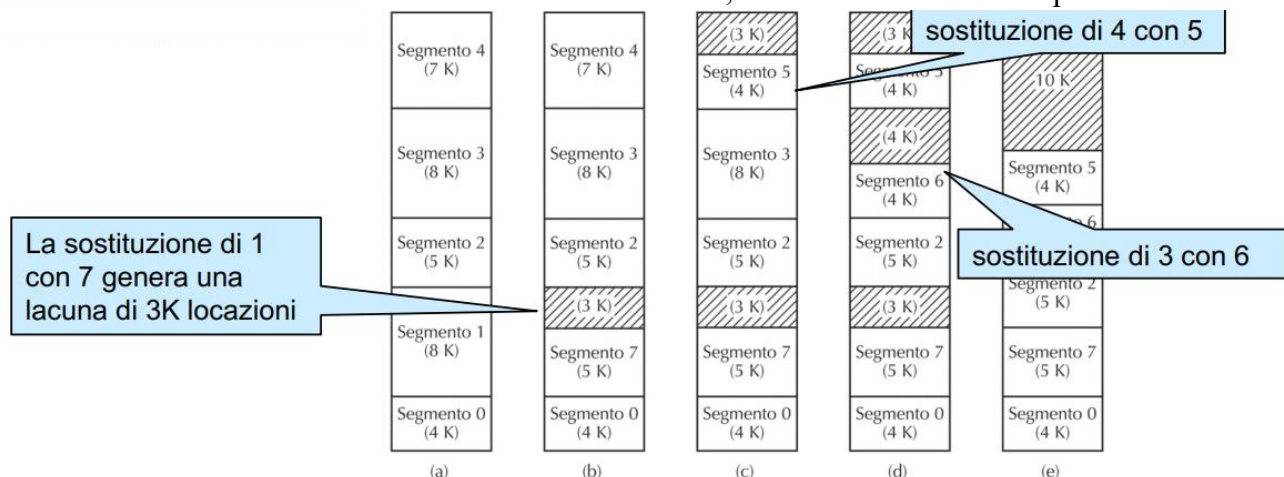
Figura 6.8 Una memoria segmentata consente a ciascuna tabella di crescere o decrescere indipendentemente dalle altre.

I segmenti sono visibili anche al programma (al codice assembly). Possiamo quindi in assembly scrivere/leggere ad un certo indirizzo di un certo segmento, usare indirizzamento indicizzato (cioè con offset) nel segmento, etc. Esempio: segmenti usati nell'architettura x86: code segment, data segment, stack segment, extra segment. A differenza della paginazione, la segmentazione non è trasparente al programma.

- Ogni segmento di dimensione n usa gli indirizzi da 0 a $n-1$
- Per disambiguare l'uso di indirizzi presenti in più segmenti (ad esempio l'indirizzo 0 è presente in tutti i segmenti!) gli indirizzi dovranno essere arricchiti con l'indicazione del relativo segmento
- L'indirizzo (n,k) indica la locazione di indirizzo k del segmento n

Come nella paginazione, possiamo virtualizzare la memoria e tenere in memoria solo alcuni segmenti al momento utilizzati, lasciando in memoria secondaria i segmenti non necessari.

- Quando serve inserire un nuovo segmento se ne toglie un altro, tramite swapping (sostituzione) di segmenti.
- A seguito di una sostituzione si possono generare zone non utilizzate dette lacune. Questo fenomeno viene detto "frammentazione esterna", da risolvere tramite compattamento.



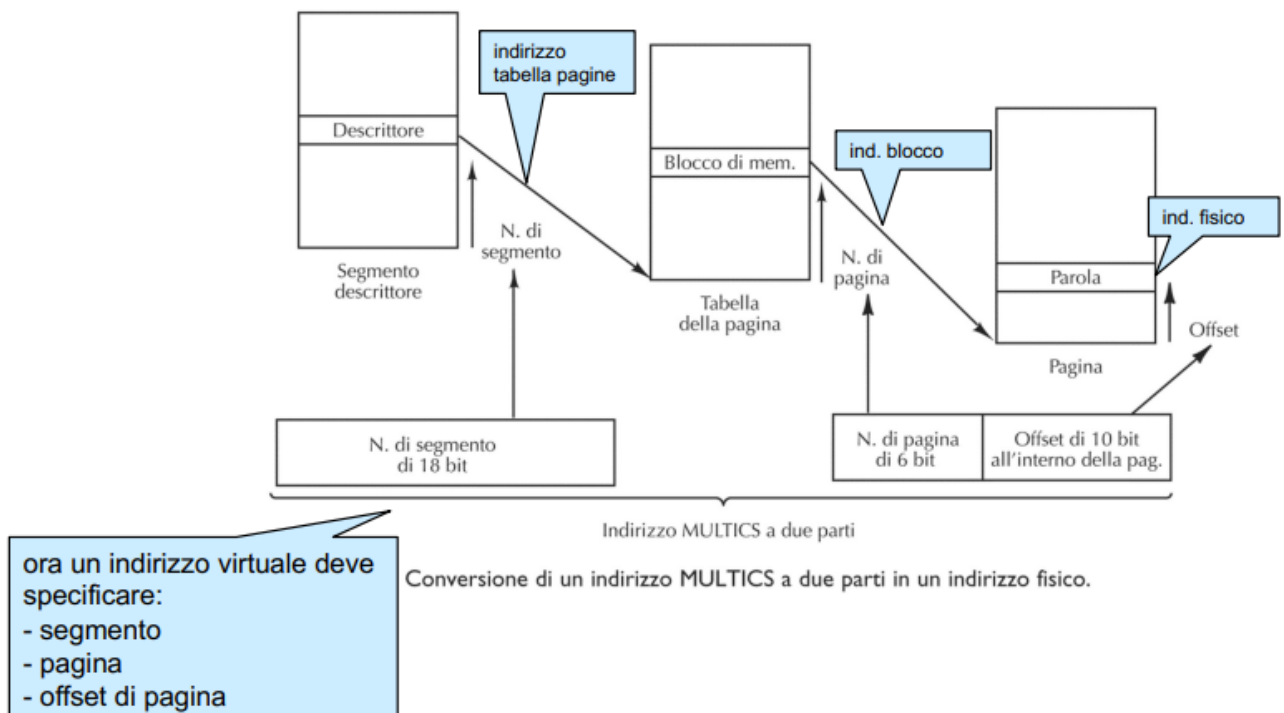
Si sono ideate varie tecniche di sostituzione di segmenti per cercare di ridurre la frammentazione esterna. Tra le tecniche più usate menzioniamo:

- Best fit: si inserisce il nuovo segmento nella lacuna più piccola sufficiente per il segmento da inserire;
- First fit: si scorrono le lacune circolarmente selezionando la prima sufficientemente grande per il segmento da inserire. Nel caso medio, First fit è più veloce nella scelta, ed inoltre limita la creazione di piccole e inutili lacune.

Un modo per evitare la frammentazione esterna, oltre al compattamento, è combinare segmentazione e paginazione. Quindi virtualizzare la memoria tramite paginazione, mantenendo in aggiunta la divisione logica offerta dai segmenti.

Intuizione:

- La memoria virtuale è paginata e segmentata. Quindi è divisa logicamente in segmenti, ed ogni segmento è diviso logicamente in pagine.
- Solo le pagine necessarie (accedute dal programma) vengono tenute in memoria principale esattamente come visto prima (cf. page swapping)
- Quindi si riduce la porzione di ciascun segmento che effettivamente si tiene in memoria, ma soprattutto visto che le pagine sono sempre di dimensione uguale ai blocchi, si evita la frammentazione esterna: si mantengono in memoria sempre multipli della dimensione dei blocchi
- C'è però più lavoro per la MMU per tradurre gli indirizzi, che deve utilizzare 2 tabelle:
 - 1 tabella dei segmenti
 - 1 tabella delle pagine per ogni segmento



Linker

Come viene preparato un programma per essere eseguito:

- Un programma può essere composto da unità distinte
- Ognuna di tali procedure può essere compilata separatamente per generare i cosiddetti “moduli oggetto”
- Per poter avere un programma pronto per l’esecuzione si rende necessaria una fase detta di “collegamento” fatta dal linker.

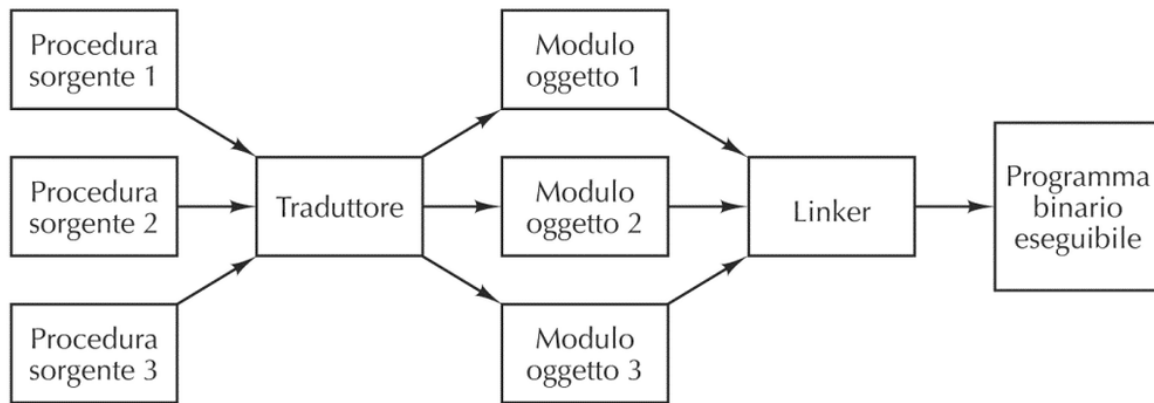
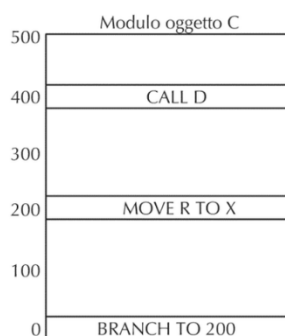
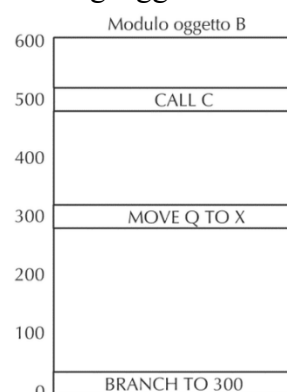
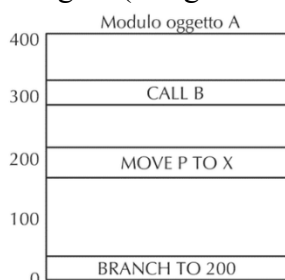


Figura 7.12 Generazione tramite un linker di un programma eseguibile binario a partire da un gruppo di file sorgente tradotti indipendentemente.

Ogni modulo oggetto (come i file .o che abbiamo visto) viene realizzato assumendo di usare un proprio spazio di indirizzi che inizia da 0. Consideriamo, per esempio, 4 moduli oggetto da collegare (in figura si assume un generico linguaggio assembly).



BRANCH: salto

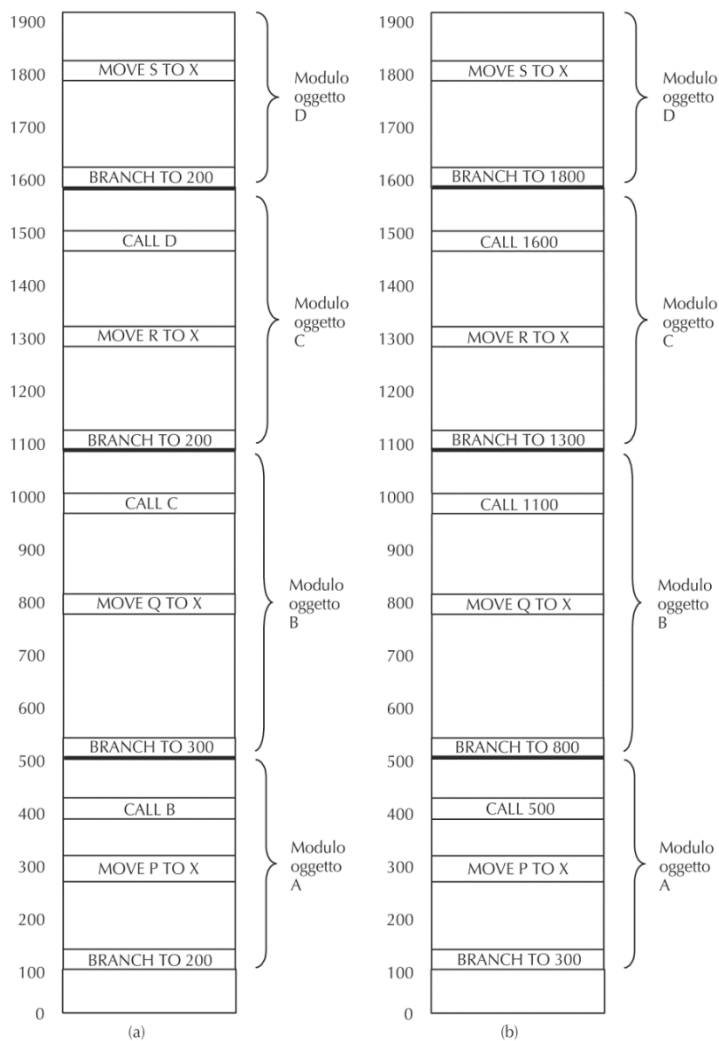
MOVE: sosta dati da registri a memoria e viceversa

CALL invoca una procedura (che può essere implementata in un altro modulo oggetto)

Gli indirizzi usati in (come operandi di) queste istruzioni, fanno riferimento allo spazio di indirizzamento del proprio modulo: dobbiamo convertire gli indirizzi in indirizzi globali (indirizzi di memoria).

Figura 7.13 Ogni modulo ha il proprio spazio d’indirizzi che inizia da 0.

Il linker decide l’ordine in cui collegare i moduli. Inizialmente, si dispongono sequenzialmente. All’inizio inserisce informazioni di servizio necessarie (**) per eseguire il programma. Per ogni modulo collegato si tiene traccia della sua lunghezza (l) e dell’indirizzo di inizio nel nuovo spazio di indirizzamento (i):



Rilocazione:

Si rende necessario modificare gli indirizzi presenti all'interno dei moduli oggetto

- Parte facile: sistemare riferimenti interni (cioè al medesimo modulo) incrementandolo del valore dell'indirizzo di inizio del modulo.
- Occorre tener traccia di quali istruzioni contengono riferimenti e quindi hanno bisogno di rilocazione.
- Si usa il dizionario di rilocazione.

Per gestire i riferimenti esterni servono più informazioni.

Esempio: il modulo C deve dire a che indirizzo interno corrisponde il simbolo "C" che "esporta", cioè rende invocabile da altri moduli. Ed il modulo B deve indicare al linker il fatto che ha bisogno del simbolo esterno "C" esportato da C.

Grazie a queste informazioni, più la tabella con gli indirizzi di inizio dei vari moduli discussa in precedenza, il linker riesce a rilocare correttamente anche i riferimenti esterni. Se il primo modulo non è caricato in memoria a partire da 0 serve una ulteriore rilocazione.

Fine del modulo
Dizionario di rilocazione
Istruzioni macchina e costanti
Tabella dei riferimenti esterni
Tabella dei punti di ingresso
Identificativo

Identificativo: nome del file o informazioni relative alla gestione dello stesso

Tabella dei punti di ingresso: tabella che contiene informazioni relative a chiamate esterne e dove la chiamate all'interno del file deve essere direzionata

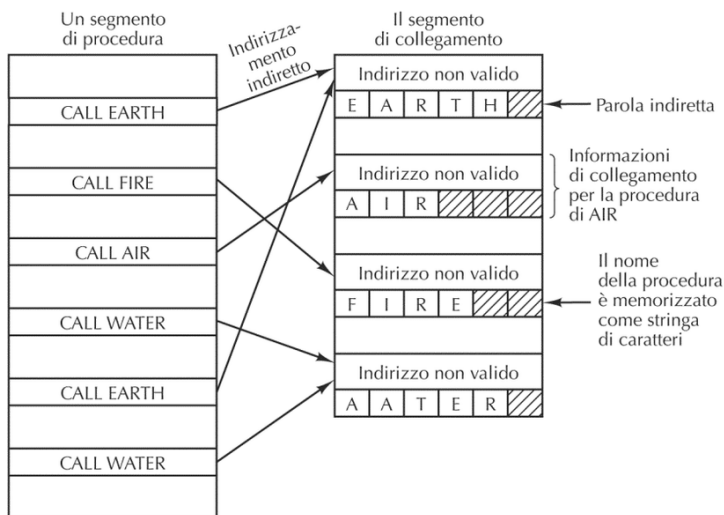
Tabella dei riferimenti esterni: tabella che indica, per ogni istruzione di chiamata esterna al file, dove si trova l'oggetto della chiamata

Istruzioni macchina e costanti: effettivo codice del file

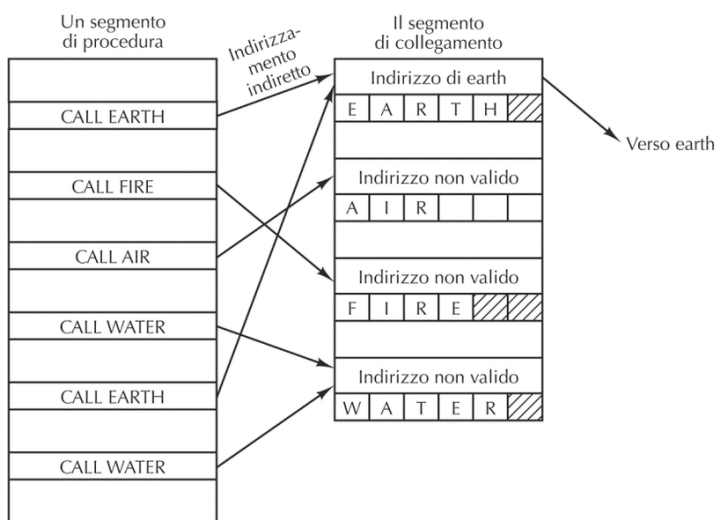
Dizionario di rilocazione: indica quali istruzioni del campo precedente necessitano di salti o hanno bisogno di rilocazione

Collegamento dinamico:

In alcuni casi, le procedure che vengono usate non vengono inserite nell'eseguibile: vengono collegate dinamicamente al momento dell'esecuzione del programma stesso. Questo può essere utile per procedure che vengono invocate solo in casi rari, o che è lecito assumere siano disponibili/installate nel sistema. Per esempio una libreria legata alla scheda grafica. Un modo per implementare il collegamento dinamico è il seguente:



(a)



(b)

- Si compila la chiamata alla procedura "dinamica" tramite una chiamata ad un indirizzo volutamente "sbagliato", per generare un trap;
- Nel programma compilato si inserisce il nome della procedura che si desidera invocare tramite link dinamico;
- La prima volta che si arriva alla chiamata "sbagliata", il gestore del trap cerca tramite il nome appropriatamente inserito nel codice, l'indirizzo in cui si trova la procedura, e sostituisce nel codice il salto a quell'indirizzo. Se si arriva alla prima istruzione di tipo "indirizzo non valido" si genera un trap. Il gestore:
 - legge la stringa "EARTH" e cerca l'attuale posizione della procedura "EARTH"
 - Sostituisce "indirizzo non valido" con l'indirizzo valido di "EARTH" ed esegue la call

In questo modo, future call non genereranno più trap.

Il collegamento dinamico viene usato per tutta una serie di procedure che solitamente fanno parte delle cosiddette "librerie dinamiche". Ad esempio, i sistemi operativi Windows usa le cosiddette DLL (Dynamic Link Library) per procedure che possono essere usate da tanti programmi diversi (ad esempio i driver per accedere ai dispositivi di I/O). Sarebbe infatti inutile inserirle nell'eseguibile di tutti i programmi che le usano, ma si assume siano installate una volta per tutte nel sistema. Inoltre, in caso di modifica/aggiornamento di tali librerie, sarebbe necessario andare a modificare tutti gli eseguibili in cui queste sono già state inserite.

Nota: Quanto visto su linking e rilocazione non ha a che vedere con la virtualizzazione della memoria. Sarebbe uguale anche con uso di sola memoria fisica e indirizzi fisici.

Virtual Machine

Una virtual machine (da non confondersi con i software di macchine virtuali che permettono - ad esempio - di usare linux dentro una finestra di windows) è un modello astratto di calcolo, a differenza dell'assembly non fa riferimento a nessuna specifica architettura e si pone come step intermedio fra linguaggi di alto e basso livello, molto usata nella compilazione a due livelli.

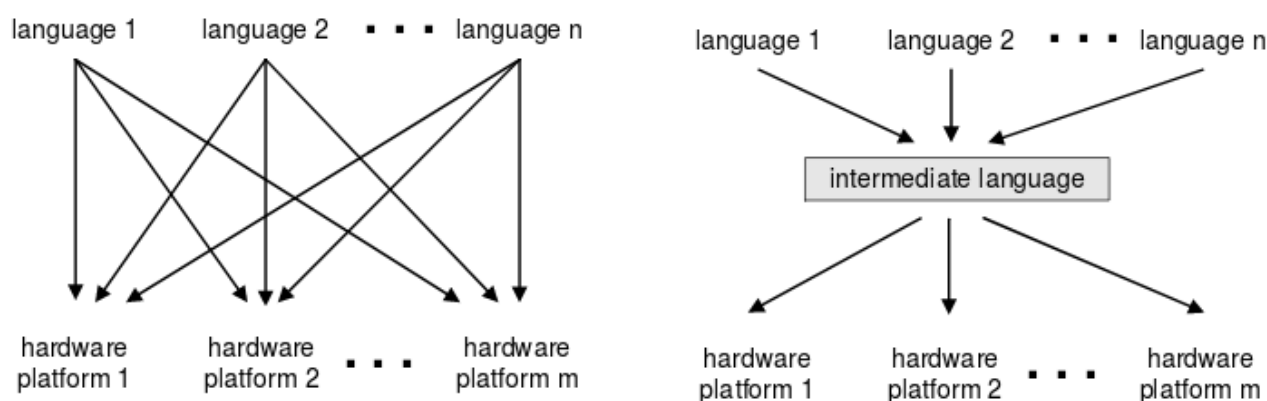
Un esempio comune di VM è la Java Virtual Machine (JVM), infatti il linguaggio Java viene compilato non in binario ma in un linguaggio intermedio, chiamato bytecode, e questo viene eseguito sui vari computer utilizzando la JVM (ecco perché è necessario installare Java per usare i relativi programmi).

Modelli di compilazione

Ci sono due modelli di compilazione: compilazione diretta e compilazione a due livelli.

Nella compilazione diretta ogni programma viene compilato direttamente dal codice sorgente al codice binario. Occorre quindi compilare più volte il programma (per ogni architettura e SO per il quale è destinato) e bisogna tenere conto sia dei dettagli del linguaggio sorgente che del linguaggio target.

Nella compilazione a due livelli invece si compila prima il sorgente in un linguaggio intermedio, in tal modo bisogna tener conto solo delle caratteristiche del linguaggio sorgente, e poi in un secondo momento si compila il linguaggio intermedio per le relative piattaforme, in modo da dover tener conto solo delle caratteristiche del linguaggio target.



Nell'immagine sono riportati due schemi raffiguranti (rispettivamente a sinistra e a destra) la procedura della compilazione diretta e quella della compilazione a due livelli.

- Primo livello: dipende solo dai dettagli del linguaggio sorgente
- Secondo livello: dipende solo dai dettagli del linguaggio target

Nel nostro caso, il livello intermedio è costituito dalla Virtual Machine Hack la quale, come ogni virtual machine, è associata con un proprio linguaggio.

Programmi in tale linguaggio sono il risultato del primo livello di compilazione. Occorre compilare i programmi in questo linguaggio per VM per ottenere codice ISA (programmi in assembly simbolico o codice macchina).

Il linguaggio della virtual machine è basato sui seguenti comandi:

Arithmetic / Boolean commands

add

sub

neg

eq

gt

lt

and

or

not

Memory access commands

pop x (pop into x, which is a variable)

push y (y being a variable or a constant)

Program flow commands

label (declaration)

goto (label)

if-goto (label)

Function calling commands

function (declaration)

call (a function)

return (from a function)

Come procediamo:

Guardiamo uno ad uno i possibili tipi di istruzioni, definendone la semantica sul modello di macchina virtuale Hack (cioè definendo come manipolano lo stack e la memoria)

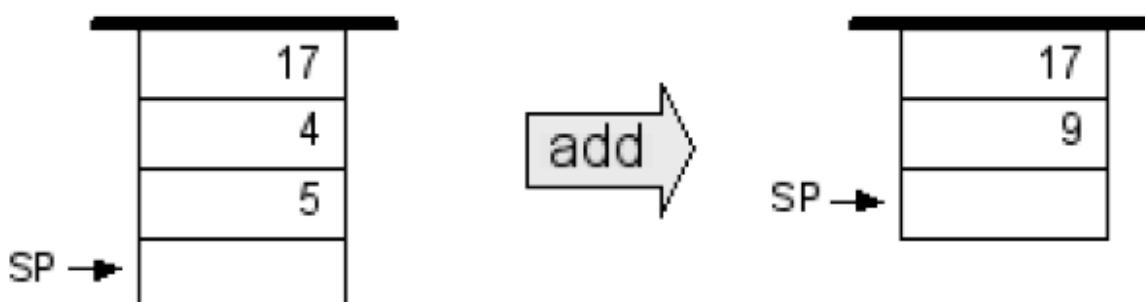
Poi vedremo esempi gradualmente di come possiamo scrivere programmi in questo linguaggio per VM Hack

Infine vediamo alcuni dettagli relativi alla traduzione di un programma per VM Hack in assembly Hack. La difficoltà di questa traduzione/compilazione è che il programma ISA dovrà occuparsi esplicitamente della gestione della memoria, che invece al livello della VM viene dato come comportamento automatico della macchina virtuale (perché il suo funzionamento è definito in astratto).

Modello a stack

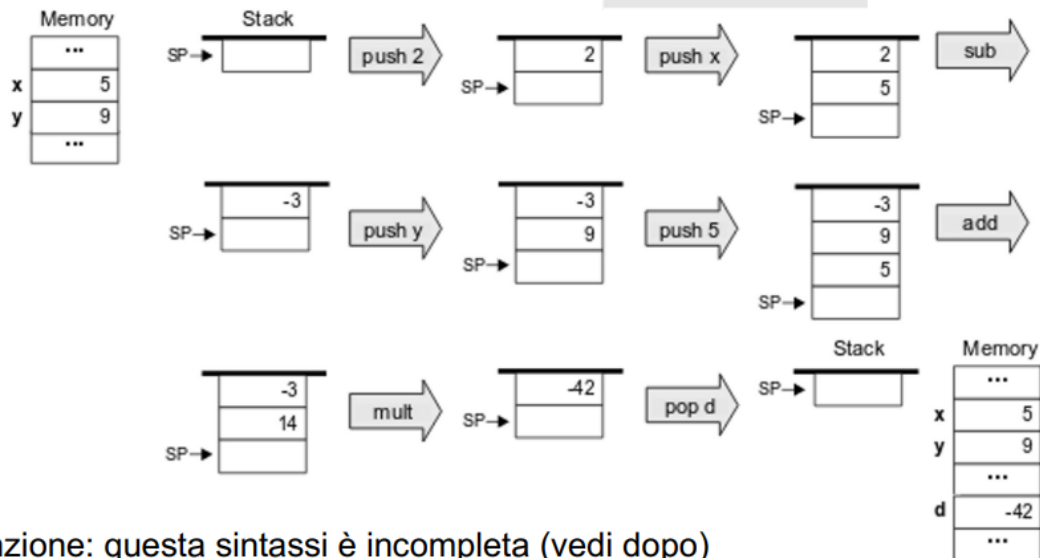
La VM Hack opera seguendo un modello a stack, per ciascuna operazione la VM fa:

- push dei valori da utilizzare nello stack;
- pop dei valori dalla cima dello stack;
- operazioni aritmetico-logiche unarie e binarie tramite funzioni $f(x,y)$, facendo pop dei valori necessari x , y dallo stack e successivamente facendo push del risultando, con l'effetto di rimpiazzare gli operandi in cima allo stack con il risultato.



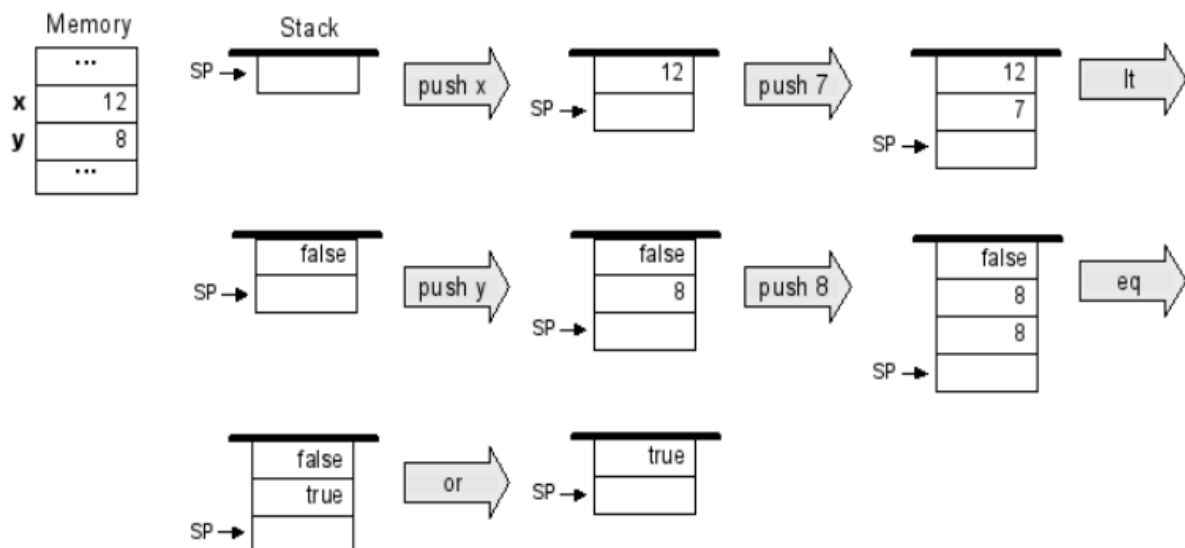
Per comodità assumiamo l'esistenza di una operazione "mult" che nel nostro caso non è presente (definiremo più avanti in un esempio una funzione "mult" per la moltiplicazione)

```
// d=(2-x)*(y+5)
push 2
push x
sub
push y
push 5
add
mult
pop d
```



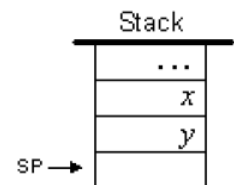
Attenzione: questa sintassi è incompleta (vedi dopo)

```
// if (x<7) or (y=8)
push x
push 7
lt
push y
push 8
eq
or
```



Sommario delle operazioni aritmetico-logiche

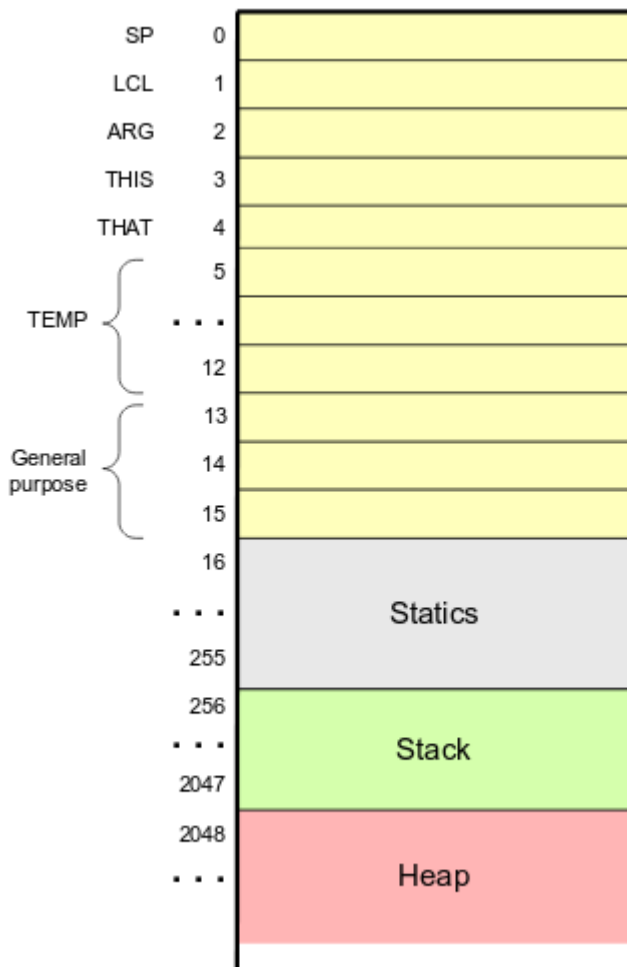
Command	Return value (after popping the operand/s)	Comment
add	$x + y$	Integer addition (2's complement)
sub	$x - y$	Integer subtraction (2's complement)
neg	$-y$	Arithmetic negation (2's complement)
eq	true if $x = y$ and false otherwise	Equality
gt	true if $x > y$ and false otherwise	Greater than
lt	true if $x < y$ and false otherwise	Less than
and	$x \text{ And } y$	Bit-wise
or	$x \text{ Or } y$	Bit-wise
not	$\text{Not } y$	Bit-wise



And, Or e Not, quando applicati a valori booleani, rappresentano l'and / or / negazione logici: true è rappresentato come -1 (in complemento a 2: 16 '1') e false come 0.

Memoria

La memoria della VM Hack è segmentata. Similmente a quanto studiato in generale, l'adozione di segmenti distinti permette di dividere logicamente la memoria a seconda di scopi, caratteristiche, dimensioni, ruoli, etc. In questo caso, ogni segmento si contraddistingue per il tipo di dato lì memorizzato: argomenti e variabili locali (nel caso di chiamate a funzione), costanti, variabili globali, etc. La sintassi intuitiva utilizzata finora va quindi estesa, menzionando esplicitamente i segmenti su cui operiamo. (cf. quanto detto nella teoria: la segmentazione della memoria non è trasparente ai programmi)



Dal punto di vista “fisico” la memoria verrà organizzata nel seguente modo:

- Gli indirizzi da 0 a 15 non sono parte di specifici segmenti, ma sono usati a livello di implementazione della virtual machine
- In particolare, la VM usa SP (Stack Pointer), LCL (Local Segment Pointer), ARG (Argument Segment Pointer)
- Gli indirizzi da 16 a 255 sono usati per il segmento "static"
L'accesso a questo segmento si mappa sulla gestione dei simboli a livello assembler
- Gli indirizzi da 256 a 2047 sono usati per lo stack (operazioni push e pop) e per i segmenti "argument" e "local" creati al momento dell'invocazione di funzioni (SP inizializzato a 256)
- Segmenti di memoria

Per la VM si utilizzano diversi segmenti di memoria (parti della memoria con connotazione più o meno uniforme), ovvero spazi di indirizzamento virtuali usati per scopi diversi con dimensioni e caratteristiche differenti. Ogni funzione in esecuzione avrà due propri segmenti specifici che vengono creati e distrutti prima e dopo l'esecuzione:

- **"argument"** per gli argomenti (parametri di invocazione)
- **"local"** per le proprie variabili locali

Le funzioni, inoltre, condividono un segmento **"static"** contenente le variabili globali e si assume che esista un segmento "fittizio" chiamato **"constant"**, da cui si può solo leggere e che contiene costanti (cioè un segmento dove all'indirizzo 0 è scritto '0', all'indirizzo 1 è scritto '1', etc)

Accesso alla memoria:

accesso di un segmento in memoria:

Considerando i segmenti di memoria static, local, argument, constant, per accedere ad essi si usano i comandi:

<code>pop segment addr</code>	<i>// pop dalla cima dello stack, salvando il valore in (segment,addr)</i>
<code>push segment addr</code>	<i>// push in cima dello stack del valore all'indirizzo (segment,addr)</i>

Al posto di segment si mette static, local, argument, oppure constant, al posto di addr si mette:

- L'indirizzo di interesse (per local e argument)
- La costante numerica di interesse (per constant)
- Per static si usa un valore numerico n che viene tradotto nel simbolo nomeFile.n assumendo che il file con il programma si chiami nomeFile.vm

Esempio:

```
push local 0      // push sullo stack di local[0]
push constant 1   // push sullo stack di constant[1] (quindi costante 1)
sub              // sottrazione
pop local 0       // pop del valore in cima nella locazione local[0]
```

Esercizi e teoria

Si scriva codice per la VM Hack che calcola il valore della seguente espressione. Notare che non ci serve alcun segmento a parte constant (che è solo di lettura, e non va inizializzato).

$((5-1) + (4+2)) > (7+2)$

```
push constant 5
push constant 1
sub
push constant 4
push constant 2
add
add
push constant 7
push constant 2
add
gt
```

Si scriva codice per la VM Hack che calcola il valore della seguente espressione booleana, assumendo che i segmenti siano già inizializzati e che le variabili x e y siano salvate in argument[0] e argument[1] (cioè che siano gli argomenti di una chiamata a funzione f(x,y) il cui corpo stiamo ora implementando... vedi dopo).

$x = ((y-1) > (x+2)) \text{ OR } (x == 7)$

```
push argument 1
push constant 1
sub
push argument 0
push constant 2
add
gt // ora in cima abbiamo un boolean
push argument 0
push constant 7
eq
or
pop argument 0 // aggiorniamo il valore dell'argomento x in
argument[0]
```

Comandi per il controllo del flusso

Per implementare costrutti del flusso di controllo (if-then-else, while, etc) procediamo come in assembly:

- label NOME: dichiara una etichetta NOME
- goto NOME: jump incondizionato all'etichetta NOME
- if-goto NOME: jump condizionato: si effettua il salto se true (in binario in complemento a due: -1) è in cima allo stack. Esegue anche un pop!

esempio: `if local[0]>3 then local[0]=0 else ...`

`push local 0`

`push constant 3`

`gt`

`not // la valutazione della condizione negata è in cima allo stack`

`if-goto ELSE // salto condizionato push constant 0 pop local 0`

`label ELSE`

Convenzione di chiamata:

- La funzione chiamante (o chiamante) fa push degli argomenti, poi fa la “call” del chiamato e aspetta il “return”
- Prima di terminare, il chiamato deve fare push del valore di ritorno
- Al momento della “return” le risorse usate dal chiamato vengono rilasciate ed il chiamante è ripristinato nel medesimo stato in cui era al momento della “call”

Effetto netto: il chiamato consuma gli argomenti e lascia il valore di ritorno in cima allo stack esattamente come succede nelle operazioni di base (add, or,..)

Per le chiamate si utilizzano questi comandi:

- **function nomefunzione numVarLocali**

Inizio della definizione della funzione. Nota: deve specificare il numero di variabili locali di cui ha bisogno (così che queste vengano allocate in local);

- **call f numArgomenti**

Invoca la funzione f. Nota: specifica quanti argomenti sono attesi, così che venga fatto il numero giusto di operazioni pop, e si allochi anche la memoria necessaria nel segmento argument;

- **Return**

Istruzione di ritorno. Il risultato della funzione è già stato inserito in cima allo stack, quindi il controllo può tornare al chiamante.

Esercizio 1

```
int myfun(int x) {
    z = x+1
    if(z>=5) {
        return(0);
    } else {
        return(z);
    }
}
```

```
function myfun 1 // 1 variabile locale
push argument 0 // x è salvata in argument[0]
```

```

push constant 1
add
pop local 0 //z è una variabile locale, la salviamo in local[0]
push local 0
push constant 5
lt
if-goto ELSE
push constant 0
return
label ELSE
push local 0
return

```

Esercizio 2.

```

function dummy (x,y){
    int z;
    z=0;
    if (x>y) {
        return x
    } else {
        z = y + 3;
        return z>x;
    }
}

function dummy 1
    push constant 0
    pop local 0 // z= 0;
    push argument 0
    push argument 1
    gt
    not
    if-goto ELSE
    push argument 0
    return
label ELSE
    push argument 1
    push constant 3
    add
    pop local 0 // z=y+3
    push local 0
    push argument 0
    gt
    return

```

Esercizio 3

```

int fun (int a){
    a+=2;
}

```

```

        do{
            a+=4;
        }
        while (a<15)
            return (-a)
    }
function fun 0
push constant 2
push argument 0
add
pop argument 0
label WHILE // inizia corpo del while
push constant 4
push argument 0
add
pop argument 0
push argument 0 // inizia test while push constant 15
lt
if-goto WHILE
push argument 0
neg
return

```

Esercizio 4

```

function mult(x,y) {
    int sum, j;
    sum = 0;
    j = y;
    while (j != 0) {
        sum = sum + x;
        j = j - 1;
    }
    return sum;
}

```

```

function mult 2
    push constant 0
    pop local 0
    push argument 1
    pop local 1
label WHILE
    push constant 0
    push local 1
    eq
    if-goto END
    push local 0
    push argument 0
    add

```

```

    pop local 0
    push local 1
    push constant 1
    sub
    pop local 1
    goto WHILE
label END
    push local 0
    return

```

Esercizio 5

```

int f(int x, int y) {
    int z;
    z = 3;
    while(x>0) {
        y+=z;
        x-=2;
    }
    return x;
}

```

```

function f 1
    push constant 3
    pop local 0
label WHILE
    push argument 0
    push constant 0
    gt
    not
    if-goto END
    push local 0
    push argument 1
    add
    pop argument 1
    push argument 0
    push constant 2
    sub
    pop argument 0
    goto WHILE
label END
    push argument 0
    return

```

Esercizio 6

```

int f(int a, int b, int c) {
    int x, y;
    x = a + c;
}

```

```

    if(b < x)
        y = x*2;
    else
        y = a+b+c;
    return y;
}

function f 2
    push argument 0
    push argument 2
    add
    pop local 0
    push argument 1
    push local 0
    lt
    not
    if-goto ELSE
    push local 0
    push constant 1
    call mult 2 OPPURE
    push local 0;
    push local 0;
    add
    pop local 1
    goto END
label ELSE
    push argument 0
    push argument 1
    add
    push argument 2
    add
    pop local 1
label END
    push local 1
    return

```

Esercizio 7

```

int myfunction(int x) {
    int sum=100;
    int c=1;
    while(x<=sum) {
        x=x+(4-(c++));
        sum=sum-2;
    }
    return((x-sum)>10);
}

```

```
function myfunction 2
    push constant 100
    pop local 0
    push constant 1
    pop local 1
label WHILE
    push argument 0
    push local 0
    gt
    if-goto END
label CORPO
    push argument 0
    push constant 4
    push local 1
    push constant 1
    add
    pop local 1
    sub
    add
    pop argument 0
    push local 0
    push constant 2
    sub
    pop local 0
    goto WHILE
label END
    push argument 0
    push local 0
    sub
    push constant 10
    gt
    return
```

Simulazione d'esame